DRAFT

Semantic Governance

Services, Architecture & Integration

Version 1.7 · Draft · October 13, 2026

PART I **Semantic Governance Services**
Why semantic governance matters and how it governs agentic AI; the operating model, inbound services, deterministic knowledge, demand management, the neuro-symbolic foundation, and context engineering.

PART II **Semantic Governance Architecture & Integration**
The MVP1 solution architecture (Federated RAG 2.0), industry alignment, process flows, features, vector database design, and the reusable inbound microservices, agents, and search facade.

PART III **Knowledge Engineering**
Design principles, naming and URI standards, change control, governance against ISO and IEEE standards, the single source of truth for rules and logic, the target operating model, the term definition standard, labels and personas, the taxonomies of relations and attributes, and BFO and RDF foundations.

APPENDIX A **Finance (FP&A) Domain**
An illustrative, end-to-end example of the framework applied to Financial Planning & Analysis: domain model and ontology, requirements and change management, LKIF mappings, vector databases, sequence diagrams, use cases, and MCP integration.

APPENDIX B **Conversational AI**
A design and reference architecture for conversational AI that lets subject-matter experts talk to their data: components, layers, phased delivery plan, KPIs, and the context engineering module.

APPENDIX C **Simulation and Causal Reasoning**
Emerging techniques that extend context engineering: simulation-driven context with the Simulator, Reflector, and Curator pattern, and causal reasoning over ontologies with CausalGraph2LLM.

APPENDIX D **Statement of Work (MVP1 Prep)**
The Statement of Work for semantic services development and the governance framework: scope, deliverables, timeline, team, assumptions, pricing, and acceptance criteria.

APPENDIX E **Applying Semantic Standards Using RDF Studio**
A stepwise guide to applying the Part III naming, definition, label, taxonomy, mapping, and change control standards when modeling in RDF Studio.

APPENDIX F **Terms & Acronyms**
Definitions of the acronyms and specialist terms used throughout this document.

Contents

PART I Semantic Governance Services

1.1 Overview
1.2 Value and Agentic Governance
1.3 Semantic Governance Operating Model
1.4 Inbound Semantic Services Design
1.5 Deterministic Knowledge in the Knowledge Fabric
1.6 Demand Management and Agility
1.7 Neuro-Symbolic Foundation for AI
1.8 Context Engineering

PART II Semantic Governance Architecture & Integration

2.1 Solution Architecture
2.2 Industry Alignment
2.3 Primary Process Flows
2.4 Feature and Functional Breakdown
2.5 Vector Database Design
2.6 Inbound Microservices, Agents, and Search

PART III Knowledge Engineering

3.1 Design
3.2 Standards
3.3 Change Control
3.4 Governance
3.5 Single Source of Truth
3.6 Target Operating Model
3.7 Term Definition Standard
3.8 Labels and Personas
3.9 Attributes, Properties and Relations
3.10 BFO and RDF Foundations

APPENDIX A Finance (FP&A) Domain

A.1 Finance Domain
A.2 FP&A Ontology
A.3 FP&A Ontology: Process-Centric Model
A.4 Ontology Requirements Specification (ORSD)
A.5 KGCL-to-ORSD Mapping
A.6 KGCL-to-ORSD Mapping (Detailed)
A.7 Mapping FP&A Metrics to LKIF Statements
A.8 Mapping FP&A Classes to LKIF Statements
A.9 Scenario Mapping in Vector Databases
A.10 Vector Database Integration with Ontologies
A.11 FP&A Sequence Diagrams
A.12 FP&A Use Cases
A.13 MCP and Microservices Integration

APPENDIX B Conversational AI

B.1 Conversational AI Design
B.2 Conversational AI Architecture

APPENDIX C Simulation and Causal Reasoning

C.1 Simulation for Context Engineering
C.2 Causal Reasoning with CausalGraph2LLM

APPENDIX D Statement of Work (MVP1 Prep)

D.1 Statement of Work (MVP1 Prep)

APPENDIX E Applying Semantic Standards Using RDF Studio

E.1 Purpose, Roles, and Setup
E.2 Stepwise Guide
E.3 Checklists and Tool Behaviors

APPENDIX F Terms & Acronyms

F.1 Terms & Acronyms

Document Control

Item	Value
Title	Semantic Governance: Services, Architecture & Integration
Version	1.7
Status	Draft
Date	October 13, 2026
Versioning	MAJOR.MINOR. A minor version adds or revises content within the existing structure; a major version changes the structure or scope, or marks formal approval.

Revision History

Version	Date	Status	Changes
1.0	October 2026	Draft	Combined reference: Part I Services, Part II Architecture & Integration, Part III Knowledge Engineering (Sections 3.1–3.6), Appendices A–E; review feedback applied.
1.1	October 13, 2026	Draft	Added versioning and document control. Part III: added Section 3.7 Term Definition Standard, 3.8 Labels and Personas, 3.9 Attributes, Properties and Relations, 3.10 BFO and RDF Foundations. Appendix E: added IAO, Persona, RDF 1.2, RO, SKOS-XL.
1.2	October 13, 2026	Draft	Section 3.3 Change Control: added the change log model (change records in a dedicated append-only graph, added and removed triples per change set, reporting and lineage), the rule for versions in identifiers, and the release pipeline (build once, promote one package through development, test, stage, and production, deployment records, safe loading and rollback, drift prevention). Section 3.6: traceability chain extended to releases and deployments.
1.3	October 13, 2026	Draft	Added Section 2.6 Inbound Microservices, Agents, and Search: Kafka-based microservice catalog for the Section 1.4 inbound flow, naming conventions, standard service contract and logging, agent catalog and guardrails, exception handling and resiliency patterns, and search as a Semantic Knowledge Layer utility with a facade of microservices and federated-query agents. Cross-reference added to Section 1.4; Appendix E updated.
1.4	October 13, 2026	Draft	Section 1.4: replaced the step-by-step inbound flow listing with a six-stage diagram showing steps, microservices, the Kafka event backbone, and the agents.
1.5	October 13, 2026	Draft	Section 2.1: replaced the semantic governance and homogeneity diagram with a Keel and North Star diagram, establishing it as the foundational architecture, and added a table mapping the section's layers to the vessel layers.
1.6	October 13, 2026	Draft	Section 1.5: added Developing the Knowledge Fabric with an Agentic Fabric (component design, federated star topology, development agents, development loop, and diagram) and Alignment with Keel and North Star. Added Appendix F, Applying Semantic Standards Using RDF Studio, a stepwise guide. Appendix E updated.

Version	Date	Status	Changes
1.7	October 13, 2026	Draft	Moved Terms & Acronyms to the last appendix (now Appendix F); Applying Semantic Standards Using RDF Studio is now Appendix E. Cross-references updated; entries above refer to the appendix letters in use at the time.

Overview

This reference brings together the Company's semantic governance work. Semantic governance harmonizes meanings, ontologies, and taxonomies across the enterprise so that people, analytics, and AI share one machine-readable business language, turning knowledge into a governed asset that grounds AI and makes cause and effect traceable.

Architecture at a glance (detail in Section 2.1)

- **Knowledge fabric:** core and domain ontologies authored in the Semantic Modeling Workbench and stored in GraphDB.
- **Operations Workbench:** a low-code layer for monitoring, visualization, impact analysis, and human-in-the-loop review.
- **Intelligence layer:** domain-aligned vector databases for semantic search and AI agents that map, enrich, and reason.
- **Integration backbone:** microservices and the Model Context Protocol (MCP) for secure, standard communication.
- **Governance for AI:** policies expressed in the semantic layer, with SHACL validation, registries, and monitoring.

How this document is organized

Part I — Semantic Governance Services

Sections 1.1–1.8

Value and agentic governance, the operating model, inbound services, deterministic knowledge, demand management, the neuro-symbolic foundation, and context engineering.

Part II — Semantic Governance Architecture & Integration

Sections 2.1–2.6

The MVP1 solution architecture (Federated RAG 2.0), industry alignment, process flows, features, and vector database design.

Part III — Knowledge Engineering

Sections 3.1–3.10

How knowledge is designed, named, changed, governed, and operated: the process-step-centric ontology design, naming and URI standards, change control, governance, the single source of truth for rules and calculations, the target operating model, and the detailed modeling standards for term definitions, labels and personas, relations, and attributes.

Appendices — Applying the Framework

Appendix A is an illustrative, end-to-end example of the framework applied to Financial Planning & Analysis, the pattern other domains would follow. Appendix B covers conversational AI, Appendix C simulation and causal reasoning, and Appendix D the Statement of Work for the MVP1 build. Appendix E is a stepwise guide to applying the semantic standards in RDF Studio, and Appendix F defines terms and acronyms.

Value and Agentic Governance

A semantic layer gives enterprise AI structured, business-aware knowledge: what terms mean, how concepts relate, which actions are permitted, and which outcomes matter. That knowledge turns AI from a passive analysis tool into an agent that can interpret intent, plan, and act within business rules. Semantic governance keeps the layer accurate and trusted, and increasingly it is also how the agents themselves are governed.

How a semantic layer enables autonomous AI

- **Contextual reasoning.** Business logic, definitions, and relationships are encoded as ontologies and knowledge graphs, so AI can reason about how concepts connect rather than match patterns.
- **Consistent, trustworthy action.** Standard definitions for terms, metrics, and processes give every agent a single source of meaning and remove errors caused by ambiguous metadata.
- **Cross-system automation.** Entity resolution across sources (a CRM customer is the billing client) lets agents orchestrate approvals, reconciliation, and root-cause analysis across systems.
- **Goal-driven planning.** Models of cause and effect, rules, and hierarchies let agents plan multi-step actions toward business objectives and explain their rationale in business terms.
- **Machine-accessible business logic.** APIs and query languages let agents retrieve context, transform data, and trigger processes in line with formal semantics.
- **Guardrails.** Access rules, compliance constraints, and permitted actions defined at the business-concept level keep autonomy within policy, regulatory, and ethical bounds.

Where semantic governance fits

Semantic governance does not replace other governance disciplines; it supplies the business meaning that connects them.

Discipline	What it governs	What semantic governance adds
Data governance	Quality, access, privacy, and compliance of raw data	Bridges technical fields to business meaning
Metrics governance	How KPIs are calculated and displayed	Resolves ambiguity beyond the numbers
Analytic governance	Analytics tools and dashboards	Explains how a metric fits the business context
AI governance	Ethical, explainable models; bias prevention	Feeds AI trustworthy, meaningful concepts
Infrastructure	Pipelines, scale, performance, uptime	Adds meaning and trust, not just capacity
Semantic governance	Definitions, metadata, relationships, traceability	Unified, actionable business logic across all of the above

Business value and how to measure it

Value	What it means	How to measure
Smarter, more reliable AI and analytics	AI and BI get the right words, meanings, and context	Fewer AI/BI errors; faster decisions; higher tool adoption
A single business language	Sales, finance, and operations share definitions	Fewer metric disputes; consistent KPIs across reports
Standards-based knowledge	Knowledge integrates with new systems and partners	Faster onboarding; lower integration cost
One semantic layer	Metrics mean the same thing in every tool	Tools on the shared layer; fewer duplicate definitions
Living knowledge	Glossary and graph track how the business operates	Update frequency; speed of adapting to change
Explainable, causal answers	Every answer traces to a business term and its causes	Time from question to decision; decision accuracy

Alignment with industry direction

- **Analyst guidance.** Gartner research on data and analytics governance stresses active metadata, glossaries, catalogs, lineage, and a semantic layer to reduce AI hallucinations and improve explainability.
- **Standards.** ISO 11179 (metadata registries), ISO 704 (concepts and terminology), and the OMG Ontology Definition Metamodel provide the formal basis for consistent, reusable semantic assets.
- **Practice.** Leading organizations link business glossaries to catalogs and knowledge graphs, and treat semantic layers in BI tools and data fabrics as governed assets.

Governing agentic AI

As AI agents gain autonomy, governance must move from static rules to machine-readable policy that agents can interpret and that can be checked at run time. The semantic layer supports this in six ways:

- **Contextual policies:** rules, ethical constraints, and compliance requirements expressed in a form agents can apply.
- **Explainability:** decisions linked to the semantic framework produce traceable, human-readable explanations.
- **Lifecycle integration:** a central record of each agent's purpose, data use, and risk profile across build, deploy, and monitor.
- **Risk detection:** semantic reasoning flags behavior that departs from defined acceptable operation.
- **Interoperability:** shared terms and protocol semantics make multi-agent communication secure and compliant.
- **Scalable automation:** compliance checks and escalation run without constant human intervention.

Governance requirements for autonomous agents

Requirement	What it involves
Accountability and traceability	Clear ownership of AI-driven decisions; immutable audit trails with unique agent and human identifiers
Scope and boundaries	Explicit limits on data access, tools, and decision authority; escalation of ambiguous or high-risk cases
Transparency	Intermediate steps, assumptions, and reasoning disclosed in a form suited to each stakeholder
Identity-centric security	Verifiable agent identities with zero-trust, least-privilege access that adapts to behavior
Continuous monitoring	Behavioral thresholds, anomaly detection, and the ability to shut down or constrain an agent
Risk-based frameworks	Privacy, safety, and fairness controls extended for agents, aligned with ISO/IEC 42001 and the NIST AI RMF
Continuous governance	Regular evaluation and red-teaming, with feedback loops that update policy
Cross-functional coordination	Legal, technical, security, and compliance teams working from shared definitions

The risk is not hypothetical. Agents already coordinate through open protocols such as MCP and Agent-to-Agent (A2A), switch between protocols, and improve themselves. If monitoring and controls do not keep pace, inter-agent communication could bypass human-imposed guardrails, and analysts expect agents to run entire business functions with minimal oversight by around 2030. Observability, auditability, and dynamic security across agent communication are therefore core requirements, not add-ons.

Agent registries

Registries are the authoritative catalog of AI agents and the backbone of agent governance. They:

- **Identify every agent** as a verifiable, identity-bound entity with its capabilities, risk level, and permissions.
- **Track the lifecycle** of each agent, including purpose, version history, and compliance checks before deployment.
- **Classify risk** by autonomy and criticality so that oversight and escalation scale with exposure.
- **Hold audit trails** of behavior, communication, and decisions for monitoring and incident investigation.
- **Secure discovery** through cryptographic identity and federated approaches such as MCP and A2A agent cards.
- **Feed governance KPIs** such as trust, autonomy, efficiency, and incident frequency, and connect to CI/CD, access control, and incident response.

Frameworks and protocols to build on

Category	Examples	Governance role
Agent protocols	Model Context Protocol (MCP),	Structured, policy-aware messages carrying

Category	Examples	Governance role
	Agent-to-Agent (A2A)	agent identity, capabilities, and constraints
Orchestration frameworks	Semantic Kernel, Copilot Studio, LangGraph	Embed machine-readable policies and validation checks in multi-step workflows
Multi-agent frameworks	AutoGen, CrewAI, SuperAGI, smolagents	Role taxonomies, delegated permissions, and semantic task coordination
Formal ontologies	RDF, OWL, SHACL	Explicit vocabularies for governance concepts, risk levels, and agent capabilities
Evaluation consoles	Conversational agent consoles with observability	Benchmark agent responses against semantic policy standards

Semantic Governance Operating Model

This section describes the Operations Workbench that monitors, visualizes, and governs lifecycle changes to knowledge artifacts such as vocabularies, taxonomies, and ontologies in a graph database. The workbench is designed for this type of federated, dynamic knowledge graph environment, with strong integration and visualization capabilities.

Workbench Capabilities

Unified Knowledge Graph Modeling

The Operations Workbench includes a schema editor for designing or importing knowledge graph models directly from existing graph databases or RDF sources. Ontology-based data (RDF, OWL, RDFS) can be bound to the schema, enabling visualization and monitoring of knowledge artifacts from multiple repositories without duplicating the data.

Real-Time Data Integration and Change Binding

The workbench supports automatic schema extraction and real-time integration from federated data sources, including Neo4j, Amazon Neptune, Apache TinkerPop, and SPARQL endpoints. These connectors allow the workbench to display live data views of ontologies and vocabulary relationships, making it easy to detect and visualize changes as they propagate across taxonomies or linked models.

- Dynamic Binding: Updates in the graph database automatically refresh within workbench visualizations.
- Change Detection: Schema metadata can include timestamps or version identifiers, allowing new ontology versions or modified relationships to be highlighted.

Visualization and Impact Analysis

The platform provides rule-based, interactive graph visualizations and dependency mapping tools for analyzing ontology relationships. Capabilities include:

- Node-based impact graphs to see how a taxonomy update affects dependent vocabularies or instances.
- Centrality, clustering, and pathfinding analytics to show semantic influence or dependency strength.
- Overlay metadata (like version number, date, or steward) via styles or color-coding for governance oversight.

Lifecycle and Governance Monitoring

Graphs can be augmented with attributes such as:

- Version history
- Approval status
- Lifecycle state (draft, validated, deprecated)

- Authorized steward or domain owner

Using workbench dashboards and schema-linked attributes, administrators can monitor governance workflows, visualize change approval pipelines, and assess compliance against ontological or taxonomic policy guidelines. These dashboards transform abstract RDF governance rules into a live operational interface.

Example Workbench Configuration

Function	Implementation in the Operations Workbench
Change Monitoring	Link version metadata to nodes/edges and use visual cues to show modifications or pending approvals
Impact Visualization	Pathfinding analysis to display dependency propagation when a concept or class changes
Artifact Versioning	Visual panels to compare ontology versions and highlight added/removed entities
Governance Workflow View	Dashboard combining schema state, change history, and steward assignments
Integration with Graph DBs	Real-time API integration to Neo4j, Neptune, or any SPARQL endpoint
Role-based Access Control	Configurable access layers to manage who can modify or approve ontology changes

Architecture Integration

The Operations Workbench is deployed as an operations layer above the semantic stack:

Knowledge Repositories: Neo4j / GraphDB / Amazon Neptune

Integration Layer: Workbench schema bindings and federated connectors

Visualization Layer: Workbench UI built with the workbench designer

Governance Framework: Lifecycle metadata (status, steward, policy) stored directly within RDF or external SG system

Event Source (Optional): Kafka or graph triggers for asynchronous updates

Practical Outcome

With this setup, the Operations Workbench becomes:

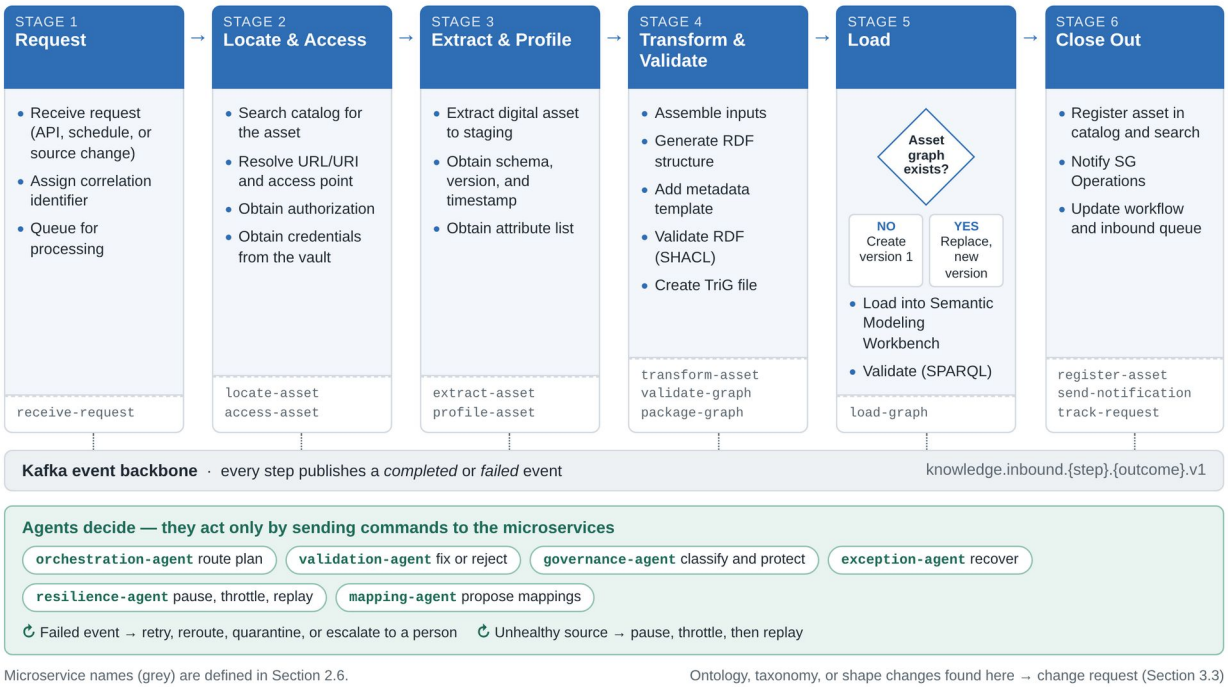
- A real-time control panel for ontology evolution and data integrity oversight.
- A semantic impact workspace showing downstream effects of vocabulary refinement.
- A governance-driven visualization suite allowing SG teams to monitor, validate, and authorize knowledge artifact changes as they occur.

In practice, the Operations Workbench integrates with SPARQL-compatible graph databases, detects and visualizes ontology-level changes, and manages governance workflows within an interactive operations environment.

Inbound Semantic Services Design

Inbound Flow

Inbound processing moves each asset through six stages, from request to close-out. Microservices perform each step and publish an event to the Kafka backbone; agents make the decisions and handle exceptions. Section 2.6 defines each microservice and agent.



Microservice names (grey) are defined in Section 2.6.

Ontology, taxonomy, or shape changes found here → change request (Section 3.3)

A robust automation architecture for Inbound Processing in Semantic Governance can be built using a network of Python-based microservices, coordinated AI Agents for decision-driven orchestration, and semantic tools like RDFLib integrated with Semantic Modeling Workbench's APIs. Each service handles one atomic function for clear traceability and schema compliance, while agents use orchestration logic to sequence the services, handle exceptions and retries, and enforce governance checkpoints between steps.

This framework merges microservices, agentic orchestration, and semantic governance automation, producing a repeatable and scalable inbound data handling model that conforms to MBSE-aligned digital engineering architectures.

Architecture Overview

This pipeline aligns with semantic governance principles, combining AI orchestration, microservice modularity, and knowledge graph operations.

Core components include:

1. Microservices Layer (Python REST Services) for discrete operations.
2. AI Agent Layer using Semantic Kernel or LangChain for sequencing, reasoning, and error resolution.

3. Semantic Data Layer using RDFLib for RDF modeling, and Semantic Modeling Workbench Graph

   Store API for ingestion and validation.

Workflow

1. Inbound request enters queue and invokes pipeline.
2. AI orchestrator calls microservices per order.
3. RDF file generation (`.TRIG`) occurs post validation.
4. File uploaded to Semantic Modeling Workbench Graph Store; ontology version validated.
5. Workflow entry updated; SG notified.

AI Agents and Orchestration Logic

Using Semantic Kernel's multi-agent orchestration framework, three main AI agents can manage this pipeline:

- Orchestration Agent: Central decision unit that invokes sequential microservices (Sequential Pattern).
- Validation Agent: Evaluates ontology and SHACL constraint compliance.
- Governance Agent: Audits metadata and ensures policy conformance for schema naming and version control.

Each agent runs under Semantic Kernel orchestration, utilizing Sequential and Handoff patterns:

1. Scheduler → Asset Retrieval → RDF Generation → Validation → Loading → Ontology Versioning → Notification.
2. Agents share context stores to ensure schema, metadata, and governance coherence.

Semantic Governance Compliance

Governance rules align roles, privileges, and review checkpoints:

- Policy Framework: Defines approval thresholds for ontology promotion.
- Metadata Governance: Ensures proper schema linkage and provenance attributes.
- Lifecycle Tracking: Captures versioning and timestamps in RDF metadata.
- Audit Trail: Updates Semantic Governance (SG) workflow with activity logs.

Microservices Breakdown

Service	Function	Tools / Libraries
Request Scheduler	Queue inbound requests, assign IDs, and trigger workflow.	Celery + Redis
Search Digital Asset	Query asset repository with metadata filters.	ElasticSearch / Custom API
Search Catalog	Locate relevant schema or ontology source.	Semantic Modeling Workbench Query Catalog API
Extract Digital Asset	Download or parse content from stored archive.	Python requests , JSON / XML parser
Obtain URL/URI &	Resolve persistent identifiers.	W3C URIRef via RDFLib

Service	Function	Tools / Libraries
Access Point		
Obtain Authorization & Credentials	Token exchange with OIDC or API keys.	Authlib / Keycloak
Obtain Schema, Version, Datetime	Retrieve ontology meta-data and constraints.	SPARQL Query via RDFLib
Obtain Attributes (list)	Parse RDF predicates or class properties.	RDFLib Graph.triples()
Assemble Inputs → Generate RDF	Serialize data into RDF triples.	RDFLib Graph.serialize()
Add Metadata for Ontology Template	Append custom annotations.	RDFLib Namespace DCTERMS , , OWL RDFS
Validate RDF Structure	Apply SHACL constraints within Semantic Modeling Workbench.	SHACL API in Semantic Modeling Workbench
Create .TRIG file	Use RDFLib .serialize(format="trig") .	RDFLib
Load Digital Asset to Semantic Modeling Workbench	POST to Graph Store API ( ). /rdf-graphs/service	Semantic Modeling Workbench Graph Store API
Ontology Versioning Service	Check and auto-increment ontology versions.	SPARQL Query + Version registry
Validate Ontology (SPARQL)	Check consistency, domain/range types.	RDFLib SPARQL engine
Queue & Workflow Updating	Update inbound and completed job states.	REST DB / Kafka topic
Notification Service	Notify SG Operations of completion.	SMTP / Slack API

Section 2.6 turns this breakdown into a reusable catalog of Kafka-based microservices and decision-making agents, with standard naming, logging, exception handling, and a search facade; where the tools listed here differ, Section 2.6 applies.

Deterministic Knowledge in the Knowledge Fabric

Incorporating deterministic knowledge through a top-down knowledge fabric—built with taxonomies, ontologies, federated graphs, and AI—enables strict policy, controls, and the emergence of adaptive behaviors within business processes.

Top-Down Knowledge Fabric Construction

- Taxonomies as Foundation: Taxonomies classify concepts and entities hierarchically, providing clear categories and preferred vocabulary for enterprise data and business objects. They are invaluable for structuring information and supporting efficient search and retrieval.
- Ontologies for Enriched Semantics: Ontologies map relationships, attributes, and rules governing how business objects connect and behave, turning static classifications into a dynamic semantic network. Unlike pure taxonomies, ontologies add context, constraints, and reasoning capabilities, crucial for deterministic policy enforcement and compliance.
- Federated Knowledge Graphs: By connecting multiple ontologies (internal and external) into federated graphs, organizations create a unified semantic layer. This enables contextual policies and enterprise-wide controls to propagate dynamically, supporting federated compliance, audit trails, and knowledge-sharing.
- AI for Automation and Emergence: Embedding taxonomies and ontologies in AI platforms—especially LLMs, semantic tagging, and graph-based reasoning—enables automatic validation, classification, and the surfacing of emergent behaviors. AI tools can enrich taxonomies, suggest new concepts from unstructured data, and enforce semantic rules in real-time process automation.

Delivering Policy, Controls, and Emergent Behaviors

- Policy and Controls: Deterministic knowledge models enable rule-based compliance, auditability, and controlled vocabularies. Semantic tagging and graph relationships ensure that assets, decisions, and user actions are governed by centrally defined policies and ontological rules.
- Emergent Process Behaviors: When business processes are modeled within an ontology-driven fabric, AI can leverage relationships, constraints, and semantic context to adapt workflows in response to dynamic triggers—such as shifting requirements, compliance checks, or new business insights.
- Unified Traceability and Responsiveness: Every process touchpoint, asset, or decision tagged to the knowledge graph reflects the current policy landscape. Automated reasoning embedded in the graph fabric enables deterministic action

  with room for emergent adaptation aligned to best practices and business outcomes.

Implementation Highlights

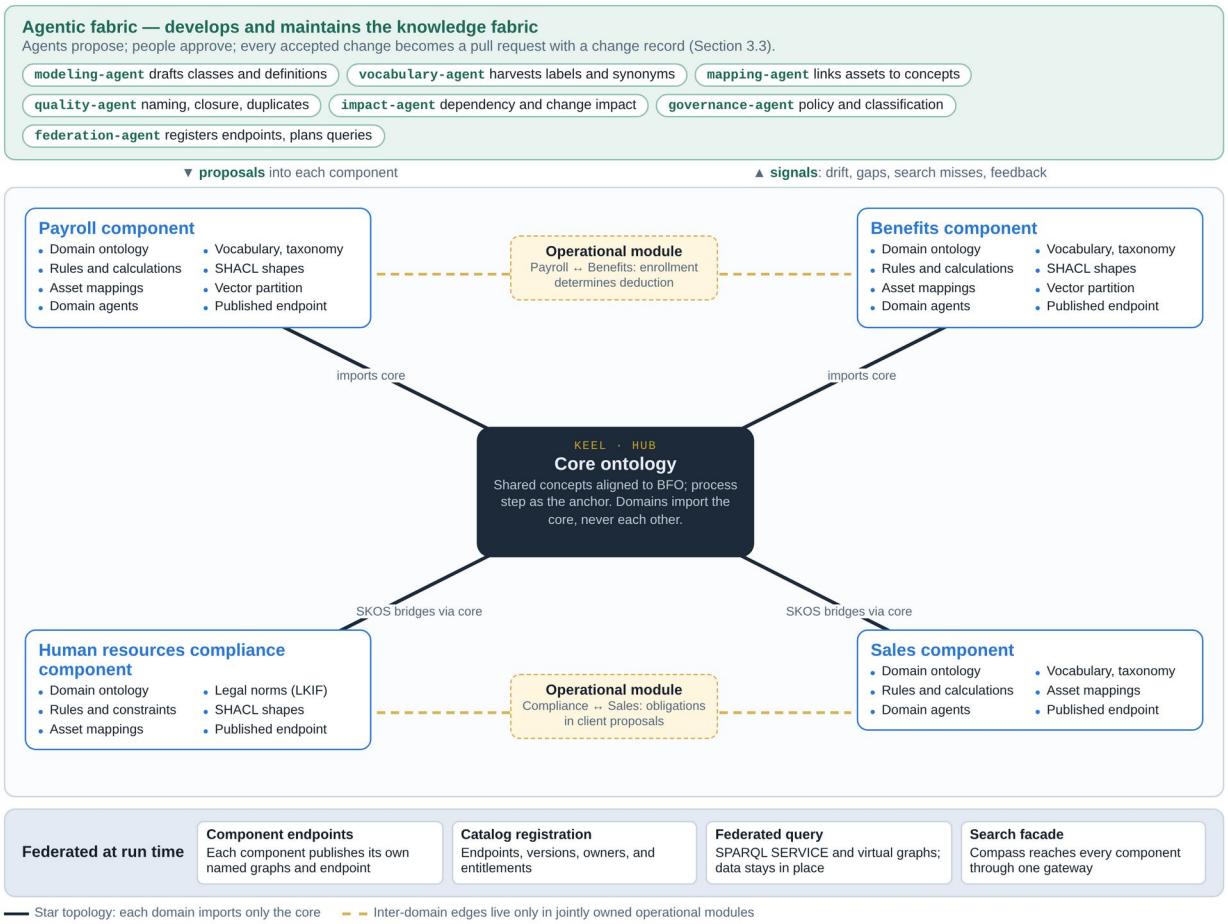
- Transform existing taxonomies into enriched ontologies and integrate them into federated knowledge graphs.

- Use Semantic Web standards (like RDF, OWL, SKOS) to link vocabularies and support automated reasoning.
- Integrate AI-powered entity extraction and tagging to surface new relationships and ensure policy adherence in real time.
- Map all process activities to ontology classes so contextual policy and controls are enforced automatically while enabling workflows to evolve responsively.

This approach delivers highly regulated, transparent, and adaptable processes—utilizing deterministic knowledge frameworks and AI to balance compliance and innovative emergent behaviors in service delivery

Developing the Knowledge Fabric with an Agentic Fabric

The **knowledge fabric** is the governed, federated body of meaning described in this section: ontologies, vocabularies, taxonomies, rules, and mappings, connected across domains. The **agentic fabric** is the governed set of AI agents that help build, maintain, and use it. In development, agents do the repetitive work of drafting, harvesting, checking, and connecting; people decide what becomes authoritative. Two design choices make this work at enterprise scale: a **component design** that packages each domain as a self-contained, versioned unit, and a **federated topology** that connects those components through the core rather than to each other.



Component Design

Each business domain is delivered as a knowledge component with the same standard parts, so any team can build, test, release, and reuse a domain the same way:

Part	Contents	Owner
Domain ontology	Classes and intra-domain properties that extend the core (Section 3.1)	Domain knowledge engineer
Vocabulary and taxonomy	SKOS concept schemes, preferred, alternate, and persona labels (Section 3.8)	Domain data steward
Rules and calculations	The domain's entries in the single source of truth (Section 3.5)	Domain subject-matter expert
Shapes and norms	SHACL shapes for data quality; LKIF norms where regulation applies	Domain knowledge engineer
Asset mappings	Mappings from source schemas and digital assets to domain concepts	Domain data steward
Vector partition	Domain retrieval partition anchored to ontology IRIs (Section 2.5)	Platform team
Domain agents	Agents scoped to the domain, with their allowed actions and thresholds	Domain owner and Architecture
Published interface	Named graphs, endpoint, Decision service rules, and search registration	Platform team
Component manifest	Version, dependencies on the core and operational modules, owners, and release notes	Domain owner

- **Depends on the core only:** a component imports the core ontology and never imports another domain component.
- **Released as a unit:** each component has its own semantic version, source repository or folder, code owners, and release through the pipeline in Section 3.3.
- **Replaceable and reusable:** a component can be upgraded, rolled back, or reused by another business unit without changing the others, as long as its published interface stays compatible.

Federated Topology

Components are connected in a **star topology** with the core ontology at the hub. The same shape applies at every level, which keeps the number of connections linear as domains are added, rather than growing with every pair of domains:

Level	Topology rule
Logical (imports)	Every domain imports the core; inter-domain properties live only in operational modules that import the domains they connect and are owned jointly (Section 3.1)
Terminology (mappings)	Each domain maps its terms to the core through reviewed SKOS bridges; domains never map directly to each other (Section 3.5)
Physical (storage)	Each component publishes its own named graphs and endpoint, registered in the catalog with version, owner, and entitlements
Run time (queries)	The federation-agent plans federated queries across registered endpoints and virtual graphs; data stays in place (Section 2.6)

Agentic Fabric for Development

The development agents extend the run-time agents in Section 2.6 and follow the same guardrails: they act only by proposing, every proposal is reviewed, and every accepted change becomes a pull request with a change record.

Agent	Development task	Approved by
modeling-agent	Drafts classes, definitions, and properties from approved requirements (ORSD, Appendix A.4) using the term definition standard (Section 3.7)	Domain knowledge engineer
vocabulary-agent	Harvests candidate alternate and persona labels from search logs, documents, and source systems	Domain data steward
mapping-agent	Proposes mappings from new source schemas and assets to domain concepts	Domain data steward
quality-agent	Checks naming conventions, missing definitions, orphan terms, and near-duplicate concepts before review	Automated gate
impact-agent	Lists every rule, mapping, metric, service, and agent affected by a proposed change	Change reviewer
governance-agent	Applies classification, access, retention, and licensing policy to new components and assets	Governance owner
federation-agent	Registers component endpoints in the catalog and keeps the federation plan current	Platform team

Development Loop

1. A requirement is approved with its competency questions (ORSD).
2. Development agents draft the change in a sandbox copy of the component.
3. The quality-agent and SHACL shapes check the draft; the impact-agent lists what it affects.
4. The owners review and accept, amend, or reject each proposal.
5. Accepted changes are committed as a pull request with a change record and released through the pipeline (Section 3.3).
6. The released component is registered for federation and search; run-time signals (drift, search misses, feedback) return to step 1 as new requirements.

Alignment with Keel and North Star

The knowledge fabric and the agentic fabric are expressions of the Keel and North Star foundational architecture (Section 2.1). The fabric is the keel; the agents work on the bridge, in the hull, and at the waterline; governance is the ballast; and every outcome is measured against the North Star.

Fabric element	Vessel layer	Alignment
Core ontology and domain components	Keel	Governed meaning, authored once and versioned; holds no run-time state
Operational modules and SKOS bridges	Keel	Cross-domain meaning made explicit and owned, never implied in code
Federated endpoints, virtual graphs, mappings	Hull	Connect the keel to moving sources and keep source changes out of the meaning

Fabric element	Vessel layer	Alignment
Run-time agents (search, answers, exceptions)	Bridge and hull	Read meaning from the keel; hold none of their own
Development agents and drift signals	Waterline	Detect change and turn it into proposals and change requests
Change control, approvals, provenance	Ballast	Keep the fabric upright: nothing becomes authoritative without review
Measured outcomes	North Star	Trustworthy AI, drift-proof integration, audit-grade lineage, faster delivery

- **Meaning lives only in the keel:** agents, services, and applications reference keel IRIs; none defines meaning of its own.
- **Stack-independent:** components are standard RDF, OWL, SKOS, and SHACL, so any platform or vendor tool can consume them through the published interfaces.
- **Steer by the North Star:** every component release reports against the North Star outcomes, measured in the target operating model (Section 3.6).

Demand Management and Agility

To build an agile workforce and intelligent service delivery platform, implement unified demand management that integrates external requirements with internal customer needs, systematically vet demands through compliance and corporate priorities, and rationalize these into actionable solution design. This approach enables responsive adaptation while maintaining strategic alignment and governance.

Unified Demand Management Framework

- Centralized Demand Hub: Establish a platform where all external demands (clients, regulatory, partners) and internal needs (operational teams, business units) are captured, tracked, and integrated for visibility and traceability.
- Prioritization Engine: Apply decision criteria to rank and vet demands—beginning with internal customer impact, then subjecting shortlisted demands to compliance checks (risk, regulatory, security) and corporate strategic needs (growth, cost management, ESG).
- Governance and Rationalization: Introduce a governance workflow where compliance, regulatory, and executive teams review, rationalize, and re-prioritize demand pools, ensuring only high-value, feasible initiatives progress to solution design.
- Agile Solution Design Integration: Connect prioritized demands directly to agile solution teams and service architects, who build and extend platforms around the most urgent validated needs, with continuous feedback loops for adaptation and improvement.

Agile, Intelligent Workforce Enablement

- Real-time Orchestration: Use telemetry-driven orchestration mechanisms (for example, event-driven workflow engines fed by operational telemetry), enabling intelligence to dynamically allocate resources, reconfigure processes, and manage workloads based on the current mix of demands and operational realities.
- Transparent Resource Allocation: Managers and teams utilize dashboards and portfolio planning tools to match workforce capacity and expertise to the rationalized priority list, fostering agility and cross-team collaboration.
- Compliance-Led Adaptability: Security, compliance, and policy controls operate seamlessly within this framework, ensuring every solution adheres to standards while empowering innovation.

Steps to Implement

1. Consolidate all demand streams (external, internal) into a unified digital hub.
2. Apply multi-layered prioritization: internal value, compliance, and strategic alignment.
3. Governance teams vet and rationalize, resolving conflicts and ensuring cohesiveness.
4. Feed prioritized, rationalized demands to agile solution design teams.
5. Deploy intelligent orchestration and adaptive resource management for workforce and service delivery.

This approach creates a dynamic, compliant, and value-driven ecosystem for extending agile workforce capabilities and intelligent service delivery, tightly aligning technology solutions with real business needs in a fast-evolving environment.

Neuro-Symbolic Foundation for AI

Enterprise AI needs two kinds of intelligence. Neural models, such as large language models and the embeddings held in vector databases, bring fluency, pattern recognition, and the ability to work with unstructured content. Symbolic models, such as ontologies, knowledge graphs, and rules, bring explicit meaning, logical constraints, and reasoning that can be inspected. A neuro-symbolic architecture combines the two so that AI can think with knowledge rather than only learn from data. Semantic governance is what makes the symbolic half trustworthy: it decides which meanings are authoritative, how they change, and how they are enforced.

Why the symbolic layer matters

- **Explainability.** When relationships, hierarchies, and constraints are encoded explicitly, an AI answer can be traced back to the facts and rules that produced it, rather than to opaque model weights.
- **Contextual intelligence.** Pattern recognition alone cannot tell an agent what a term means in this business, which actions are permitted, or how two concepts relate. The knowledge graph supplies that context.
- **Interoperability.** Open W3C standards let internal and external data, ontologies, and applications connect without bespoke translation for every pair of systems.

RDF: the atomic unit of meaning

In RDF every fact is a triple of subject, predicate, and object, each identified by a global IRI. Triples are small enough to update one fact at a time, yet any triple can link to any other, so the graph grows without breaking. This is how the core ontologies (Process, Service, Asset, Product, Party, Location, Time, Event, Issue) and the domain ontologies built on them become one connected fabric, and how structured and unstructured sources are unified without forcing them into a single schema.

From facts to knowledge: RDFS, OWL, and SHACL

RDF Schema and OWL add meaning on top of facts. Class hierarchies and property characteristics allow the graph to infer relationships that were never stated directly, for example that a variance report is a kind of financial analysis. SHACL adds the governance half: shapes declare what valid data looks like and which policies apply, so changes can be validated before they are accepted. In this architecture SHACL acts as a policy gate for agents and workflows, as shown in the FP&A sequence diagrams (Section A.11).

SPARQL: querying with graph thinking

SPARQL queries patterns across the graph rather than joining tables. Multi-hop questions, such as which cost centers depend on a driver that changed last quarter, become a single query. The same ability to extract a precise, context-rich subgraph is what grounds retrieval-augmented generation: the Federated RAG 2.0 architecture (Section 2.1) retrieves governed facts by meaning before the language model writes its answer.

RDF and property graphs

Property graphs suit fast, localized traversal and analytics inside one application; RDF is built for ecosystems, with standard identifiers, inference, validation, and federation. Keep authoritative meaning in the governed RDF fabric and project it into property graphs where an application needs them.

How the pieces fit in this architecture

Layer	Role	Where it appears
Symbolic knowledge	Core and domain ontologies, taxonomies, and SHACL policies authored in the Semantic Modeling Workbench and stored in the graph database.	Sections 2.1, 1.3
Neural intelligence	Language models and embeddings in domain-aligned vector databases for semantic search and generation.	Section 2.5
Bridge	The semantic scaffold links each embedding to ontology IRIs; domain agents maintain mapping ontologies.	Sections 2.5, 2.1
Governance	ORSD requirements and KGCL change management keep the symbolic layer versioned and auditable.	Sections A.4–A.6

Design principles

- **Make meaning explicit.** Define business concepts once, in the ontology, and reference them everywhere.
- **Validate before acting.** Route agent actions and data changes through SHACL policy checks.
- **Ground every answer.** Retrieve governed facts before generation, and cite the IRIs used.
- **Stay standards-based.** Use RDF, RDFS, OWL, SHACL, and SPARQL to keep the fabric portable and federated.
- **Evolve incrementally.** Record every schema change as a KGCL statement tied to an ORSD entry.

Context Engineering

Context engineering is the practice of assembling everything a model sees at inference time: instructions, retrieved knowledge, memory, tool outputs, system state, and policy. It is where the governed knowledge described in this document is put to work. Prompt engineering is one part of it; vector databases are one of its sources.

Three complementary disciplines

	Semantic governance	Knowledge engineering	Context engineering
Purpose	Policy, compliance, audit	Formalize domain knowledge	Assemble dynamic context for AI
Produces	Rules, constraints, metadata layers	Ontologies, taxonomies, knowledge bases	Context pipelines, prompt templates, memory
Owned by	Governance and compliance teams	Knowledge engineers, ontology managers	AI engineers, context designers
Role in AI	Sets the rules and trust framework	Supplies structured meaning	Delivers the right context at the right time

From prompts to context

	Prompt engineering	Context engineering
Focus	Wording of a single prompt	The full information environment at inference
Scope	One-off or short interactions	Multi-turn workflows and agents
Inputs	Instructions and examples	Prompts, retrieval, memory, tools, system state, policy
Scalability	Brittle as tasks grow	Designed to be reusable, governed, and robust

Components of a production pipeline

- **Composition:** select inputs such as retrieved documents, tool definitions, instructions, session history, and metadata.
- **Ranking and filtering:** score relevance by similarity, keywords, and recency; remove duplicates and noise.
- **Memory management:** summarize prior turns and recall preferences and history across sessions.
- **Tool integration:** bring results from code, queries, and APIs into the context in a usable format.
- **Instruction orchestration:** versioned, reusable prompt templates for multi-step and branched flows.
- **Dynamic retrieval:** fetch information as the task evolves rather than front-loading static context.
- **Format optimization:** clear sections, labels, and summaries to improve comprehension and token use.

- **Evaluation and feedback:** test context relevance and output quality; diagnose context versus model failures.
- **Policy and compliance:** guardrails, compliance checks, and audit metadata built into the pipeline.
- **Cost and scale controls:** scoping, compression, and prioritization to balance completeness with latency and cost.

Context engineering and vector databases

Vector databases remain the semantic retrieval layer: they store embeddings and return relevant documents, memories, and data quickly and at scale. Context engineering sits above them. It combines what is retrieved with memory, tool output, and policy; manages token limits; and adapts what the model sees at each step of a multi-turn or multi-agent workflow. Used together, they address the limits of retrieval alone, such as information overload, hallucination, and context-window pressure. Section 2.5 describes the vector database design, and Appendix C extends context engineering with simulation.

Solution Architecture

Solution Architecture: Federated RAG 2.0 with Semantic Modeling Workbench & Gemini

This architecture creates an intelligent, self-optimizing system for managing enterprise knowledge. It establishes a dynamic link between abstract business concepts and concrete digital assets, powered by a component-based design that promotes scale and domain-specific learning. It is built on the Keel and North Star foundational architecture described at the end of this section.

Top-Down Knowledge Layer (Core & Domain Ontologies)

This is the foundation of the architecture. It's the central source of business truth, defining what the company is, how it operates, and what it knows.

- **Core Ontologies:** A foundational set of ontologies (Process, Service, Asset, Product, Party, Location, Time, Event, Issue) that provides a consistent, high-level vocabulary for the entire enterprise. All domain-specific ontologies will extend and reuse these core concepts.
- **Domain-Specific Ontologies:** For each business domain (e.g., Sales, Marketing, HR), an ontology is built on top of the core ontologies. This layer enriches the core concepts with domain-specific terms, relationships, and business rules, representing the organization's unique knowledge in a machine-readable format.
- **Semantic Modeling Workbench's Semantic Modeling Assistant:** This agent is used to author and manage the core and domain-specific ontologies. It provides a visual, user-friendly interface for knowledge engineers and subject matter experts to create and extend these top-down structures.

Bottom-Up Knowledge Layer (Digital Assets)

This layer represents the raw data and metadata from across the enterprise. It's the physical reality that the top-down layer seeks to model.

- **Schemas and Metadata:** This includes database tables, columns, business intelligence metrics, hierarchies, and other metadata from all digital assets (e.g., Salesforce, ERP, data lakes).
- **Digital Asset Inventory:** The schemas and metadata are automatically or semi-automatically ingested into a central repository (e.g., Data Plex in GCP), creating a comprehensive inventory of all digital assets.

Mapping Layer & Federated Agents

This is the most critical and intelligent part of the architecture, where the connection between the top-down and bottom-up layers is established and continuously improved.

- **Mapping Ontologies:** Each bottom-up schema (e.g., `users_table`, `sales_transactions`) will have its own dedicated mapping ontology. This ontology formally defines the relationships (e.g., `isRepresentedBy`, `hasAttribute`) between specific top-down ontological classes and the bottom-up schema elements. This granular approach supports polymorphic meaning.
- **Semantic Modeling Workbench's Search & Discover Agent:** This agent acts as a federated interface. It uses the top-down and mapping ontologies to enable users to search for data using

business terms. It translates a business query like "Find customer emails for the Q3 sales campaign" into the technical query required to retrieve `users_table.email_address` from the data source.

- **Domain-Specific AI Agents:** An individual AI agent is assigned to each business domain (Sales, Marketing, HR, etc.). These agents are responsible for two core functions:

1. **Mapping at Scale:** They continuously learn and propose new mappings between top-down concepts and bottom-up schemas, accelerating the manual process. They identify patterns and suggest relationships that human experts might miss.
2. **Extending Knowledge:** They analyze unstructured data and user queries within their domain to propose extensions to the domain-specific ontologies, vocabularies, and taxonomies, ensuring the knowledge layer remains comprehensive and up-to-date.

Federated RAG 2.0 with Domain Vector Partitions ("Virtual SLMs")

Terminology note: in this document, a "virtual SLM" means a domain-specific vector database partition that gives a single general-purpose LLM the knowledge a specialized small language model would otherwise hold. The MVP1 target topology deploys **one LLM plus domain vector partitions**; it does not deploy separate SLM instances. Dedicated SLMs remain an optional future optimization for high-volume domains.

This layer explains how the system interacts with a single large language model (Gemini) to provide grounded, context-aware responses without the need for multiple, costly specialized language models (SLMs).

- **Vector Databases per Domain:** A separate vector database is created for each business domain. These are not "virtual SLMs" in the traditional sense, but they serve as a virtual, domain-specific knowledge cache. They store vectorized representations of:

1. The top-down and mapping ontologies for that domain.
2. Conversational queries and their successful mappings.
3. Positive and negative feedback from users.

- **The Single LLM (Gemini):** A single, powerful LLM acts as the core reasoning engine. It doesn't need to be specialized for each domain because it has the knowledge layer to rely on.
- **Federated RAG 2.0 Workflow:**

1. **Query from User:** A user submits a query to the conversational AI interface (e.g., "What are our most profitable products?").
2. **Query Decomposition & Domain Routing:** The conversational AI and a separate routing agent (a simple rule-based agent or a specialized LLM call) analyze the query and identify the most relevant business domains (e.g., Sales, Finance).
3. **Federated Retrieval:** The query is sent to the vector databases of the relevant domains. The system retrieves the most relevant semantic mappings, scenarios, and terms. This is the **Retrieval** step of RAG.
4. **Enriched Context for LLM:** The retrieved information (the mappings, relevant ontology classes, and past successful queries) is combined with the user's original question to create a highly specific and grounded prompt.

5. Generation & Response: Gemini generates a response based on this enriched, grounded prompt. The response is then traced back to the mappings and ontology terms it used; only responses that pass the feedback controls below update the vector database.

Feedback Validation Controls

To keep incorrect or hallucinated answers from being learned back into the vector space, feedback is written only through a controlled path:

1. **Traceability check:** every answer must cite the ontology IRIs and mappings it used. Answers that cannot be traced are never stored as feedback.
2. **Policy validation:** the cited mappings are validated with SHACL against the domain ontology before any update.
3. **Confidence threshold:** feedback is written automatically only when the mapping confidence is at or above the domain's threshold (initially 0.85) and the user has confirmed the answer; anything lower is queued for review.
4. **Human-in-the-loop review:** a Knowledge Engineer or Data Steward approves queued items, and every new mapping proposed by an agent requires approval before it becomes authoritative.
5. **Quarantine and rollback:** new feedback is versioned and held in a staging partition for one release cycle; it is promoted only if retrieval quality metrics do not degrade, and can be rolled back by version.
6. **Audit:** every write records the source query, response, reviewer, and version, so feedback can be traced and reversed.

Thresholds and the staging period are initial values to be calibrated during MVP1.

Recursive Learning & Governance

The system is designed for continuous improvement and strong governance.

- **Recursive Learning:** Every successful query and mapping is stored as positive feedback in the domain-specific vector database. When a new query is submitted, the system first consults its learned mappings, continuously improving its accuracy and speed. If a query fails, the negative feedback is captured, allowing the AI agents to propose corrections or extensions to the ontologies or mappings.
- **Component Design:** The architecture is modular. Each domain is a self-contained component with its own ontology, vector DB, and AI agents. This allows for scalability and independent development without impacting other domains.
- **Central Governance:** The core ontologies and the mapping process are governed by a central team. This ensures consistency and prevents data fragmentation. A governance dashboard tracks the health of each domain's knowledge layer, monitors mapping confidence scores, and flags areas for human intervention.
- **LookML Integration:** The metadata for the schemas is automatically generated from the LookML models in Looker, ensuring that the bottom-up layer is always in sync with the live analytics layer. This data is then consumed by the Semantic Modeling Workbench agents and vector databases.

Foundational Architecture: Keel and North Star

The solution architecture is built on the Keel and North Star architecture, which is the foundational architecture for this document. The **keel** is the governed, deterministic semantic foundation: ontologies, vocabularies, the single source of truth for rules and calculations, shapes, and metric definitions, authored once and versioned. The **North Star** is the bearing every layer steers toward: every decision, insight, and AI answer rests on governed, traceable meaning. Between them, the hull mediates between the keel and moving sources, the bridge holds the two core capabilities (Compass for finding and explaining meaning, Copilot for answering and acting on it), and the ballast of governance keeps the vessel upright.



Foundational architecture: Keel and North Star. Section references show where each layer is specified in this document.

The layers of this section map onto the vessel as follows:

Layer in this section	Vessel layer	Role
Top-down knowledge layer (core and domain ontologies)	Keel	Governed meaning that every other layer reads
Bottom-up knowledge layer (digital assets)	The sea, described in the keel	Sources stay where they are; their RDF descriptions live in the keel
Mapping layer and federated agents	Hull	Links assets to concepts and answers queries in place
Federated RAG 2.0 with domain vector partitions	Bridge (Copilot) and hull	Grounded answers from one LLM, retrieval anchored to ontology IRIs
Search and discover agent	Bridge (Compass)	Finds assets by meaning through the search facade (Section 2.6)
Feedback validation controls	Waterline and ballast	Nothing is learned back without traceability, validation, and approval
Recursive learning and governance	Ballast	Change control, release pipeline, and monitoring (Part III)

Industry Alignment

Why Sales Operations here? The research alignment and the vector database design in this part (Sections 2.2 and 2.5) were developed against **Sales Operations** (leads, quotes, orders) as the first pilot domain. Appendix A uses **FP&A** as a second, fully worked domain. The pattern is domain-agnostic: core ontologies, a domain ontology, mapping ontologies, a domain vector partition, and domain agents. Any domain, including Sales Operations, FP&A, Marketing, or HR, follows the same structure.

This document outlines how our approach to building domain-specific knowledge—leveraging top-down ontologies mapped to bottom-up metadata via a process-centric model—aligns with the concepts and methodologies presented in the following two authoritative resources:

- Pliks et al., 2010, "Knowledge management and decision support in adaptive case management platforms" (pulled from IEEE - link below https://ieeexplore.ieee.org/abstract/document/7321628?signout=success)
- (alternate link): https://www.academia.edu/72947572/Knowledge_Management_and_Decision_Support_in_Adaptive_Case_Management_Platforms?email_work_card=view-paper
- Search-O1 Project, Enterprise Semantic Search Framework (https://search-o1.github.io/) (recommended by Rajender R Nednur)

Overview of Our Approach

Specifically, our approach:

- Combines top-down ontology engineering with bottom-up metadata extraction, reflecting the dual-layer knowledge modeling advocated by Pliks et al. (see Sections 2 and 5);
- Leverages a process-centric semantic model aligned with Pliks' emphasis on business process impact on ontology structure (Section 4);
- Incorporates iterative validation and refinement mechanisms consistent with Pliks et al.'s recommendations (Section 6);
- Implements semantic vector embeddings and unified ontology-schema mappings that enable semantic and conversational search, which aligns strongly with Search-O1's architecture and goals (as detailed in their Concepts and Architecture pages);
- Enforces modular subdomain decomposition and context filtering, vital in Search-O1 and supported by Pliks et al. for scalable, domain-aware knowledge management;
- Provides a knowledge fabric foundation that enables intelligent agents to operate under a consistent top-down knowledge topology at enterprise scale.

This combined framework delivers a resilient, explainable, and scalable semantic search and conversational agent environment, practical for complex domains such as sales operations.

Integration of Top-Down and Bottom-Up Ontologies

- Pliks et al. (Section 2, p. 3-5) highlight the critical value of integrating *expert-curated top-down ontologies* with *bottom-up extracted knowledge* to comprehensively capture enterprise semantics.
- Our approach mirrors this by utilizing a domain-specific sales operations ontology (top-down) mapped explicitly to metadata from digital asset schemas (bottom-up). This hybrid data model maintains semantic rigor while grounding ontology classes in practical, data-driven instances.

Process-Centric Semantic Modeling

- In Section 4 (p. 9-11), Pliks et al. emphasize modeling business processes and workflows to appropriately structure ontologies, improving alignment with operational semantics and data source contexts.
- Our architecture organizes sales operations into subdomains (lead management, pipeline, order management), using a process-centric model to clarify concept relationships and improve mapping accuracy—a direct application of these principles.

Iterative Mapping Validation and Refinement

- Section 6 (p. 14-16) describes the importance of continuous validation of ontology-to-data mappings supported by evidence and confidence metrics, suggesting iterative refinement to maintain semantic integrity.
- Our database captures mapping confidence scores, validation statuses, evidence texts, and training labels (positive/negative) to enable this ongoing refinement, ensuring conceptual accuracy and robustness.

Ontology-Driven Semantic Search

- Pliks et al. discuss how ontology-driven integration supports semantic querying and retrieval (Abstract, Section 7, p. 17-18). Semantic layers allow for *context-aware*, *concept-based search* transcending simple keyword matching.
- Our use of dense vector embeddings aligned with ontology vocabulary and mapping tables operationalizes semantic search, empowering intelligent agents to understand queries at a conceptual level rooted in the knowledge fabric.

Modularity and Scalability

- The paper advocates modular ontology design to accommodate the complexity of enterprise domains and simplify maintenance (Section 5, p. 12-13).
- Our design incorporates subdomain indexing and modular service areas to manage complexity modularly, in line with these recommendations.

Semantic Embeddings and Vector Databases

- Search-O1's core architecture rests on transforming ontological and schema knowledge into vector spaces to support semantic similarity search (Concepts page).
- Our use of vector embeddings for ontology entities and schema metadata directly implements this principle, enabling robust semantic retrieval crucial for enterprise-scale knowledge access.

Unified Ontology-Schema Mapping

- Search-O1 stresses mapping domain ontologies to heterogeneous metadata schemas for uniform enterprise insight and searchability (Architecture page).
- The mapping tables in our model reflecting entity correspondences, types, confidence, and evidence match this, providing organized, machine-readable links to unify disparate data assets.

Contextual Subdomain Filtering

- The project promotes sophisticated knowledge layer modularization where subdomains or domains can be scoped and filtered during query or agent interactions.
- Our subdomain index and process-centric model embody this concept, enabling agents to operate with semantic precision by focusing on the most relevant domain slices.

Intelligent Agents and Conversational Search

- Search-O1 envisions agents leveraging the knowledge fabric to power natural language search and complex workflows (Agent and Conversational Search demos).
- Our curated knowledge fabric, enriched with validated mappings, enables agents to perform contextually informed conversational search, consistent with their vision for scoping, reasoning, and retrieval.

Capability Map & Alignment with Best Practices

Capability	Supported By Our Approach	Corresponding Article Evidence
Comprehensive, accurate domain representation	Top-down ontology + bottom-up metadata mapping	Pliks et al. Sections 2, 5, 6
Context-aware, process-centric semantic relations	Process-centric domain modeling	Pliks et al. Section 4; Search-O1 modular domain design
Semantic vector search and retrieval	Entity embeddings with vector indexes	Search-O1 vector space architectures; Pliks et al. semantic querying
Iterative validation and refinement	Mapping confidence, labeling, and evidence tracking	Pliks et al. Section 6
Unified enterprise knowledge fabric	Schema-ontology mappings for digital asset consistency	Search-O1 unified ontology-schema architecture
Intelligent agent & conversational search support	Semantic layers plus mapping driven agent guidance	Search-O1 agent-based conversational frameworks

Primary Process Flows

Ingestion & Initial Mapping

This phase describes how a new or updated data source is introduced into the system and how the initial mapping process is triggered.

1. **Source Schema Ingestion**: The process begins when a new data source schema is ingested or an existing one is updated. This event triggers the start of the lifecycle.
2. **Process Path Selection**: The system evaluates whether the ingested schema is completely new or an update to an existing one. This decision point, or exclusive gateway, directs the flow down one of two paths:

- **Manual Mapping**: If it's a new schema, a **Knowledge Engineer** or Data Steward must manually initiate the mapping process. This approach ensures human oversight for complex, new data structures.
- **Automated Mapping**: If it's an update, the process automatically begins an automated mapping routine. This path is used for efficiency when changes are minor and can be handled by AI agents.

Domain-Specific Mapping & Guidance

Regardless of whether the process is manual or automated, the core mapping work is assisted by specialized agents to ensure accuracy and context.

3. **Mapping Assistance**: Both the manual and automated paths lead to a sub-process where the **Semantic Modeling Workbench** and its agents work to map the schema. This step is guided by a **Company Domain Agent**, a specialized AI agent trained on a specific domain. The domain agent provides critical context and guidance to ensure the mapping aligns with internal business knowledge.
4. **Mapping Execution**: The Semantic Modeling Workbench system, with the help of the domain agent,

performs the actual mapping. It links the new or updated schema to existing ontologies and identifies new relations. The output of this stage is a mapped schema ready for further processing.

Parallel Post-Mapping Tasks

Once the core mapping is complete, the process branches into several parallel tasks to propagate the new knowledge throughout the system.

5. **Concurrent Updates**: A parallel gateway activates four concurrent service tasks to update and extend various components of the semantic ecosystem:

- **Vector Database Update**: The new semantic embeddings and mappings are immediately used to update the **Vector Database**, ensuring that the knowledge graph is current for retrieval-augmented generation (RAG).
- **Semantic Model Extension**: The core semantic models and ontologies are extended with the new relationships, concepts, and mappings. This maintains a centralized and authoritative source of knowledge.

- **AI Model Training**: The new data is fed to the **AI Training System** to further refine the models, improving their ability to understand, classify, and generate insights based on the updated knowledge.
- **Monitoring Metrics Capture**: Key metrics related to the mapping process—such as success rates, time taken, and quality scores—are captured and fed to the **Monitoring & Dashboards** system for analysis.

Quality Assurance & Iteration

After all parallel tasks are complete, the system performs a critical quality check to ensure the integrity of the knowledge.

6. **Quality Assurance**: The concurrent tasks join at another gateway, and the process moves to a quality check. The system evaluates whether the mapping quality is satisfactory.
7. **Iterative Feedback Loop**: This is a key decision point in the process:

- If the mapping is **not satisfactory**, the process is routed back to the **Manual Mapping** step. A Knowledge Engineer must then review and correct the issues, re-initiating the cycle.
- If the mapping **is satisfactory**, the process proceeds to the final publishing stage.

Optional extensions: at the Quality Assurance gateway, proposed mapping changes can be tested in simulation before approval (Appendix C.1), and the causal relationships in the ontology can be checked with causal reasoning (Appendix C.2). Both are optional enhancements to steps 6 and 7, not prerequisites for MVP1.

Publishing & Release

This final phase ensures that the approved, high-quality knowledge is formally published and made available to the broader organization.

8. **Scheduled Publishing**: The process waits for a **weekly timer event**. This ensures that approved mappings are released in a controlled, predictable manner rather than immediately.
9. **Ontology Publishing**: When the timer triggers, the **Publishing & Release Management** system publishes the new and updated ontologies. This makes the approved knowledge available for consumption by data products, applications, and end-users.
10. **Completion**: The process concludes once the publishing is successfully completed.

Feature and Functional Breakdown

Functional Features

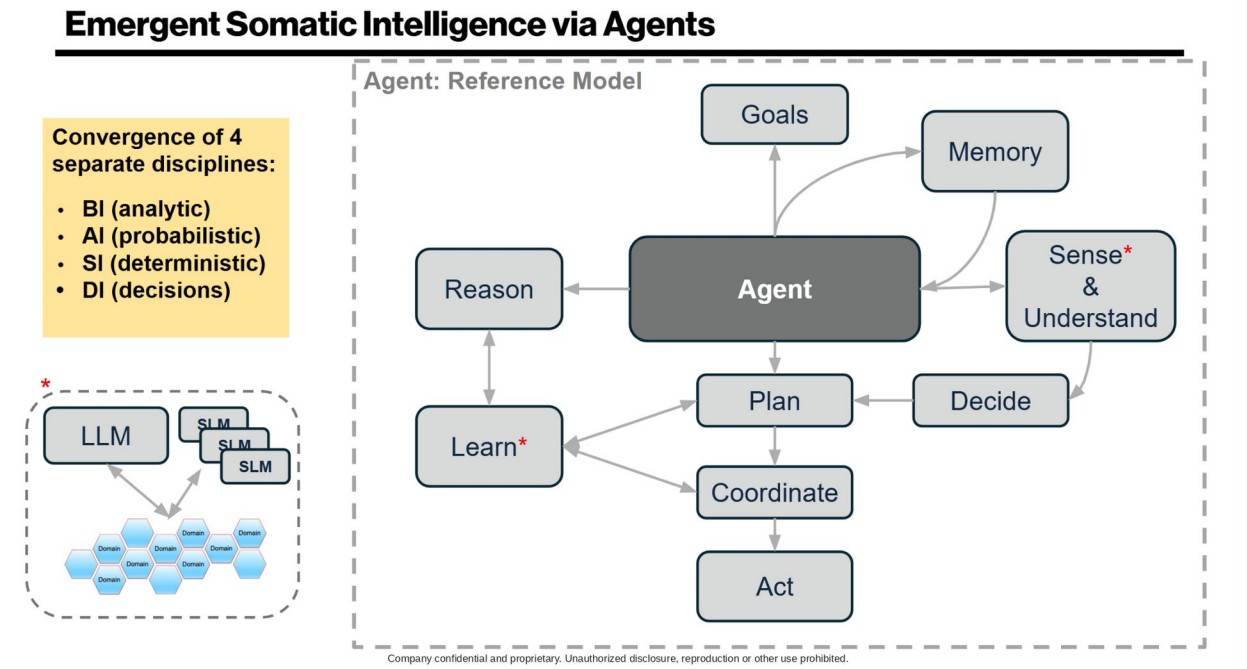
Functional features describe what the system does to support each step of the process.

- **Schema & Metadata Ingestion**: The ability to import data schemas and associated metadata from various sources (e.g., APIs, databases, files) into the platform.
- **Automated Mapping Engine**: An AI-powered service that automatically identifies, evaluates, and proposes mappings between ingested data schemas and existing ontologies.
- **Manual Mapping Interface**: A user interface that allows Knowledge Engineers to manually review, edit, and create mappings.
- **AI Agent Guidance**: A system for specialized AI agents (like the Company Domain Agent) to provide context and suggest mappings for a given domain.
- **Ontology Extension Service**: A service that validates and applies new classes, properties, and relationships to the core semantic models and ontologies.
- **Vector Embedding Generation**: The capability to convert structured data and relationships into high-dimensional vectors for use in a vector database for Retrieval Augmented Generation (RAG).
- **AI Model Fine-Tuning**: A process that updates and trains large language models (LLMs) with new data and mappings to improve their performance.
- **Dashboard & Metrics**: A user interface that visualizes key performance indicators (KPIs) related to data quality, mapping progress, and system health.
- **Publishing & Release Workflow**: An automated or semi-automated system that publishes approved ontologies and mappings to a production environment on a set schedule.

Non-Functional Features

Non-functional features describe how the system operates, focusing on its performance and reliability.

- **Scalability**: The system's ability to handle an increasing volume of schemas and mapping operations without a degradation in performance.
- **Performance**: The speed and responsiveness of the system, particularly for automated mapping, AI training, and data retrieval tasks.
- **Reliability**: The system's ability to operate continuously with minimal downtime, including robust error handling and recovery mechanisms.
- **Security & Access Control**: Features that ensure data integrity, confidentiality, and proper authorization, such as role-based access to the mapping interface and data encryption.
- **Usability**: The ease of use of the platform's interfaces, especially for Knowledge Engineers and Data Stewards responsible for mapping and quality assurance.
- **Interoperability**: The capability of the system to integrate seamlessly with various external data sources and downstream applications.



Note on the agent reference model: the LLM and SLM boxes in this diagram are logical reasoning roles. In the MVP1 deployment both roles are served by the single LLM and the domain vector partitions described in Section 2.1; physical SLMs are not part of the initial topology.

Functional Breakdown

Top-Down Knowledge Knowledge Assembly

- Core Ontologies
- DS Ontologies
- DS Search Agent
- DS Mapping Agent
- DS Vector DB
- Orchestration Agent

SG Operational Dashboard

- Demand Management (Intake)
- Lifecycle Management (all six SKL stages: ideation, development, testing, deployment, maintenance, retirement)

Lifecycle terminology: this document uses three distinct sequences. The **Semantic Knowledge Lifecycle (SKL)** has six stages (ideation, development, testing, deployment, maintenance, retirement) and governs every semantic asset (Section 2.4; Appendix D, 3.3.3). The **mapping process flow** in Section 2.3 is the operational workflow that moves a schema through those stages. The **seven delivery phases** in Appendix B are a project plan for building the conversational AI solution, not an asset lifecycle.

- Service Level Management
  - Inbound
  - Processing
  - Outbound
- Issue Management
- Capacity Management
- Business Continuity Management

AI Monitoring & Training Dashboard

- **For EACH Agent…**
- Monitoring
- Measurement
- Training (Learn = Pos + Neg)
- Transparency (Decide)
- Recommendations (Act)

SG Scheduler

- Ingestion (initial, ongoing)
- Mapping at Scale (Bottom-Up)
- Extending Knowledge at Scale
- Batch Configuration
- Orchestration & Threading

Prompt Engineering

- Sense
  - Intake
  - Reverse Engineering
- Understand
  - Relevancy (Domains)
  - Relevancy (Persona)
- Act
  - Orchestration
  - Search & Discover
  - Generate Results
    - Mapping Recommendations (Digital Schemas & Filtering)
    - Respond (tailoring and refining the results)

Vector Database Design

This design uses Sales Operations as its worked example, consistent with Section 2.2. The same schema applies to FP&A; Appendix A.9 shows the FP&A variant of the vector table.

The following is a design for building vector databases to capture domain-specific context for guiding a sales operations agent—one constrained by a domain-specific ontology and tasked with mapping ontology classes to schema ontologies for digital assets—requires an architecture that integrates several nuanced components and processes.

Core Architecture Overview

The database should be a hybrid system, combining:

- Vector DB (Domain) A vector store for semantic retrieval
- Mapping Ontology (Schema) - Relational/graph storage for explicit mappings and relations
- Metadata and embedding support for context-rich queries

Key Components and Data Flows

- **Ontology Entities Table**: Stores classes and properties from both the sales operations domain ontology and schema ontologies for digital assets. Includes fields for entity type, metadata, embedding (vector representation), and links to sub-domains.
- **Embeddings Index:** Each ontology class/property (e.g., “Lead,” “Account,” “Opportunity,” “Contract” in sales ops) is embedded into a vector space, using semantic embedding models suitable for domain-specific alignment. **This supports the approximate nearest neighbor (ANN) search for mapping discovery.**
- **Mapping Table:** Explicit links between domain ontology classes and schema ontology classes representing digital assets, with fields for:
  - Source class
  - Target class
  - Mapping confidence (numeric or categorical)
  - Mapping type (equivalent, broader, narrower) (relation from Upper Ontology)
  - Validation status and history.
- **Sub-domain Context Index**: Sales operations consists of sub-domains (e.g., lead management, quoting, order fulfillment, pricing, sales analytics). Each sub-domain has distinct classes that may overlap or intersect. The index tracks associations, context vectors, and provides scope filtering for querying and agent guidance.

Flow of Activities

1. **Entity Extraction:** Upon agent request, sales ops ontology classes are extracted and embedded. Metadata (name, description, relationships, sub-domain context) is stored.

2. **Semantic Mapping:** Matches between domain ontology classes and digital asset schema classes are proposed using vector similarity (semantic matching), then refined/ranked with syntactic, lexical, and structural features.
3. **Mapping Validation:** High-confidence mappings are validated and persisted with evidence; ambiguous mappings are flagged for review. Mappings are linked with sub-domain context for traceability.
4. **Sub-domain Navigation:** Agent queries can be scoped by sub-domain (e.g., “Order Fulfillment”), narrowing mapping search to relevant ontology sections.

Example Schema

Table	Key Fields	Purpose
Ontology_Entities	entity_id, name, description, type, embedding, subdomain	Stores ontology classes/properties with vectors and context
Schema_Entities	schema_id, name, type, description, embedding	Digital asset schema classes/properties
Entity_Mappings	mapping_id, source_entity_id, target_entity_id, confidence, type, status, evidence	Stores class-to-class mappings with context and validation
Subdomain_Index	subdomain_id, name, entities	Allows filtering and contextual search for agent queries

Sales Operations Sub-domains

Subset of “Level 1” sub-domains for the Domain (Sales Operations) can include:

- Lead Management
- Quote & Pricing
- Contract Administration
- Order Processing
- Sales Analytics
- Commission Management

Each sub-domain should have its own section of the ontology and mappings, supporting modular and scalable expansion.

The following is a design specification for integrating domain-specific ontologies with vector databases and agentic AI, built to improve accuracy, relevancy, and recall. The standard references ISO, IEEE, and OMG specifications to ensure technical rigor and interoperability.

Design Specification: Sustainable Semantic Scaffold for Agentic AI and Vector Databases

Purpose

The specification defines a modular architecture for domain-centric ontological integration with vector databases in support of agentic AI applications. The goal is to enable robust scenario-

mapping, semantic search, high recall, and precision, leveraging international standards to maximize interoperability and sustainability.

Key Steps for Integration

- Use the OMG Ontology Definition Metamodel (ODM) for designing conceptual models and mapping UML/ER schemas to OWL and RDF/JSON-LD, guided by ISO/IEC/IEEE 24765 nomenclature for consistent terminology.
- Transform ontology classes/properties into vector embeddings using formalized graph-based sequence models mapped according to recommended OMG and W3C vocabularies, extending DCAT for metadata.
- Employ the OMG Distributed Ontology, Model and Specification Language (DOL) to unify models across heterogeneous data sources, supporting cross-domain interoperability and ontology linking.
- Ensure conformance by specifying two conformance levels:
  1. *Specification-level*: Applications formally import the ontologies (owl:imports) and resolve logical consistency.
  2. *Linked Data-level*: Applications reference one or more ontologies and use standard OMG/ISO URIs.
- Maintain modularity and reuse across deployments by referencing ISO standards for naming conventions, versioning, and documentation.

Practical Strategies to Improve Search and Relevancy

- Encode ontological data as dense embeddings, mapping classes, relations, and instances into vector representations using IEEE-endorsed neural models (such as semantic encoders or graph neural networks), ensuring compatibility with OWL2/JSON-LD.
- Use the OMG Data Products Ontology (DPROD) spec to describe data products, metadata, entitlements, and data lineage for search and discovery, leveraging controls for compliance and distributed access.
- Apply RDF and DCAT vocabularies to structure entity metadata, ground semantic search, and facilitate cross-platform discovery and interoperability.
- Enable hybrid search and reranking pipelines that combine symbolic reasoning (OWL2/DOL-based inference) and vector similarity (cosine distance, reranked by ontological rules), guided by scenario-specific ontologies.
- Implement continuous integration and quality assurance for ontological objects following ISO standards for data quality, error reduction, and extensibility.

Implementation Outline

Component	Specification	Standard Ref
Ontology Design	ODM/OWL/RDF/JSON-LD for domain model and semantic mappings	OMG ODM, ISO/IEC/IEEE 24765, W3C RDF/OWL
Data Product Metadata	DCAT extension, DPROD schema for data products and lineage	OMG DPROD, W3C DCAT, ISO naming/versioning
Embedding	IEEE-endorsed deep graph encoders for semantic	IEEE ML/AI Guidance, OWL-API,

Component	Specification	Standard Ref
Procedure	vectorization	ODM
Vector DB Integration	Schema alignment, rich metadata indexing, linkage to ontological URIs	OMG ODM, ISO/IEC JTC 1/SC 7, IEEE interoperability
Hybrid Retrieval System	Pipeline for vector search reranked by ontological inference	OMG DOL, ISO error reduction standards
Quality & Compliance	Automated tests, versioning, data quality monitoring	ISO documentation, OBO Foundry practices
Scenario Mapping	Participatory ontology engineering, use-case embedding, modular workflows	OMG PSIG, domain-specific ontologies, W3C standards

Inbound Microservices, Agents, and Search

This section defines a reusable set of microservices and agents that implement the inbound semantic services in Section 1.4 for any digital asset, whether held in-house or by a third party. It supersedes the tool choices in the Section 1.4 microservices breakdown (for example, Celery and Redis for queuing) with one event backbone, Apache Kafka, and one set of naming conventions.

The design rests on a strict division of labor. **Microservices do the work**: simple, single-purpose, deterministic steps that access, extract, transform, and load an asset and log what happened. **Agents make the decisions**: which route an asset takes, how a failure is handled, when a source is paused, and when a person must decide. Search is a utility built into the Semantic Knowledge Layer: a facade of microservices that agents extend with federated graph queries.

Division of Responsibilities

	Microservices	Agents
Purpose	Perform one step the same way every time	Decide what to do when the path is not obvious
Logic	Deterministic; no business rules in code; behavior set by configuration and the request	Reason over catalog metadata, policies, history, and SKL context
State	Stateless; idempotent	Read state from events and the SKL; keep a decision log
Failure	Retry transient errors a bounded number of times, then publish a failure event	Classify the failure and choose a recovery action
How they act	Consume one command topic; publish success or failure events	Publish command events to microservices; never write to the SKL directly
Change control	Released through the software pipeline	Prompts, policies, and allowed actions versioned as knowledge artifacts (Section 3.3)

End-to-End Flow

```
receive-request → orchestration-agent (route plan)
  → locate-asset → access-asset → extract-asset → profile-asset
  → transform-asset → validate-graph → package-graph → load-graph
  → register-asset → catalog event → search-index, search-vector
  → send-notification;  track-request follows every event

on any failed event: exception-agent → retry | reroute | quarantine | escalate
on source health:    resilience-agent → pause | throttle | resume | replay
```

Naming Conventions

Names follow the Company standards in Section 3.2: full words, no acronyms, lower case with hyphens for services, and dot-separated segments for topics.

Element	Pattern	Example
Microservice	{verb}-{object}	extract-asset, load-graph
Agent	{responsibility}-agent	exception-agent, federation-agent
Command topic	knowledge.inbound.{step}.requested.v{major}	knowledge.inbound.extract.requested.v1
Event topic	knowledge.inbound.{step}.{completed | failed}.v{major}	knowledge.inbound.extract.completed.v1
Retry topic	knowledge.inbound.{step}.retry.v{major}	knowledge.inbound.transform.retry.v1
Dead-letter topic	knowledge.inbound.{step}.dead-letter.v{major}	knowledge.inbound.load.dead-letter.v1
Exception topics	knowledge.inbound.exception.{raised | resolved}.v{major}	knowledge.inbound.exception.raised.v1
Catalog events	knowledge.catalog.asset.{registered | updated | withdrawn}.v{major}	knowledge.catalog.asset.registered.v1
Consumer group	{service-or-agent-name}	transform-asset
Event type	com.insperity.knowledge.{area}.{step}.{outcome}	com.insperity.knowledge.inbound.extract.completed
Schema subject	{topic}-value	knowledge.inbound.extract.completed.v1-value
Asset graph	https://www.insperity.com/graph/asset/{source}/{asset-identifier}	https://www.insperity.com/graph/asset/benefits-portal/plan-guide-2026

- **Versions apply to contracts, not names:** the topic suffix is the major version of the event schema. A breaking schema change creates a new topic version and both run in parallel until consumers move, consistent with the versioning rules in Section 3.3.
- **One envelope for every event:** events use the CloudEvents specification (id, source, type, subject, time, data schema) with three Company extensions: correlationid, assetiri, and traceparent (W3C Trace Context).
- **Partition key is the source:** events are keyed by source system, which keeps ordering per source and isolates a misbehaving source from the others.
- **Claim check for content:** asset content never travels on Kafka. Extracted content is written to a staging object store; the event carries its location and checksum.

Microservice Catalog

Twelve microservices cover the inbound flow in Section 1.4. Each consumes its own command topic and publishes a completed or failed event; the step name in each topic matches the service.

Microservice	Does	Section 1.4 steps covered
receive-request	Accepts requests from an API, a schedule, or a source change notice; assigns the correlation identifier	Request; Scheduler
locate-asset	Resolves the asset's identifier, location, and access point from the source registry and catalog	Search for Digital Asset; Search Catalog; Obtain URL/URI; Obtain Access Point
access-asset	Obtains a short-lived access grant from the secrets vault; passes a reference, never the credential	Obtain Authorization; Obtain Credentials

Microservice	Does	Section 1.4 steps covered
extract-asset	Retrieves content and technical metadata through a connector adapter; writes it to staging with a checksum	Extract Digital Asset
profile-asset	Detects media type, schema, version, timestamps, and the attribute list	Obtain Schema, Version, & Datetime Stamp; Obtain Attributes
transform-asset	Applies the approved mapping (RML for structured sources; metadata extraction for documents) and adds the metadata template (DCAT, Dublin Core, PROV-O)	Assemble Inputs; Generate RDF Structure; Add Metadata for Ontology Template
validate-graph	Runs the SHACL shapes for the asset type and publishes the validation report	Validate RDF Structure
package-graph	Serializes the asset's named graph as TriG with its manifest	Create .TRIG file
load-graph	Replaces the asset's named graph in the SKL in one transaction; records the asset graph version	Load Digital Asset; Validate (SPARQL)
register-asset	Updates the asset catalog entry and publishes a catalog event for search and vector indexing	Close out Request
send-notification	Sends notices to people and systems through approved channels	Notify SG Operations
track-request	Maintains request status from the event stream for dashboards and the workflow queue	Update SG Workflow; Inbound Queue

Inbound loads are not ontology releases. The inbound flow loads descriptions of assets (instance data) into asset graphs, each with its own version. Changes to ontologies, taxonomies, or shapes discovered during ingestion are raised as change requests and follow the release pipeline in Section 3.3; the inbound flow never modifies them directly. This replaces the "increment ontology version" step in Section 1.4.

Standard Service Contract

- **Single purpose:** one command topic in; one completed and one failed topic out.
- **Idempotent:** the idempotency key is the correlation identifier plus the step plus the content checksum, so a repeated event produces the same result once.
- **Bounded retries:** transient errors are retried up to three times with exponential back-off and jitter; any other failure is published immediately as a failed event.
- **Classified failures:** every failed event carries an error category (transient, access, data, contract, or unknown), an error code, and a reference to the input.
- **Same image everywhere:** one container image per service version, promoted through development, test, stage, and production with configuration-only differences.
- **Observable:** health and readiness endpoints, metrics, and OpenTelemetry traces linked by the traceparent value in each event.

Standard Logging

Every service and agent writes structured JSON logs with the same fields, provided by a shared logging library:

Field	Content
timestamp, level	Time in UTC (ISO 8601) and severity
service, serviceVersion, environment	Who wrote the entry, which release, and where
correlationId, requestId, assetIri	Links the entry to the request and the asset
step, eventType, outcome	What was attempted and whether it completed, failed, or was skipped
durationMilliseconds	Time taken by the step
errorCategory, errorCode, errorMessage	Present on failures only
traceId, spanId	Links to the distributed trace
decisionId, rationale	Agents only: the decision taken and why

- **Never logged:** asset content, credentials or tokens, and personal data. Logs reference the asset by IRI and checksum.
- **Provenance, not just logs:** completed loads also produce a PROV-O record in the provenance graph, linking the asset graph to the source, the services, and the mapping version used, so lineage is queryable alongside the change log (Section 3.3).

Agent Catalog

Agent	Decides	Acts by
orchestration-agent	The route for each asset: which steps, which connector, which mapping and shapes, based on source, asset type, and policy	Publishing the first command with a route plan; reuses an approved plan for known asset types without reasoning again
exception-agent	How to recover from a failed event	Choosing one action from the allowed list below and publishing the matching command or exception event
mapping-agent	How an unfamiliar schema maps to SKL classes and properties, using labels and definitions (Section 3.8)	Proposing a mapping for human review; approved mappings are stored in the mapping registry and reused
validation-agent	Whether a SHACL violation is a correctable data issue (for example, a date format) or a real defect	Requesting a corrected transform, or raising a data defect to the source owner
governance-agent	Data classification, access restrictions, retention, and licensing terms for each asset, especially third-party assets	Setting the asset graph's access labels and retention; blocking assets that fail policy
resilience-agent	When a source or downstream system is unhealthy, and how much load each source can take	Pausing and resuming a source's consumers (circuit breaker), throttling to third-party rate limits, and scheduling replays

Agent Guardrails

- **Act through events only:** agents publish commands to microservices; they never call source systems or write to the SKL themselves.

- **Allowed actions are a closed list,** versioned under change control. An agent cannot invent a new kind of action.
- **Confidence thresholds:** below a set confidence an agent escalates to a person instead of acting; thresholds are set per action.
- **Every decision is logged** with its inputs, the action taken, the rationale, and the model and prompt versions, so it can be audited and replayed.
- **Feedback improves future decisions:** outcomes of agent decisions, and human overrides, are fed back into the approved route plans, mappings, and thresholds (the recursive learning loop in Section 3.6).

Exception Handling and Resiliency

1. A microservice retries a transient error up to three times, then publishes a failed event.
2. The exception-agent classifies the failure using the error category, the source's recent history, and the asset's route plan.
3. It chooses one allowed action: retry later, retry with alternate parameters, reroute (alternate connector or mapping), quarantine the asset, skip a non-critical attribute and continue, or escalate.
4. Escalations create an issue in run-time issue management (Section 3.6) with the full event history attached.
5. Events still failing after the allowed actions go to the step's dead-letter topic; nothing is silently dropped.
6. When the cause is fixed, the resilience-agent replays affected events from Kafka retention or the dead-letter topic.

Pattern	Provided by	Purpose
Idempotent consumers	Every microservice	Safe retries and replays
Retry with back-off and jitter	Every microservice	Absorb brief outages without overloading the source
Circuit breaker per source	resilience-agent	Stop calling a failing source; resume after a health check
Bulkhead by source	Partitioning and consumer configuration	One slow or failing source cannot block the others
Rate limiting	resilience-agent with access-asset	Respect third-party limits and contractual terms
Claim check	extract-asset	Large content stays off the event backbone
Quarantine and dead-letter	exception-agent	Isolate bad assets for review without stopping the flow
Replay	resilience-agent	Reprocess from retained events after a fix

Search as an SKL Utility

Content stays where it lives, in Company repositories and third-party systems. The SKL holds an RDF description of each asset: what it is, which concepts, process steps, products, and clients it is

about, who owns it, how it is classified, and where it can be accessed (DCAT access and download links). Search finds assets by querying these descriptions and then resolves each result to its source location, applying the user's entitlements.

```
@prefix asset: <https://www.insperity.com/resource/asset/> .

asset:benefits-portal-plan-guide-2026 a dcat:Dataset, core:DigitalAsset ;
    dcterms:title        "2026 Medical Plan Guide"@en ;
    dcterms:subject      benefits:MedicalPlan, benefits:OpenEnrollment ;
    core:supportsStep    benefits:EnrollInBenefitsStep ;
    core:hasClassification core:Internal ;
    core:sourceSystem    source:BenefitsPortal ;
    dcat:distribution    [ a dcat:Distribution ;
        dcat:accessURL   <https://benefits-portal.example/documents/plan-guide-2026> ;
        dcat:mediaType   <https://www.iana.org/assignments/media-types/application/pdf> ] ;
    prov:wasGeneratedBy  provenance:load-2026-10-13-0042 .
```

Search Facade Microservices

Microservice	Does
search-gateway	The single search interface for applications and agents: keyword, faceted, semantic, and graph queries; applies the caller's entitlements before anything else
search-index	Full-text and faceted search over asset descriptions, kept current from catalog events
search-vector	Similarity search over the domain vector partitions (Section 2.5)
query-graph	Runs SPARQL against the SKL, including federated queries (SPARQL SERVICE) to other graphs and virtual graphs over relational sources
query-external	Calls third-party search interfaces through adapters that translate results into the SKL vocabulary
merge-results	Combines and de-duplicates results from all sources by asset IRI and ranks them
resolve-asset	Turns a result into a usable link to the source, after checking the user's access at the source

Search Agents

Agent	Extends search by
query-planning-agent	Turning a question into a graph query using concept labels, persona vocabulary (Section 3.8), and process-step context
federation-agent	Choosing which graphs, endpoints, and third-party sources to include, based on the catalog, source health, cost, and the user's entitlements; plans the federated query
result-explanation-agent	Explaining why each result matched, with the path through the graph and its provenance, for people and for AI answers

- **Filter first, then rank:** entitlement filtering happens in search-gateway before any query is planned, so agents only ever see what the user may see (Section 3.8).
- **The facade is the only entry point:** applications and agents use search-gateway rather than calling the index, vector store, or graph directly, so security, logging, and ranking stay consistent.

- **Extensible by configuration:** adding a new repository means registering a source, a connector adapter, and a mapping; the facade, agents, and conventions do not change.

Design

Knowledge engineering at the Company follows four design principles. A small **core ontology** gives every domain the same foundation; a **unifying ontology** makes the process step the anchor for every fact; **domain ontologies** add detail inside a domain; and **operational domains** connect domains to one another. OWL 2 is used in a deliberately simple way: deep subclass hierarchies carry the meaning, so little inference is needed at run time.

Core Ontology

The core ontology is small, stable, and owned by Architecture. It defines the concepts every domain shares: Process, Process Step, Service, Asset, Product, Party, Role, Location, Time, Event, Issue, and Measure. It is aligned to Basic Formal Ontology (ISO/IEC 21838-2), so its top-level distinctions (things, processes, qualities, roles) follow an international standard. The core never imports domain terms; domains extend the core, never the reverse.

Unifying Ontology: the Process Step as First-Class Citizen

The unifying ontology makes the **process step** the anchor for every other concept. Instead of modeling data around tables or systems, each fact is attached to the step in which it is created, used, or governed:

Relationship	Meaning	Example
performedBy	The party or role that carries out the step	Approve payroll run is performed by Payroll Manager
consumes / produces	Assets the step reads or creates	Calculate gross pay consumes Timesheet, produces Pay Statement
triggers / triggeredBy	Events that start or result from the step	Employee termination triggers Final pay calculation
governedBy	Rules, constraints, and legal norms that apply to the step	Final pay calculation is governed by State final pay law
measuredBy	Metrics that evaluate the step	Payroll run is measured by On-time payment rate
precedes / follows	Order of steps within a process	Validate timesheet precedes Calculate gross pay

Because every system, document, rule, and metric is linked to a process step, the graph can answer impact questions directly: what breaks if this rule changes, which reports use this field, which steps a regulation touches.

Domain Specificity: Intra-Domain Edges and Properties

A domain ontology (for example, Payroll, Benefits, or Human Resources Compliance) specializes core classes and defines the relationships that stay **inside** the domain. Domain subject-matter experts own these definitions.

- **Subclasses:** Payroll Run is a subclass of Process Step; Pay Statement is a subclass of Asset.

- **Intra-domain properties:** relationships whose domain and range are both in the same domain, such as Payroll Run hasPayPeriod Pay Period.
- **Domain vocabulary and taxonomy:** preferred terms, synonyms, and category hierarchies for the domain, linked to the ontology classes they describe.

Operational Domains: Inter-Domain Edges and Properties

Real work crosses domains: a benefits enrollment changes a payroll deduction; an onboarding step creates a compliance obligation. These **inter-domain** relationships are defined in a separate **operational** module that imports the domains it connects, so no domain has to import another.

- **Inter-domain properties:** for example, Benefits Enrollment Step determinesDeduction Payroll Deduction, or Onboarding Step createsObligation Compliance Requirement.
- **Joint ownership:** changes to the operational module need approval from every domain it connects, plus Architecture.
- **Concept alignment:** where two domains use similar terms, each maps its term to the core through a reviewed SKOS bridge (Section 3.5); domains never map directly to each other.

Use of OWL 2: Maximize Subclasses, Minimize Inferencing

Meaning is carried by an explicit, deep subclass hierarchy. Properties, constraints, and documentation are attached as high in the hierarchy as they are true, so every subclass inherits them. This keeps the ontology predictable, fast to query, and easy for people to read, and it avoids reasoning that is expensive or hard to explain.

Prefer	Avoid	Why
Asserted rdfs:subClassOf hierarchies	Defined classes (owl:equivalentClass with complex restrictions)	Asserted classification is transparent and needs no reasoner to compute
Properties with simple domain and range	Long property chains and inverse-functional tricks	Simple properties are inherited cleanly and query quickly
SHACL shapes for data constraints	OWL restrictions used as data validation	OWL is open-world; SHACL checks data the way the business expects
OWL 2 RL- or EL-friendly constructs	Full OWL 2 DL expressivity and SWRL rules	Keeps reasoning tractable inside GraphDB rulesets
Materialized hierarchy at publish time	Run-time inference for every query	Answers stay fast and repeatable

Standards

All knowledge artifacts follow one set of naming and identifier conventions. Every identifier lives under the base URL **https://www.insperity.com**, and names are spelled out in full: **acronyms are not used** in identifiers or preferred labels.

Naming Conventions

Artifact	Element	Convention	Example
Vocabulary	Term	Preferred label in sentence case; synonyms as alternative labels	"Pay period" (alternative: "Payroll cycle")
Taxonomy	Concept	UpperCamelCase identifier; preferred label in sentence case; codes in skos:notation	WagesAndHours, label "Wages and hours"
Ontology	Class	UpperCamelCase singular noun	PayrollRun, WorksiteEmployee
Ontology	Object property	lowerCamelCase verb phrase	performedBy, governedBy, hasPayPeriod
Ontology	Data property	lowerCamelCase noun phrase with has- or is- prefix	hasEffectiveDate, isExempt
Ontology	Individual	UpperCamelCase name, or class name plus stable identifier	TexasMinimumWageAct, Worksite-000123
Shapes	Shape	Target class name plus Shape	PayrollRunShape

Avoiding Acronyms

- **Identifiers and preferred labels use full words:** ProfessionalEmployerOrganization, not PEO; FamilyAndMedicalLeaveAct, not FMLA.
- **Acronyms are recorded only as alternative labels** (skos:altLabel), so search still finds them.
- **Appendix F lists approved expansions;** a new acronym is added there before it is used in any label.

URI Conventions

URIs are persistent, readable, and never reused. Slash-based namespaces are used so each term can be resolved on its own. Core and domain elements follow the same pattern, with the domain name spelled out in lower case:

Element	Pattern	Example
Core ontology	https://www.insperity.com/ontology/core/{Term}	https://www.insperity.com/ontology/core/ProcessStep
Domain ontology	https://www.insperity.com/ontology/{domain}/{Term}	https://www.insperity.com/ontology/payroll/PayrollRun
Operational module	https://www.insperity.com/ontology/operational/{term}	https://www.insperity.com/ontology/operational/determinesDeduction

Element	Pattern	Example
Vocabulary	https://www.insperity.com/vocabulary/{domain}/{term}	https://www.insperity.com/vocabulary/payroll/pay-period
Taxonomy	https://www.insperity.com/taxonomy/{domain}/{Concept}	https://www.insperity.com/taxonomy/compliance/WagesAndHours
Mapping (bridge)	https://www.insperity.com/mapping/{domain}/{term}	https://www.insperity.com/mapping/benefits/enrollment
Shapes	https://www.insperity.com/shapes/{domain}/{Shape}	https://www.insperity.com/shapes/payroll/PayrollRunShape
Instance data	https://www.insperity.com/resource/{domain}/{Class}/{id}	https://www.insperity.com/resource/payroll/PayrollRun/2026-10-15
Version	https://www.insperity.com/ontology/{domain}/{version}/	https://www.insperity.com/ontology/payroll/2.1.0/

```
@prefix core:    <https://www.insperity.com/ontology/core/> .
@prefix payroll: <https://www.insperity.com/ontology/payroll/> .

payroll:PayrollRun a owl:Class ;
    rdfs:subClassOf core:ProcessStep ;
    rdfs:label "Payroll run"@en ;
    skos:definition "A scheduled process step that calculates and issues pay
                     for one pay period."@en .
```

Change Control

Every change to a knowledge artifact is traceable to an approved requirement, tested before release, and published with a version. Change records are stored as triples in a dedicated change log in the same knowledge graph, so history can be queried and joined to the artifacts it describes. Releases are built once from source control and promoted unchanged through development, test, stage, and production, so no environment is ever edited by hand.

Versioning

- **Semantic versioning:** MAJOR for changes that break consumers (removing or redefining a class or property), MINOR for additions, PATCH for label and documentation fixes.
- **Diff-based classification:** a schema-diff check in the build pipeline determines the required version bump, and the build fails if the declared version does not match.
- **Version identifiers:** each release carries owl:versionIRI and owl:priorVersion, and consumers declare the version range they depend on.
- **Version in identifiers:** only ontology and vocabulary releases receive versioned IRIs (owl:versionIRI), because consumers must be able to import or cite an exact release. Class, property, and concept IRIs never contain a version: their meaning is stable across releases, and a versioned term IRI would break every mapping, stored fact, and link at each release. When a term's meaning changes, it is deprecated and replaced by a new term.

Change Records and the Change Log

Each change records what changed, why, and which approved requirement feature it implements (see the ORSD in Appendix A.4), together with the KGCL command that describes the change (Appendix A.5). Three placements for these records were considered:

Option	Strengths	Weaknesses	Decision
Separate RDF store	Isolates volume and retention; separate access control	Every lineage question needs a cross-store join; two stores to synchronize	Only for high-volume run-time events (Section 3.6)
Same store, dedicated named graphs	One SPARQL query joins a change to the artifacts it touched; append-only; simple reporting	Needs graph naming and access discipline	Adopted for design-time change records
Extensions on existing elements	Current state is easy to read	History accumulates on every term; current and past values mix; records become editable	Limited to a light pointer on each element

Change Log Structure

- **Change records are resources of their own,** stored in the append-only change log graph (https://www.insperity.com/graph/change-log). A record is never edited; a correction is a new record that references the one it corrects.
- **One record per change set, not per triple:** a change record describes a reviewed unit of work (one pull request). The exact triples it added and removed are kept in two small named graphs

per change set and referenced from the record, which keeps the log compact and the detail available.

- **Links, not copies:** a record points to the artifacts it affects by their stable IRIs, to the feature it implements, to the release it ships in, and to the people who made and approved it (PROV-O).
- **A light pointer on each element:** artifacts carry only dcterms:modified, owl:versionInfo, and optionally core:latestChange. The full history stays in the change log.
- **Automatic tracking is a safety net only:** triple-level history kept by the triple store, where enabled, does not replace governed change records, because it cannot record why a change was made or who approved it.

```
@prefix change: <https://www.insperity.com/change/> .
@prefix req:    <https://www.insperity.com/requirement/> .
@prefix person: <https://www.insperity.com/resource/core/Person/> .

GRAPH <https://www.insperity.com/graph/change-log> {
  change:2026-0412 a core:ChangeRecord, prov:Activity ;
    dcterms:description    "Rename Budget Allocation to Budget Distribution."@en ;
    core:implementsFeature req:FinancialPlanning-Feature-017 ;
    core:changeType        kgcl:NodeRename ;
    core:changeCommand     "rename finance:BudgetAllocation from 'Budget Allocation'
                            to 'Budget Distribution'" ;
    core:affectsArtifact   finance:BudgetAllocation ;
    core:versionBump       "MINOR" ;
    core:addedTriples      change:2026-0412-added ;
    core:removedTriples    change:2026-0412-removed ;
    core:pullRequest       <https://source-control.example/knowledge/pull/412> ;
    prov:wasAssociatedWith person:knowledge-engineer-04 ;
    core:approvedBy        person:finance-domain-owner ;
    prov:endedAtTime       "2026-10-13T14:02:00-05:00"^^xsd:dateTime ;
    core:releasedIn        finance:2.4.0 .
}
```

Reporting, Traceability, and Lineage

- **Traceability and lineage** are answered in the graph with SPARQL: from a requirement to its changes, releases, and deployments, or from a term back to every change and approval that shaped it.
- **Operational reporting** (change volume, cycle time, approval lead time, defects per release) is fed on a schedule from the change log to the enterprise analytics warehouse, so dashboards use the standard business intelligence tools.
- **Explicit statements only:** change review and lineage queries read authored statements, never inferred ones (Section 3.9).

Lifecycle Management

Artifacts move through explicit states: **Draft → Candidate → Released → Deprecated → Retired**. Released versions are immutable. Deprecated versions carry a sunset date, and terms are deprecated rather than deleted so history is kept. These states run inside the ten-stage governance lifecycle in Section 3.4, which refines the six Semantic Knowledge Lifecycle stages used elsewhere in this document.

Lifecycle states map to environments: **Draft** artifacts exist only in source control and development; **Candidate** releases are deployed to test and stage; **Released** versions run in production.

Testing and Impact Assessment

- **Competency questions:** each requirement's competency questions run as SPARQL tests and must return the expected answers.
- **Consistency and quality:** a reasoner checks logical consistency, and SHACL shapes check data quality and naming conventions.
- **Impact analysis:** dependency queries list every mapping, rule, service, report, and agent that uses the changed terms, using the process-step links from Section 3.1.
- **Consumer compatibility:** the build checks every consumer's declared version range against the new release.

Release Pipeline and Environment Promotion

Knowledge artifacts are released the way software is: from version-controlled source, built once into a fixed package, and promoted through environments by an automated pipeline. Changes are never copied or re-applied from one environment to another, which removes replication and drift problems at the source.

Principles

- **Source control is the source of truth.** Ontologies, taxonomies, vocabularies, SHACL shapes, rules, persona definitions, and change records are kept as Turtle and KGCL files in source control. Every change enters through a reviewed pull request.
- **Build once.** A tagged commit is built into one immutable release package: a TriG file with one named graph per artifact, the version IRIs, the change records, and a manifest with a checksum for each graph.
- **Promote the same package.** Development, test, stage, and production receive the identical package; only the configuration (endpoint, credentials, data sources) differs.
- **Replace, do not patch.** A deployment makes each target graph equal to the release ("make these graphs equal release 2.4.0"), never "apply these forty changes". Deployments can be repeated safely, and a failed deployment is simply rerun.
- **Same IRIs everywhere.** Identifiers are identical in every environment; an IRI names a thing, not a server. Only the endpoint an application connects to changes.
- **KGCL is for review, not transport.** Change commands explain and classify changes for reviewers and release notes; they are not the mechanism that moves changes between environments.

Pipeline Stages

Stage	Environment	What happens	Gate to proceed
1. Author	Development	Knowledge engineers change artifacts in the workbench, export to source control, and open a pull request with its change record	Pull request complete; change record linked to an approved feature
2. Build	Pipeline	Validate syntax, naming, and SHACL; run competency questions; classify the version bump; assemble the release package	All automated checks pass; declared version matches the diff

Stage	Environment	What happens	Gate to proceed
3. Test	Test	Deploy the package; run consumer contract tests, impact queries, and search and AI evaluation sets on synthetic or masked data	No failed tests; impact assessment reviewed
4. Stage	Stage	Deploy the identical package against production-like data volumes; rehearse the rollback	Human approval by the owners in the Human in the Loop table
5. Release	Production	Deploy the identical package; switch over; publish; send release notes	Post-deployment checks pass; deployment record written

What Is Promoted and What Is Not

Content	Promoted?	How each environment gets it
Ontologies, taxonomies, vocabularies, shapes, rules, personas	Yes	From the release package
Change records	Yes	In the release package; identical in every environment
Instance data	No	Loaded from each environment's own sources: synthetic or masked data in development and test, systems of record in production
Inferred statements	No	Recomputed by each environment's ruleset after loading; never copied
Deployment records	No	Written by the pipeline in each environment

Deployment Records

History is split into two kinds. **Change records** say what changed and why; they are authored once and travel with the release. **Deployment records** say what arrived where and whether its checks passed; each environment writes its own in a local deployment log graph. Lineage then runs from requirement to feature, change record, release, and deployment, and a single query answers what is running in production and why.

```
@prefix deploy: <https://www.insperity.com/deployment/> .

GRAPH <https://www.insperity.com/graph/deployment-log> {
  deploy:stage-2026-10-13-01 a core:DeploymentRecord, prov:Activity ;
    core:deployedRelease   finance:2.4.0 ;
    core:targetEnvironment core:StageEnvironment ;
    core:packageChecksum   "sha256:9f2c…41ab" ;
    core:validationResult  core:Passed ;
    core:pipelineRun       <https://pipeline.example/knowledge/runs/1187> ;
    prov:startedAtTime     "2026-10-13T14:05:00-05:00"^^xsd:dateTime ;
    prov:endedAtTime       "2026-10-13T14:10:00-05:00"^^xsd:dateTime .
}
```

Safe Loading and Rollback

1. Load the release into new graphs, or a standby repository, alongside the live content.
2. Run SHACL validation and smoke-test queries against the newly loaded content.

3. Switch over in one step: replace the live graphs in a single transaction, or point the service to the standby repository. If any check fails, the live version is untouched.
4. Recompute inferred statements, refresh search indexes and domain vector partitions, and write the deployment record.
5. Roll back by redeploying the previous release package; no reverse changes are written by hand.

Preventing Drift

- **No direct edits:** people have read-only access to stage and production knowledge graphs; only the pipeline writes to them.
- **Urgent fixes use the same path:** a hotfix follows the same pipeline on an expedited approval track, with its own change record and release.
- **Nightly drift check:** a scheduled job compares each environment's graph checksums with its release manifest and opens an issue on any mismatch (Section 3.6).

Release Cadence

Changes are bundled into scheduled releases (weekly for MINOR and PATCH; planned and announced for MAJOR). Each release has release notes generated from its change records, a rollback plan, and a sign-off record.

Human in the Loop

Change touches	Required approvals
A domain ontology, vocabulary, or taxonomy	Domain subject-matter expert, plus automated checks
The core ontology	Architecture, plus every affected domain owner
A mapping (bridge) or operational module	Each connected domain owner and Architecture
Legal or regulatory norms	Legal or Compliance reviewer
Agent-proposed mappings or extensions	Knowledge Engineer review before any change becomes authoritative

Publishing

After the production deployment succeeds, released artifacts are registered in the metadata registry (ISO/IEC 11179) and published at their persistent URIs, with human-readable pages and machine-readable downloads (Turtle, JSON-LD), a SPARQL endpoint, and the Semantic Modeling Workbench. Search indexes and domain vector partitions are refreshed from the same release.

Communications

- **Release notes** to all subscribers of the affected artifacts, generated from change records.
- **Deprecation notices** with sunset dates and replacement terms, sent at least one release before retirement.
- **Change calendar** showing upcoming MAJOR releases and their impact assessments.

Governance

Knowledge engineering is governed against Company policy and against the international standards catalogued in the **PEO Standards & Compliance Map** and its guide. No single ISO standard covers the whole knowledge lifecycle; ISO/IEC 5392:2024 (reference architecture for knowledge engineering) is the umbrella, and each lifecycle stage is guided by the standards below.

Adherence to Policies

- **Company policies:** data classification, privacy, records retention, information security, and responsible use of AI apply to every knowledge artifact and service.
- **Quality gates:** SHACL release gates enforce naming, provenance, and verification rules; an artifact that fails a gate cannot be released.
- **Verification status:** every reference fact (for example, a law or wage rate) records whether it has been checked against a published source, and unverified facts are reported as warnings, not hidden.
- **Audit:** change records, approvals, and release sign-offs are kept in the graph and reviewed each quarter.

Governance Lifecycle and Guiding Standards

Stage	What happens	Guiding standards
1. Scope and competency questions	Agree what the knowledge must answer, for whom, and who owns it	ISO/IEC 5392, ISO 30401, ISO/IEC 42001
2. Acquire knowledge and provenance	Collect authoritative sources and record where each fact came from	ISO/IEC 5392, ISO 30401, W3C PROV
3. Define terms	Preferred terms, intensional definitions, term metadata	ISO 704, ISO 1087, ISO 12620-1, ISO 30042, ISO 26162-1, ISO 30400
4. Organize concepts	Hierarchies, associative relations, mappings to external vocabularies	ISO 25964-1 and -2, W3C SKOS
5. Formalize the ontology	Classes, relations, and axioms aligned to a top-level ontology	ISO/IEC 21838-1, -2 (BFO), -5, ISO/IEC 5392, W3C OWL 2
6. Validate	Reasoner consistency, SHACL data quality, expert and legal review	ISO/IEC 5259, W3C SHACL
7. Register and publish	Record each artifact, version, and status in a metadata registry	ISO/IEC 11179-3, -32, -35; ISO/IEC 19763-1, -3
8. Deploy as persistent or virtual graphs	Load into a store, or expose existing databases as a graph	ISO/IEC 39075 (GQL), ISO/IEC 9075-16 (SQL/PGQ), W3C SPARQL, W3C R2RML
9. Operate, protect and measure	Access control, AI impact assessment, quality monitoring, reporting	ISO/IEC 27701, ISO/IEC 42001, ISO/IEC 42005, ISO/IEC 5259, ISO 30414
10. Change, version and retire	Track changes, issue versions, deprecate rather than delete, retain history	ISO 30300, ISO/IEC 11179-3, ISO 30401

Six-Stage and Ten-Stage Lifecycles

The ten stages refine the six Semantic Knowledge Lifecycle stages used in Section 2.4 and Appendix D: Ideation = stages 1–2; Development = stages 3–5; Testing = stage 6; Deployment = stages 7–8; Maintenance = stage 9; Retirement = stage 10.

Engineering, Requirements, and Quality Standards

Standard	Applies to
ISO/IEC/IEEE 29148 (requirements engineering)	Ontology requirements and competency questions (Appendix A.4)
ISO/IEC/IEEE 24765 (systems and software engineering vocabulary)	Engineering terminology in specifications
ISO/IEC/IEEE 42010 (architecture description)	Architecture views of the knowledge fabric and services
ISO 30401 (knowledge management systems)	The knowledge management system as a whole
ISO/IEC 42001 and ISO/IEC 42005 (AI management and impact assessment)	Agents, search, and AI services that use the graph
ISO/IEC 27701 (privacy information management)	Worker and client data exposed through the graph

The Standards & Compliance Map records the edition and status of each standard (current, under review, draft, or withdrawn); it is the reference for which edition applies.

Single Source of Truth

Business logic is often scattered across code, database triggers, rules-engine screens, and ontologies, and the same rule drifts apart in each place. The Single Source of Truth (SSOT) defines every rule, constraint, logic component, and calculation **once**, in a declarative specification under version control, and generates what each runtime needs from it.

The Right Home for Each Kind of Logic

Category	Governing question	Authored as	Runs in	Example
Constraints	What must always be true about the data?	Ontology terms and SHACL shapes, or JSON Schema	Graph store, API validation	A payroll run must have exactly one pay period
Rules	What decision follows from the current state?	DMN decision tables	Rules engine behind the Decision API	Overtime applies above 40 hours unless the employee is exempt
Logic	How is a multi-step process carried out?	Code, referencing SSOT terms and versions	Microservices	Orchestrating the inbound pipeline
Calculations	How is a value computed?	Declarative formula specification with test vectors	Generated code in services; documented in the graph	Gross pay = hours × rate + overtime premium

Governance Pipeline

- **Nothing is edited in production:** every change is committed to the SSOT repository and reaches production only through continuous integration.
- **Automated checks:** a reasoner checks ontology consistency, DMN tables are checked for gaps and overlaps, and calculations run against their test vectors.
- **Generated artifacts:** each merge generates SHACL/OWL files for the graph store, DMN files for the rules engine, and typed interfaces for microservices, so they cannot drift apart.
- **Decision API:** services call one Decision API, which resolves the policy version they declare and routes to the backend that owns the decision, including wrapped legacy engines during migration.

Core and Domain Ownership

The repository is split into a core namespace and one namespace per domain, mirroring the core and domain ontologies in Section 3.1. Domains extend the core by composition and cannot redefine it. Ownership is enforced by review routing: domain changes need the domain owner's approval; core and mapping changes also need Architecture.

Interfaces for Subject-Matter Experts

- **Constraints:** a data-dictionary form (field, type, required, allowed values), never raw SHACL or JSON Schema.
- **Rules:** DMN decision tables, edited as grids.
- **Logic and calculations:** owned by engineers; experts see a read-only trace of which services use each rule or constraint, and approve calculation test vectors.

Choosing Storage

Structural constraints and taxonomies can be stored as versioned rows in a relational registry. A graph store is used where cross-domain semantic reasoning or process-step traceability is required, which is the case for the core, unifying, and operational ontologies. SWRL is not used; inference-style business rules belong in DMN.

Target Operating Model

The target operating model describes how knowledge engineering runs day to day: how knowledge artifacts and the services that use them (inbound, processing, outbound) are monitored, measured, traced, corrected, and improved.

Traceability

Every link in the chain below is stored as triples, so any item can be traced forward to its effects or back to its origin:

```
Requirement → Feature → Change record → Release → Deployment (dev → test → stage → prod)
            → Service (inbound / processing / outbound) → Monitored outcome
            → Defect or issue → New requirement
```

- **Knowledge artifacts:** vocabularies, taxonomies, ontologies, mappings, shapes, rules, and calculations.
- **Services:** inbound ingestion and mapping (Section 1.4), processing and enrichment, and outbound search, conversational AI, and APIs.

Monitoring

- **Artifact health:** SHACL violations, unverified facts, orphan terms with no path to the core, and deprecated terms still in use.
- **Service health:** availability, latency, error rates, queue depth, and freshness of each inbound source.
- **AI and search behavior:** answer traceability, hallucination flags, low-confidence mappings, and zero-result searches.

Measurement

Measure	What it shows	Initial target
Mapping coverage	Share of inbound schema elements mapped to ontology terms	90% for in-scope domains
Mapping accuracy	Share of agent-proposed mappings accepted on review	80% or higher
Competency question pass rate	Requirement tests passing in each release	100% before release
Change lead time	Approved requirement to released artifact	Two weeks or less for MINOR changes
Defect escape rate	Design-time defects found after release	Below 5% of changes
Search success	Searches returning a result the user accepts	85% or higher
Answer traceability	AI answers that cite governed terms and mappings	100%

Targets are starting values to be calibrated during the first release.

Human in the Loop

- **Design time:** approvals defined in Section 3.3 for every change to core, domain, mapping, and legal artifacts.
- **Run time:** review queues for low-confidence mappings, agent-proposed extensions, and AI answers flagged by users.
- **Escalation:** any item affecting legal norms or worker data is routed to Legal, Compliance, or Privacy.

Defect Management (Design Time)

A **defect** is a fault in a knowledge artifact found before or after release: a wrong definition, a broken mapping, a missing constraint, or a failed competency question.

1. **Log** the defect against the artifact and version, with the failing test or review note.
2. **Triage** severity (blocking, major, minor) and assign the artifact owner.
3. **Fix** through a normal change record, tied to the defect and the original requirement.
4. **Verify** with a new or updated test so the defect cannot recur, then release.

Issue Management (Run Time)

An **issue** is a problem observed in operation: a failed ingestion, a wrong search result, an agent error, or a breached service level.

1. **Detect** through monitoring or a user report, and record the affected service and request.
2. **Restore** service first (retry, roll back, or fall back to the previous release).
3. **Find the root cause** using traceability: which artifact version, rule, or mapping produced the result.
4. **Raise a defect** when the cause is a knowledge artifact, so design-time correction follows.

Service Level Monitoring and Management

Service	Service level objective	Measured by
Inbound ingestion	New or changed schemas mapped within one business day	Time from change event to published mapping
Processing and enrichment	99.5% of pipeline runs complete without manual intervention	Pipeline run outcomes
Search and APIs	99.9% availability; 95% of queries under 2 seconds	Service monitoring
Conversational AI	95% of answers traceable to governed terms	Answer audit logs
Publishing	Releases published on schedule with release notes	Release calendar

Breaches open an issue automatically, are reviewed in a monthly service review, and feed capacity and backlog planning.

Recursive Learning

The operating model learns from its own use. Signals from AI agents, microservices, and search are collected, reviewed, and turned into improvements through the feedback controls in Section 2.1:

Source	Signal	Improvement
AI agents	Low-confidence or rejected mappings; flagged answers	New mappings, synonyms, or constraints
Microservices	Validation failures, retries, and schema drift	Updated shapes, mappings, or inbound contracts
Search	Zero-result and abandoned queries; reformulations	New vocabulary terms and alternative labels
Users	Feedback on answers and search results	Prioritized requirements and competency questions

Nothing learned this way becomes authoritative until it passes the same change control as any other change.

Term Definition Standard

Every class, concept, and property is defined with the same set of fields, so that a term means the same thing to people, applications, and AI agents. The fields below match the class editor in the Semantic Modeling Workbench (Annotations, OWL Axioms, SKOS Annotations, Referenced By, and Data Properties). Required fields must be complete before a term can move from Draft to Candidate (Section 3.3).

Class Definition Fields

Editor field	RDF / OWL property	Required	Rule
IRI	(the class identifier)	Yes	https://www.insperity.com/ontology/{domain}/{ClassName}; UpperCamelCase, no acronyms, no hierarchy in the path
Label	rdfs:label, skos:prefLabel	Yes	One per language, tagged @en; sentence case; full words
Is defined by	rdfs:isDefinedBy	Yes	The IRI of the ontology that owns the term
Definition	skos:definition	Yes	One sentence, genus plus differentia (ISO 704): "A [parent] that [distinguishing feature]"
Alternate labels	skos:altLabel, skosxl:altLabel	As needed	Synonyms, acronyms, legacy names, persona terms (Section 3.8)
Hidden labels	skos:hiddenLabel	Optional	Misspellings and search variants; never displayed
Scope note	skos:scopeNote	Recommended	What is included and excluded; points to neighboring terms
Example	skos:example	Recommended	One or two real instances
Notation	skos:notation	Optional	Codes from source systems; the only place a code or acronym may stand alone
Sub-Class Of	rdfs:subClassOf (named class)	Yes	At least one named parent; the hierarchy carries the meaning (Section 3.1)
Restrictions	rdfs:subClassOf (owl:Restriction)	Optional	Existential (someValuesFrom) restrictions only, to document necessary links
Equivalent class	owl:equivalentClass	Avoid	Only with Architecture approval; defined classes force run-time inference
Disjoint with	owl:disjointWith	When true	Assert only where membership truly excludes; prefer at the highest level
Key properties	owl:hasKey	Optional	Data properties that uniquely identify an instance (for example, a document identifier)
All disjoint groups	owl:AllDisjointClasses	When true	For sibling sets that are mutually exclusive
Enumeration	owl:oneOf	Closed sets only	Only for fixed value sets such as a lifecycle status; never for open business classes
Disjoint union of	owl:disjointUnionOf	Avoid	Use subclasses plus AllDisjointClasses instead
Referenced by	(computed)	Review	Axioms that use the term; generated by the

Editor field	RDF / OWL property	Required	Rule
			workbench and checked in impact assessment
Data properties	owl:DatatypeProperty with this domain	As needed	Drawn from the attribute taxonomy (Section 3.10)
Governance	core:hasLifecycleStatus, owl:versionInfo, dcterms:created, dcterms:modified, dcterms:creator	Yes	Status, version, dates, and owner; set by change control

Worked Example: Documentation Standards

The example below completes the class editor entry for **Documentation Standards**. In the source entry most fields were empty and the identifier carried its hierarchy and an acronym in the path (sm:Artifact:Develop:SE:Documentation_Standards). Under the Company standard, the hierarchy moves into rdfs:subClassOf, "SE" is spelled out, and the identifier moves under the Company base URL.

Field	Value
IRI	https://www.insperity.com/ontology/service-model/DocumentationStandards
Label	"Documentation standards"@en
Is defined by	https://www.insperity.com/ontology/service-model/
Definition	"A software engineering artifact that specifies the required content, structure, format, review, and retention of the documentation produced during development."@en
Alternate labels	"Documentation guidelines"@en; "Documentation rules"@en; persona labels in Section 3.8
Scope note	"Includes templates, style rules, and review checklists for documentation. Excludes coding standards and testing standards, which are separate classes."@en
Example	"Application programming interface documentation standard; release note template."@en
Sub-Class Of	Software Engineering Artifact (which is a subclass of Development Artifact, a subclass of Artifact)
Restrictions	appliesTo some Software Engineering Work Product; ownedBy some Role
Equivalent class	None (asserted hierarchy preferred)
Disjoint with	None asserted at this level; inherits Artifact disjoint with Process Step
Key properties	hasDocumentIdentifier
All disjoint groups	None
Enumeration	None (open class)
Disjoint union of	None
Referenced by	Software Engineering Process Step governedBy some Documentation Standards (computed)
Data properties	hasDocumentIdentifier, hasEffectiveDate, hasReviewCycleInMonths
Governance	Status Draft; version 0.1.0; owner Software Engineering Practice Lead

The same entry in Turtle:

```
@prefix sm:      <https://www.insperity.com/ontology/service-model/> .
@prefix core:    <https://www.insperity.com/ontology/core/> .
@prefix owl:     <http://www.w3.org/2002/07/owl#> .
@prefix rdfs:    <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos:    <http://www.w3.org/2004/02/skos/core#> .
@prefix dcterms: <http://purl.org/dc/terms/> .
@prefix xsd:     <http://www.w3.org/2001/XMLSchema#> .

sm:DocumentationStandards a owl:Class ;
    rdfs:label        "Documentation standards"@en ;
    skos:prefLabel    "Documentation standards"@en ;
    skos:altLabel     "Documentation guidelines"@en , "Documentation rules"@en ;
    skos:definition   "A software engineering artifact that specifies the required
                       content, structure, format, review, and retention of the
                       documentation produced during development."@en ;
    skos:scopeNote    "Excludes coding standards and testing standards."@en ;
    rdfs:isDefinedBy  <https://www.insperity.com/ontology/service-model/> ;
    rdfs:subClassOf   sm:SoftwareEngineeringArtifact ,
        [ a owl:Restriction ; owl:onProperty sm:appliesTo ;
          owl:someValuesFrom sm:SoftwareEngineeringWorkProduct ] ,
        [ a owl:Restriction ; owl:onProperty core:ownedBy ; owl:someValuesFrom core:Role ] ;
    owl:hasKey        ( sm:hasDocumentIdentifier ) ;
    core:hasLifecycleStatus core:Draft ;
    owl:versionInfo   "0.1.0" ;
    dcterms:created   "2026-10-13"^^xsd:date .

sm:SoftwareEngineeringArtifact rdfs:subClassOf sm:DevelopmentArtifact .
sm:DevelopmentArtifact         rdfs:subClassOf core:Artifact .
```

Multi-word domain names in identifiers are written in lower case and joined with hyphens (service-model, human-resources-compliance).

Labels and Personas

One concept is called different things by different people: a payroll specialist, a worksite employee, a developer, and an auditor rarely use the same words. Labels capture those words without creating duplicate concepts, and **personas** record who uses which words and what each group needs. Together they drive search, user-interface text, AI answers, and access decisions.

Use of Alternate Labels

Label type	Property	Use for	Rules
Preferred	skos:prefLabel	The single display name	Exactly one per language; unique within the concept scheme; full words, no acronyms
Alternate	skos:altLabel	Synonyms, acronyms, abbreviations, legacy system names, persona terms	Any number; must not repeat the concept's own preferred label; each has a recorded source
Hidden	skos:hiddenLabel	Misspellings and search variants	Used by search only; never shown to users or used in AI answers
Notation	skos:notation	Codes from systems of record	Typed with a datatype for the source system; not a label

- **One concept, many labels:** if two terms mean the same thing, they become labels of one concept; if they differ in meaning, they become two concepts linked by a relation.
- **No collisions:** an alternate label that is the preferred label of another concept in the same scheme is rejected by validation, because it makes search ambiguous.
- **Acronyms:** every acronym in Appendix F is recorded as an alternate label of its spelled-out concept, so "FMLA" finds Family and Medical Leave Act.
- **Language tags:** every label carries a language tag (@en, @es), so labels can be served per language.
- **Provenance:** each alternate label records where it came from: search logs, a source system, a subject-matter expert, or a regulation.

Associating Alternate Labels with Personas

A plain skos:altLabel is a text value, so nothing can be said about it, including which persona uses it. Where a label belongs to a persona, it is recorded as a **label resource** using the SKOS eXtension for Labels (SKOS-XL), and the label resource is linked to one or more personas. At publish time the plain skos:altLabel is also written, so tools that only understand SKOS still find the label.

```
@prefix persona: <https://www.insperity.com/vocabulary/persona/> .
@prefix label:   <https://www.insperity.com/vocabulary/service-model/label/> .
@prefix skosxl:  <http://www.w3.org/2008/05/skos-xl#> .

sm:DocumentationStandards
    skosxl:altLabel label:DocumentationStandards-SoftwareDeveloper ,
                    label:DocumentationStandards-ComplianceAuditor .
```

```
label:DocumentationStandards-SoftwareDeveloper a skosxl:Label ;
    skosxl:literalForm        "Docs guidelines"@en ;
    core:intendedForPersona   persona:SoftwareDeveloper ;
    dcterms:source            "Developer portal search logs" .

label:DocumentationStandards-ComplianceAuditor a skosxl:Label ;
    skosxl:literalForm        "Documentation controls"@en ;
    core:intendedForPersona   persona:ComplianceAuditor .

# written at publish time for SKOS-only consumers
sm:DocumentationStandards skos:altLabel "Docs guidelines"@en , "Documentation controls"@en .
```

- **Selection rule:** an application shows the preferred label unless the current persona has its own label for the concept, in which case it may show that label with the preferred label available on hover or in the definition.
- **Search rule:** all alternate labels are searchable by everyone; persona labels raise ranking for that persona only.

Definition and Maintenance of Personas

A persona is a governed description of a group of users who share a role, goals, vocabulary, and information needs. Personas are individuals of the class core:Persona in the persona vocabulary (https://www.insperity.com/vocabulary/persona/). A persona describes a typical user; it is not a person and not a security role.

Attribute	Property	Example (Payroll Specialist)
Name and definition	skos:prefLabel, skos:definition	"Payroll specialist": processes payroll for client worksites
Roles	core:correspondsToRole	Payroll Specialist role in the organization model
Process steps	core:performsStep	Validate timesheet; Calculate gross pay; Approve payroll run
Domains	core:hasDomainInterest	Payroll; Tax; Wages and hours
Expertise level	core:hasExpertiseLevel	Practitioner (from a controlled list: Novice, Practitioner, Expert)
Vocabulary	(persona labels, above)	"Gross-to-net", "Off-cycle run"
Information needs	core:hasInformationNeed	Deadlines, tax rule changes, exception reasons
Communication style	core:prefersReadingLevel, core:prefersTone	Concise, procedural, technical terms allowed
Channels	core:usesChannel	Payroll application, internal search, AI assistant
Owner and review	core:ownedBy, core:nextReviewDate	Payroll Operations lead; reviewed every six months
Status and version	core:hasLifecycleStatus, owl:versionInfo	Released, 1.2.0

- **Keep the set small:** a persona is added only when its vocabulary or needs differ enough to change what is shown; most organizations need ten to twenty.

- **Evidence-based:** personas are created and revised from evidence: search and assistant logs, support tickets, user research, and subject-matter experts.
- **Review cycle:** every persona is reviewed at least every six months and after organizational or product changes; unused labels are retired.
- **Change control:** persona changes follow Section 3.3 like any other knowledge artifact, with a version, an approver, and a change record.
- **Deprecate, never delete:** retired personas are marked deprecated and replaced, so logs and past AI answers remain explainable.
- **No protected attributes:** personas never encode age, gender, ethnicity, health, or other protected or sensitive characteristics; they describe work, not people.

Use of Personas for Tailoring, Prompt and Context Engineering, and Security

Use	What the persona drives	Control
Tailoring	Labels shown, search ranking and query expansion, depth of definitions, examples chosen, reading level	Same concept and same facts for every persona; only presentation changes
Prompt engineering	Persona-specific prompt templates: role statement, tone, vocabulary, allowed tasks, answer format	Templates are versioned knowledge artifacts under change control
Context engineering	Which part of the graph is retrieved: concepts linked to the persona's process steps and domains, persona labels, definitions, provenance	Retrieval is limited to a token budget and always includes definitions and sources
Security	Nothing on its own: a persona selects relevance, never permission	Access is decided by the user's authenticated entitlements, applied before persona tailoring

- **Persona from identity, not from the prompt:** the persona is set from the authenticated user's role, so a user cannot gain different context by claiming to be another persona in a prompt.
- **Filter first, tailor second:** retrieval first removes everything the user is not entitled to see (by named graph and data classification), then applies persona ranking.
- **Traceability:** every AI answer logs the persona, prompt template version, and context retrieved, supporting the traceability requirements in Section 3.6 and ISO/IEC 42001.
- **Feedback:** corrections and low-rated answers are grouped by persona, so missing labels and information needs are found and fed back into persona maintenance.

Attributes, Properties and Relations

This section sets out which attributes are recorded for taxonomies and ontologies, how ambiguous relations such as "includes" are modeled, and how asserted, subclass, and inferred relations are kept apart.

Attributes and Properties for Taxonomies

Taxonomies are published as SKOS concept schemes, following ISO 25964 for thesauri and the label rules in Section 3.8.

Level	Property	Required	Notes
Scheme	dcterms:title, dcterms:description	Yes	Name and purpose of the taxonomy
Scheme	owl:versionInfo, dcterms:modified, dcterms:publisher	Yes	Version, last change, and owning team
Scheme	skos:hasTopConcept	Yes	The first level of the hierarchy
Concept	skos:prefLabel, skos:definition	Yes	As in the term definition standard (Section 3.7)
Concept	skos:inScheme	Yes	The scheme the concept belongs to
Concept	skos:broader / skos:narrower	Yes (except top)	Classification hierarchy; top concepts use skos:topConceptOf
Concept	skos:related	Optional	Associative link to a concept that is not broader or narrower
Concept	skos:altLabel, skos:hiddenLabel, skos:notation	As needed	Synonyms, search variants, and codes
Concept	skos:scopeNote, skos:example, skos:changeNote, skos:historyNote	Recommended	Usage guidance and change history
Concept	skos:exactMatch, closeMatch, broadMatch, narrowMatch, relatedMatch	As needed	Mappings to other schemes, always through the core (Section 3.5)
Concept	owl:deprecated, dcterms:isReplacedBy	On retirement	Retired concepts point to their replacement

- **Hierarchy means classification:** skos:broader is used only for "is a kind of" or "is a category of"; part-of and membership use ontology relations instead.
- **Limited polyhierarchy:** a concept may have more than one broader concept only when both are true classifications, and validation reports more than two.
- **Transitivity:** skos:broader is not transitive; queries that need all ancestors use skos:broaderTransitive or a property path (skos:broader+).

Attributes and Properties for Ontologies

Element	Required properties	Notes
Ontology header	owl:Ontology, owl:versionIRI, owl:versionInfo, owl:priorVersion, owl:imports, dcterms:title,	owl:backwardCompatibleWith or owl:incompatibleWith on every

Element	Required properties	Notes
	dcterms:description, dcterms:creator, dcterms:license, dcterms:modified	release
Class	The fields in Section 3.7	At least one named parent class
Object property	Labels and definition, rdfs:subPropertyOf (from the relation taxonomy), rdfs:domain, rdfs:range, owl:inverseOf where needed	Characteristics (functional, transitive, symmetric) declared only when always true
Data property	Labels and definition, rdfs:subPropertyOf (from the attribute taxonomy), rdfs:domain, rdfs:range (an XML Schema datatype)	Unit, data classification, and system of record recorded as annotations
Annotation property	Labels and definition	Used for documentation and governance only; never carries business facts

Use of "includes"

"Includes" is one of the most common words in business definitions and one of the most ambiguous. It can mean at least six different things, each with different consequences for inheritance and queries. A generic property named includes is therefore **not** created. The modeler chooses the specific relation, and "includes" may be recorded as an alternate label of that relation so business users still recognize it.

Statement	Actual meaning	Model as
"Benefits include medical, dental, and vision"	Kinds of (subclass)	rdfs:subClassOf: Medical Benefit subclass of Benefit
"The payroll process includes calculating gross pay"	Part of a process	Has occurrent part (BFO), specialized as core:hasStep
"The service bundle includes the payroll service"	Component of a package	core:hasComponent (has continuant part, BFO)
"The Wages and Hours category includes Overtime"	Taxonomy narrower concept	skos:narrower
"The client list includes Acme Corporation"	Membership of a collection	core:hasMember, or rdf:type when the list is a class
"The payroll ontology includes the core ontology"	Reuse of another ontology	owl:imports

Use of Relations: Asserted, Subclass, and Inferred

Kind	Where it comes from	Where it is stored	Example	Rule
Asserted	Stated by an author, a mapping, or an ingestion pipeline	Authored named graph, with provenance	Payroll Run hasStep Calculate gross pay; Payroll Run performedBy Payroll Specialist	The only kind people edit; every assertion has a source
Subclass	Asserted rdfs:subClassOf and rdfs:subPropertyOf	Ontology graph	Payroll Run subclass of Process Step	Carries inheritance; kept explicit and deep (Section 3.1)
Inferred	Computed by the	Implicit statements,	A Payroll Run is also typed	Never edited by hand;

Kind	Where it comes from	Where it is stored	Example	Rule
	GraphDB ruleset (RDFS plus selected OWL 2 RL rules)	kept separate from explicit ones	as Process Step and Process	fix the asserted source instead

- **Approved inferences only:** the ruleset is limited to subclass and subproperty closure, inverse properties, and transitivity for named properties (for example, part-of). Anything else needs Architecture approval.
- **Domain and range are not validation:** in RDFS, using a property on the wrong kind of subject silently infers a new type. Data quality is therefore checked with SHACL shapes, and unexpected inferred types are reported.
- **Explicit versus implicit:** queries for change review and impact assessment read explicit statements only; GraphDB exposes them separately from implicit statements, so reviewers see what was actually authored.
- **Materialize at publish:** inferred hierarchy statements that applications rely on are written into the release, so consumers get the same answers without running a reasoner.

BFO and RDF Foundations

Basic Formal Ontology (BFO 2020, ISO/IEC 21838-2) gives the core ontology its top-level categories and its fundamental relations. Company relations and attributes are placed under those categories, so every new property has a defined meaning and a parent, and the ontologies stay compatible with other BFO-based ontologies such as the Information Artifact Ontology (IAO) and the OBO Relations Ontology (RO).

Use of BFO for Relations and Attributes

BFO category	Meaning	Company examples
Material entity	A physical thing	Worksite building, laptop
Process	Something that happens over time	Payroll run, enrollment, onboarding step
Quality	A measurable characteristic of a thing	Hours worked, pay rate, headcount
Role	Something a bearer has because of its situation, realized in processes	Payroll Manager, client administrator, co-employer
Disposition or function	A built-in tendency or purpose	Purpose of a service, capability of a system
Generically dependent continuant (information)	Information that can be copied across carriers	Policy, regulation text, document, rule, identifier, record

Relations. Company object properties are declared as subproperties of BFO or RO relations where the meaning fits, so that their behavior (for example, transitivity of parthood) is inherited rather than re-invented.

Company relation	BFO / RO parent	Note
hasStep / isStepOf	has occurrent part / occurrent part of	Transitive; supports roll-up of steps into processes
hasComponent	has continuant part	For packages, bundles, and assemblies
performedBy	has participant (RO: has active participant)	Shortcut for: an agent bearing a role participates in the step
consumes / produces	RO: has input / has output	Inputs and outputs of a process step
precedes / follows	precedes / preceded by	Order of process steps
hasRole	bearer of	Links a party to a role; the role is realized in process steps
locatedAt	located in	Worksites, offices, jurisdictions
governs / governedBy	IAO: is about (via governs)	A rule or policy is information about a process step
measures / measuredBy	IAO: is about (via measures)	A metric is information about a process or quality

Attributes. An attribute is modeled as a simple data property when its value is a plain fact about one thing. It is modeled as a BFO quality, role, or information entity with its own class when it changes over time, has units or currency, has its own provenance, or is shared. For example, hasEffectiveDate on a policy is a data property, while a pay rate (amount, currency, effective dates, source) is a Pay Rate entity linked to the employee.

Taxonomy of Relations

Every object property sits in one hierarchy rooted in the BFO relation families. A new relation cannot be released without a parent in this taxonomy.

Family (top property)	Company relations beneath it
Parthood (has part / part of)	hasStep, hasComponent, hasSubprocess
Participation (has participant)	performedBy, approvedBy, consumes, produces, notifies
Dependence and inherence (bearer of / inheres in)	hasRole, hasQuality, hasCapability
Realization (realizes)	fulfillsRole, deliversService
Temporal order (precedes)	precedes, triggers, supersedes
Location (located in)	locatedAt, hasJurisdiction, hasWorksite
Aboutness (is about)	governs, measures, documents, describes
Social and contractual (core:socialRelation; not in BFO)	employs, coEmploys, isClientOf, contractsWith, reportsTo

- **Leaf properties carry domain and range;** top properties carry definitions and characteristics only.
- **Inverses are declared once** with owl:inverseOf; only one direction is asserted in data.
- **Mappings are not relations:** SKOS mapping properties (exactMatch and others) are kept out of this taxonomy.

Taxonomy of Attributes and Properties

Family (top data property)	Examples	Rules
Identifier	hasDocumentIdentifier, hasEmployeeNumber, hasClientNumber	Usually functional; candidates for owl:hasKey; never reused
Temporal	hasEffectiveDate, hasEndDate, hasCreatedDate	xsd:date or xsd:dateTime with time zone
Quantitative	hasAmount, hasRate, hasHoursWorked	xsd:decimal; unit and currency always recorded
Descriptive	hasName, hasDescription	rdf:langString where text needs a language
Status and flag	isExempt, isActive	xsd:boolean; if a status has a lifecycle, use a controlled concept instead
Contact and personal	hasEmailAddress, hasPhoneNumber	Classified Confidential or Restricted; access controlled by named graph

Annotation properties follow the same discipline in four families: labels (Section 3.8), documentation (definition, scope note, example), provenance (source, creator, dates), and governance (status, owner, version, data classification, intended persona).

RDF 1.1 and Properties

RDF 1.1 (W3C Recommendation, 2014) is the base data model for every artifact in this document. Its treatment of properties is what makes the relation and attribute taxonomies above possible.

- **Properties are resources:** any IRI used as a predicate is an rdf:Property, so a property can have its own label, definition, parent, owner, and version, and can be governed like a class.
- **RDF Schema 1.1** adds rdfs:subPropertyOf, rdfs:domain, and rdfs:range, which carry the property hierarchy; OWL 2 adds object, data, and annotation properties and their characteristics.
- **Internationalized identifiers:** RDF 1.1 uses IRIs, which allow characters beyond ASCII; Company identifiers still use plain ASCII for portability.
- **Every literal has a datatype:** plain text is xsd:string and language-tagged text is rdf:langString, so data property ranges are always explicit.
- **Datasets and named graphs:** RDF 1.1 standardized datasets of named graphs and the TriG and N-Quads formats; Company artifacts use named graphs for ownership, provenance, release, and access control.

RDF 1.1 has no compact way to attach properties to a single statement (for example, the source or confidence of one fact). Until that is needed, two RDF 1.1 patterns are used: a **named graph** for provenance shared by a batch of statements, and a **qualified relation** (an n-ary pattern) when the relationship itself has business attributes. RDF 1.2, being standardized by W3C, adds triple terms and a Turtle annotation syntax for statement-level properties; GraphDB supports the earlier RDF-star form. It may be adopted for metadata such as source and confidence once the standard is final, but business facts stay in qualified relations.

```
# RDF 1.1: qualified relation for an employment with its own attributes
@prefix hr:     <https://www.insperity.com/resource/human-resources/> .
@prefix client: <https://www.insperity.com/resource/client/> .

hr:Employment-000123 a core:Employment ;
    core:hasEmployee   hr:WorksiteEmployee-004512 ;
    core:hasEmployer   client:Client-000987 ;
    core:hasStartDate  "2024-01-15"^^xsd:date .

# RDF 1.2 (annotation syntax): a property on one statement
sm:DocumentationStandards skos:altLabel "Docs guidelines"@en
    {| core:intendedForPersona persona:SoftwareDeveloper |} .
```

APPENDIX A

Finance (FP&A) Domain

An illustrative, end-to-end example of the framework applied to Financial Planning & Analysis: domain model and ontology, requirements and change management, LKIF mappings, vector databases, sequence diagrams, use cases, and MCP integration.

A.1 Finance Domain
A.2 FP&A Ontology
A.3 FP&A Ontology: Process-Centric Model
A.4 Ontology Requirements Specification (ORSD)
A.5 KGCL-to-ORSD Mapping
A.6 KGCL-to-ORSD Mapping (Detailed)
A.7 Mapping FP&A Metrics to LKIF Statements
A.8 Mapping FP&A Classes to LKIF Statements
A.9 Scenario Mapping in Vector Databases
A.10 Vector Database Integration with Ontologies
A.11 FP&A Sequence Diagrams
A.12 FP&A Use Cases
A.13 MCP and Microservices Integration

Finance Domain

A functional breakdown for the finance domain involves separating it into its core functions: accounting, financial planning and analysis (FP&A), treasury, tax, and risk and compliance. These functions work together to ensure a company's financial health, support strategic decisions, and maintain compliance.

Accounting and reporting

This function is responsible for the systematic recording, summarizing, and reporting of an organization's financial transactions to create a verifiable record of its financial position.

- General ledger (GL) management: Maintains the company's master record of financial transactions, ensuring accurate classification and aggregation.
- Accounts payable (AP): Processes and manages all money the company owes to its vendors and creditors.
- Accounts receivable (AR): Manages the collection of payments from customers and clients for goods or services rendered.
- Payroll: Ensures employees are paid accurately and on time, including managing salaries, wages, bonuses, and deductions.
- Financial statement preparation: Prepares and presents financial reports such as the balance sheet, income statement, and statement of cash flows.
- Financial controls: Implements and monitors internal controls to protect company assets and prevent fraud.

Financial planning and analysis (FP&A)

FP&A supports strategic decision-making by analyzing financial data and forecasting future performance.

- Budgeting: Develops the annual operating budget, projecting revenues and expenses for the coming year.
- Forecasting: Creates projections of future financial performance to help management anticipate trends and potential challenges.
- Performance reporting: Analyzes and reports on financial performance against budgets and forecasts, calculating variance and identifying key business drivers.
- Financial modeling: Builds "what-if" scenarios and models to evaluate potential investments, business strategies, and capital structure changes.
- Business partnering: Works directly with other departments to align financial plans with business unit strategies.

Treasury

This function manages the organization's capital and liquidity, ensuring there is enough cash to meet daily obligations while maximizing financial gain from its cash holdings.

- Cash management: Monitors and manages the company's daily cash flow and banking relationships.
- Capital structure management: Manages the balance of debt and equity used to finance the business.
- Financing: Raises capital through loans, credit facilities, or equity issuance to fund operations and expansion.
- Investment management: Oversees the investment of excess cash in short-term financial instruments to generate returns.
- Financial risk management: Identifies and mitigates financial risks, such as foreign exchange, interest rate, and credit risk.

Tax and compliance

This group handles all tax-related matters and ensures the company adheres to all legal and regulatory financial standards.

- Tax planning and compliance: Develops strategies to minimize tax liabilities while ensuring compliance with all tax laws.
- Regulatory reporting: Files all required tax returns and financial statements with relevant governmental authorities.
- Internal audit and controls: Conducts audits to ensure the integrity of financial reporting and adherence to internal policies and controls.
- International tax management: Manages tax obligations for companies operating in multiple countries, which adds complexity due to different international tax laws.

Strategic and executive functions

Led by the Chief Financial Officer (CFO), this area oversees the entire finance domain and provides strategic direction to the company.

- Executive leadership: The CFO provides high-level financial strategy to the CEO and the board of directors.
- Mergers and acquisitions (M&A): Identifies and evaluates opportunities for corporate development, such as buying, selling, or merging with other companies.
- Investor relations: Manages communication with investors, shareholders, and financial analysts.
- Stakeholder management: Serves as a key liaison for external parties, including auditors, banks, insurers, and regulators.

FP&A Ontology

This `.trig` file defines the FP&A ontology classes for processes, assets, parties, events, issues, and measures, while incorporating legal constraints via LKIF core norms. It references FIBO for process and measurement concepts and links LKIF legal constraints as regulatory obligations applicable to FP&A processes.

This ontology provides a standards-compliant semantic scaffold to support agentic AI, vector DB integration, and advanced FP&A analytics with regulatory compliance baked-in. Further expansion can model more fine-grained sub-processes and domain-specific legal norms.

```
@prefix fpaf: < http://example.org/fpaf# > .
@prefix fibo-proc: < https://spec.edmcouncil.org/fibo/ontology/BE/Process/Process/ > .
@prefix fibo-meas: < https://spec.edmcouncil.org/fibo/ontology/BE/Measurement/Measurement/ > .
@prefix lkif-core: < http://www.estrellaproject.org/lkif-core/lkif-core.owl# > .
@prefix owl: < http://www.w3.org/2002/07/owl# > .
@prefix rdfs: < http://www.w3.org/2000/01/rdf-schema# > .
@prefix xsd: < http://www.w3.org/2001/XMLSchema# > .
# FP&A Ontology Core Classes
fpaf:FinancialPlanningProcess a owl:Class ;
  rdfs:subClassOf fibo-proc:Process ;
  rdfs:label "Financial Planning Process"@en ;
  rdfs:comment "A process for strategic planning, budgeting, forecasting, and analysis in
finance."@en .
fpaf:BudgetingProcess a owl:Class ;
  rdfs:subClassOf fpaf:FinancialPlanningProcess ;
  rdfs:label "Budgeting Process"@en .
fpaf:ForecastingProcess a owl:Class ;
  rdfs:subClassOf fpaf:FinancialPlanningProcess ;
  rdfs:label "Forecasting Process"@en .
fpaf:PerformanceReportingProcess a owl:Class ;
  rdfs:subClassOf fpaf:FinancialPlanningProcess ;
  rdfs:label "Performance Reporting Process"@en .
fpaf:FinancialModelingProcess a owl:Class ;
  rdfs:subClassOf fpaf:FinancialPlanningProcess ;
  rdfs:label "Financial Modeling Process"@en .
fpaf:BusinessPartneringProcess a owl:Class ;
  rdfs:subClassOf fpaf:FinancialPlanningProcess ;
  rdfs:label "Business Partnering Process"@en .
# Assets
fpaf:FinancialAsset a owl:Class ;
  rdfs:label "Financial Asset"@en ;
  rdfs:comment "Assets such as budgets, forecasts, reports, models."@en .
# Parties
fpaf:FinanceParty a owl:Class ;
  rdfs:label "Finance Party"@en ;
  rdfs:comment "Participants such as finance teams, business units, management."@en .
# Events
fpaf:FinancialEvent a owl:Class ;
  rdfs:label "Financial Event"@en .
# Issues
fpaf:FinancialIssue a owl:Class ;
```

```
  rdfs:label "Financial Issue"@en ;
  rdfs:comment "Issues such as variances, data discrepancies, compliance risks."@en .
# Measures (from FIBO Measurement Ontology)
fpaf:Variance a owl:Class ;
  rdfs:subClassOf fibo-meas:Measurement ;
  rdfs:label "Variance"@en .
fpaf:KPI a owl:Class ;
  rdfs:subClassOf fibo-meas:Measurement ;
  rdfs:label "Key Performance Indicator"@en .
fpaf:ConfidenceInterval a owl:Class ;
  rdfs:subClassOf fibo-meas:Measurement ;
  rdfs:label "Confidence Interval"@en .
# Legal and Regulatory Constraints using LKIF Core
fpaf:LegalConstraint a owl:Class ;
  rdfs:subClassOf lkif-core:Norm ;
  rdfs:label "Legal Constraint"@en ;
  rdfs:comment "Legal constraints and obligations applicable to FP&A processes."@en .
# Properties
fpaf:hasAsset a owl:ObjectProperty ;
  rdfs:domain fpaf:FinancialPlanningProcess ;
  rdfs:range fpaf:FinancialAsset ;
  rdfs:label "has asset"@en .
fpaf:hasParticipant a owl:ObjectProperty ;
  rdfs:domain fpaf:FinancialPlanningProcess ;
  rdfs:range fpaf:FinanceParty ;
  rdfs:label "has participant"@en .
fpaf:triggersEvent a owl:ObjectProperty ;
  rdfs:domain fpaf:FinancialPlanningProcess ;
  rdfs:range fpaf:FinancialEvent ;
  rdfs:label "triggers event"@en .
fpaf:hasIssue a owl:ObjectProperty ;
  rdfs:domain fpaf:FinancialPlanningProcess ;
  rdfs:range fpaf:FinancialIssue ;
  rdfs:label "has issue"@en .
fpaf:measures a owl:ObjectProperty ;
  rdfs:domain fpaf:FinancialPlanningProcess ;
  rdfs:range fibo-meas:Measurement ;
  rdfs:label "measures"@en .
fpaf:isSubjectTo a owl:ObjectProperty ;
  rdfs:domain fpaf:FinancialPlanningProcess ;
  rdfs:range fpaf:LegalConstraint ;
  rdfs:label "is subject to legal constraint"@en .
# Example individuals (instances)
fpaf:Budget2026 a fpaf:FinancialAsset ;
  rdfs:label "2026 Annual Budget"@en .
fpaf:CFO a fpaf:FinanceParty ;
  rdfs:label "Chief Financial Officer"@en .
fpaf:BudgetApprovedEvent a fpaf:FinancialEvent ;
  rdfs:label "Budget Approved Event"@en .
fpaf:VarianceIssue a fpaf:FinancialIssue ;
  rdfs:label "Budget Variance Issue"@en .
fpaf:Budgeting a fpaf:BudgetingProcess ;
  fpaf:hasAsset fpaf:Budget2026 ;
  fpaf:hasParticipant fpaf:CFO ;
```

```
  fpaf:triggersEvent fpaf:BudgetApprovedEvent ;
  fpaf:hasIssue fpaf:VarianceIssue ;
  fpaf:measures fpaf:Variance ;
  fpaf:isSubjectTo < http://example.org/legal#SarbanesOxleyAct > .
# Semantic imports
<> owl:imports fibo-proc: .
<> owl:imports fibo-meas: .
<> owl:imports lkif-core: .
```

FP&A Ontology: Process-Centric Model

This section describes a structured FP&A ontology aligned with the mentioned core services, built around a process-centric domain reference model, and mapping to assets, parties, events, issues, and measures. The ontology adopts standards from FIBO, LKIF, and the enterprise modeling principles outlined in the references, positioning processes as the first-class citizens.

FP&A Ontology: Process-Centric Domain Reference Model

Core Classes and Relationships

- Process Class (First Class)
  - *Financial Process*: Budgeting, Forecasting, Performance Reporting, Financial Modeling, Business Partnering
  - *Associated Assets*: Financial Statements, Budgets, Forecast Reports, Models, Performance Metrics
  - *Parties*: Finance Department, Business Units, Executive Management, External Regulators
  - *Events*: Budget Submission, Forecast Updates, Variance Reports, Strategic Reviews, Model Simulations
  - *Issues*: Budget Overruns, Forecast Deviations, Data Discrepancies, Model Assumptions, Stakeholder Conflicts
  - *Measures*: Variance, Accuracy, Confidence Interval, KPIs, Scenario Outcomes

Process-Asset-Party-Event Mapping

Service	Core Process	Assets	Parties	Events	Issues	Measures
Financial Planning & Analysis	Developmen ts of Plans and Forecasts	Financial Statements , Budget Plans	Finance Team, Business Units	Budget Submissio n, Forecast Revision	Data Inaccuracy, Misalignment	Variance, KPI, Confidenc e
Budgetin g	Budget Developmen t	Budget Reports, Projections	CFO, Departmen t Heads, CFO	Budget Approval, Submissio n	Underfunding, Over-commit ment	Budget Variance, Cost Savings
Forecasti ng	Create and Update Projections	Forecast Documents , Trend Data	CFO, Strategic Planners	Forecast Revision, External Data Updates	Assumption Errors, External Shocks	Forecast Error, Confidenc e
Performa nce Reporting	Variance and KPI Reporting	Variance Reports, Performan ce Dashboard s	Manageme nt, Stakeholde rs	Report Generation , Review Cycles	Data Discrepancies, Misinterpretati on	Variance %, KPI Achievem ent
Financial Modeling	Scenario & "What-If" Simulation	Scenario Models, Investment Models	Analysts, Strategic Planners	Scenario Runs, Model Adjustmen ts	Model Assumptions, Data Gaps	Model Accuracy, Scenario Outcome s

Service	Core Process	Assets	Parties	Events	Issues	Measures
Business Partnerin g	Strategy Alignment	Collaborati on Plans, Meeting Minutes	Business Units, Executives	Strategy Sessions, Feedback Loops	Misalignment, Stakeholder Resistance	Alignment Score, Issue Resolutio n

Standardized References

- Use FIBO for financial instruments, KPIs, and process concepts.
- Integrate LKIF for legal and regulatory constraints affecting processes.
- Follow OWL/RDF principles and OMG core ontologies for formalizing the model.
- Ensure alignment with ISO/IEC/IEEE 24765 terminology and enterprise reference ontologies for consistency and interoperability.

Ontology Requirements Specification (ORSD)

About This Specification

The Ontology Requirements Specification Document (ORSD) is the requirements template defined by the **NeOn Methodology** for building ontology networks (Suárez-Figueroa, Gómez-Pérez and Villazón-Terrazas, 2009). It is a widely adopted community methodology rather than an ISO standard; no ISO standard prescribes a template for ontology requirements. The version in this section extends the NeOn template with document control, change-management, and release sections, so the same document governs both what the ontology must do and how it evolves.

How the Template Is Organized

- **Purpose, scope, and implementation language:** why the ontology exists, what it covers, and the formal language used (here, OWL 2 DL).
- **Intended end-users and uses:** who will rely on the ontology and for what tasks.
- **Functional requirements as competency questions:** the questions the ontology must be able to answer, such as "Which cost centers exceeded budget by more than 10%?".
- **Non-functional requirements:** performance, maintainability, versioning, and security expectations.
- **Pre-glossary of terms:** the key terms extracted from the requirements, which seed the ontology's vocabulary.
- **Extensions used here:** document control, a change management and release plan, and validation and verification criteria.

Related ISO Standards

Standard	Scope	Relevance to the ORSD
ISO/IEC/IEEE 29148	Requirements engineering	Quality criteria for requirements (unambiguous, verifiable, traceable) that ORSD requirements and competency questions should meet
ISO 704 and ISO 1087	Terminology work: principles and vocabulary	How the glossary defines concepts and terms consistently
ISO/IEC 21838	Top-level ontologies	Requirements for upper ontologies, relevant to the core ontologies the FP&A ontology extends
ISO/IEC/IEEE 24765	Systems and software engineering vocabulary	Consistent engineering terminology, cited elsewhere in this document

In practice, each competency question becomes a test case (typically a SPARQL query) in the validation criteria (item 10 below). That makes ORSD requirements verifiable in the sense of ISO/IEC/IEEE 29148.

FP&A Ontology Requirements

Ontology Requirements and Release Specification Document

FP&A Sub-domain in Finance Domain

1. Document Control

- Version: 1.0
- Date: 2025-10-21
- Authors: [Names of financial domain experts, ontology engineers]
- Reviewers: [Stakeholders from finance, compliance, IT]
- Approvals: [Governance committee]

2. Ontology Purpose and Goals

- To formally represent FP&A concepts including budgeting, forecasting, variance analysis, financial reporting, and driver-based planning.
- Enable consistent financial data interpretation across systems for enhanced decision-making and reporting accuracy.
- Support integration with broader corporate financial ontologies and regulatory compliance frameworks.

3. Scope

- Coverage of FP&A key processes: Budget Allocation, Financial Forecasting, Performance Metrics, Variance Analysis, Cash Flow Projections.
- Modeling of FP&A roles (analysts, managers), financial instruments, cost centers, and drivers.
- Excludes detailed accounting transactions and ledger-level details (covered in accounting ontology).

4. Implementation Language and Standards

- OWL 2 DL for formal ontology modeling.
- Conformance with Financial Industry Business Ontology (FIBO) where applicable.
- Use existing taxonomy standards for financial measures and KPIs (e.g., IFRS KPIs).

5. Intended End-Users and Stakeholders

- FP&A professionals and financial analysts for enhanced data-driven insights and planning.
- IT architects and application developers for integration with ERP and planning systems.
- Compliance officers monitoring financial governance and control.

6. Functional Requirements

- Ability to express budgeting cycles and forecast revisions.
- Support competency questions such as: "What is the forecasted revenue for Q4 2025?" or "Which cost centers exceeded budget by more than 10%?"
- Trace financial drivers impacting performance metrics and variances.
- Link financial plans to actual results and identify discrepancies.

7. Non-functional Requirements

- Ontology must support scalable real-time querying over large financial data sets.
- Must be maintainable with clear versioning aligned to financial period releases.
- Secure access control for sensitive financial concepts and data.

8. Glossary and Terminology

- Budget: Planned allocation of financial resources over a period.
- Forecast: Projections of future financial performance based on current data.
- Variance: Difference between budget and actuals.
- KPI: Key Performance Indicator measuring financial performance.

9. Change Management and Release Plan

- Changes to FP&A ontology governed via ORSD/KGCL integration for traceability.
- Version aligned with corporate financial periods (quarterly/annual).
- Reviews by finance and compliance teams prior to production deployment.

10. Validation and Verification Criteria

- Validation via competency question test cases and financial reporting scenario simulations.
- Logical consistency checked using ontology reasoners.
- Performance testing on query response times and concurrency.

11. Appendices

- References: FIBO, IFRS standards, company-specific financial policies.
- Sample data models for budgeting and forecasting instances.
- Contact information for ontology stewards and FP&A domain experts.

KGCL-to-ORSD Mapping

What Is KGCL?

The **Knowledge Graph Change Language (KGCL)** is an open standard for describing changes to ontologies and knowledge graphs. It was developed by Chris Mungall and collaborators in the OBO ontology community and published in the journal Database in 2025. Each change is a single, human-readable command (rename, obsolete, add a synonym, create a class, move a term under a new parent) backed by a formal data model written in LinkML. The same change can therefore be read by people and applied by software.

How KGCL Works

- **Controlled natural language:** commands such as rename X from "A" to "B" follow a defined grammar, so they are precise but readable.
- **Machine-applicable:** tools such as the Ontology Access Kit (OAK) and the ROBOT KGCL plugin apply commands directly to OWL ontologies, and can compute the KGCL changes between two ontology versions.
- **Workflow-friendly:** in the OBO community, change requests written as KGCL in GitHub issues are turned into pull requests by an automated agent (Ontobot) and merged after curator review.
- **Durable change log:** changes can be stored as text, JSON, YAML, RDF, or tables, giving a complete, replayable history.

Change	Example (as used in this appendix)	Effect
Rename	rename FP&A:BudgetAllocation from "Budget Allocation" to "Budget Distribution"	Changes a term's label; the identifier stays the same
Synonym	add synonym FP&A:CashFlowProjection "Liquidity Forecast"	Adds an alternative name for search and cross-team use
Obsolete	obsolete FP&A:DriverBasedPlanning	Deprecates a term without deleting it, optionally pointing to a replacement
Subclass	add subclass FP&A:VarianceAnalysis FP&A:FinancialAnalysis	Adds a parent (subclass-of) relationship

The examples follow the wording used in this document. KGCL's formal grammar names some commands slightly differently (for example, create synonym and create edge), so templates should follow the published grammar when they are automated.

How KGCL and the ORSD Work Together

The two records answer different questions. The ORSD (Section A.4) records **why** and **under what governance** the ontology changes: requirements, rationale, impact, approvals, and the release plan. KGCL records **exactly what** changed, in a form a machine can apply. Every approved change therefore has two linked records:

- An ORSD change entry with description, reason, impact, approval, and verification (ORSD item 9, Change Management and Release Plan).

- A KGCL statement that implements the change, applied to the ontology in the knowledge graph and versioned with the release.

The table below shows this pairing for several FP&A changes, and Section A.6 traces one change through every ORSD section it affects. Together they give an auditable trail from business intent to executable change.

KGCL-to-ORSD Mapping for FP&A Changes

The following example maps KGCL change commands to ORSD change documentation for FP&A ontology updates:

KGCL Change Statement	Corresponding ORSD Entry
rename FP&A:BudgetAllocation from "Budget Allocation" to "Budget Distribution".	Change Description: Renaming of BudgetAllocation concept to Budget Distribution to better reflect usage in variance analysis. Reason: Clarify terminology aligning with business glossary. Impact: All budgeting workflows and reports referencing this term will update accordingly. Approval: FP&A Committee, IT Governance.
add synonym FP&A:CashFlowProjection "Liquidity Forecast"	Change Description: Added synonym "Liquidity Forecast" to CashFlowProjection concept for improved cross-team understanding. Reason: Multiple teams use different terminology for same concept.
	Verification: Verify through end-user review and system search.
obsolete FP&A:DriverBasedPlanning	Change Description: Obsolete DriverBasedPlanning concept due to redesign of driver logic in updated financial model. Reason: Deprecated in favor of FP&A:FinancialDriverAnalysis concept introduced in latest fiscal cycle. Version: Effective from 2025 Q4 release.
add subclass FP&A:VarianceAnalysis FP&A:FinancialAnalysis	Change Description: Added VarianceAnalysis as subclass of FinancialAnalysis to support detailed variance reporting scenarios. Reason: New reporting requirements for variance details in FP&A dashboards. Testing: Included in FP&A scenario testing with validation on sample datasets.

This mapping shows how high-level change intents documented in ORSD (rationale, impact, approval) directly correspond with executable KGCL commands that modify the

ontology accordingly. This dual documentation ensures traceability, governance, and automation across the FP&A ontology lifecycle.

Such structured change management improves accuracy and communication around ontology evolution while supporting automated application and versioning in GraphDB or other knowledge graph platforms.

This approach can be expanded to include detailed competency questions, validation criteria, change request identifiers, and stakeholder comments linked bi-directionally between KGCL and ORSD artifacts.

KGCL-to-ORSD Mapping (Detailed)

This section follows a single KGCL command, the rename of BudgetAllocation, through each ORSD section it affects. Section A.5 explains KGCL and how it pairs with the ORSD.

Mapping an FP&A KGCL Change to ORSD Sections with Rationale (Example)

KGCL Change:

```
rename FP&A:BudgetAllocation from "Budget Allocation" to "Budget
Distribution"
```

Mapped ORSD Sections:

1. Change Management and Release Plan
   - Document this rename as a formal ontology change in the release plan.
   - Ensure versioning captures this change aligned with the next financial reporting period.
2. Change Description (part of Functional Requirements or Change Management)
   - Define the change clearly: renaming "Budget Allocation" to "Budget Distribution".
3. Rationale
   - Clarify that the rename reflects improved alignment with financial planning terminology and variance analysis use cases as gathered from FP&A stakeholders.
4. Impact and Stakeholders
   - Identify all reporting workflows, analyses, and systems impacted by this concept rename.
   - Note relevant users and governance approvals necessary for the change.
5. Validation and Verification Criteria
   - Outline planned tests and validation to ensure that all references to "Budget Allocation" update correctly and do not break queries or reports.
6. Glossary and Terminology
   - Update glossary with the new preferred term "Budget Distribution" and mark "Budget Allocation" as deprecated or synonym as applicable.

Summary

This KGCL change represents a controlled, traceable ontology modification. It should be documented in the ORSD change management sections to capture why and how the vocabulary evolves, its impact on FP&A processes, and the verification steps ensuring smooth adoption across the organization. This linkage supports transparent governance and automated lifecycle management using KGCL and GraphDB.

Mapping FP&A Metrics to LKIF Statements

What Is LKIF?

The **Legal Knowledge Interchange Format (LKIF)** was developed in the EU-funded ESTRELLA project as a standard way to represent and exchange legal knowledge such as legislation, regulations, and cases. Its centerpiece, the **LKIF Core** ontology, is written in OWL and defines basic legal concepts in modules, including norms (obligations, permissions, prohibitions), actions, roles, processes, time, and place. LKIF combines OWL with rules (SWRL) so that legal conditions can be evaluated automatically.

In this appendix, LKIF supplies the vocabulary for stating that an FP&A metric or process is subject to a norm. For example, a forecast carries an obligation to stay within a variance threshold, so a reasoner can check compliance instead of a person.

LKIF and ISO Standards for Legal and Compliance Matters

LKIF is not itself an ISO standard, and ISO does not publish a legal ontology. LKIF's role is to make the obligations that ISO management-system standards ask organizations to manage machine-readable and checkable:

Standard	Focus	How LKIF-based models support it
ISO 37301 (compliance management systems)	Identifying, maintaining, and monitoring compliance obligations	Each obligation is an LKIF norm linked to the processes, roles, and metrics it governs, forming a traceable obligations register
ISO 31000 (risk management)	Identifying, assessing, and treating risk	Norm violations and sanctions are modeled explicitly, so compliance risk can be queried and reported
ISO/IEC 42001 (AI management systems)	Responsible governance of AI systems	Agents check proposed actions against LKIF norms before acting, producing auditable decisions
ISO 704 and ISO 1087 (terminology)	Consistent definition of concepts and terms	LKIF concepts are formally defined, keeping legal terms consistent across systems
OASIS LegalRuleML and Akoma Ntoso	Open standards for legal rules and legal documents	Carry the source legal texts and rules that LKIF norms reference (for example, through governedBy links)

External regulations, such as Sarbanes-Oxley Section 404 in the example below, are modeled as the sources of LKIF norms. This keeps each FP&A control traceable to the regulation that requires it.

Mapping FP&A Metrics to Deontic Statements

Mapping FP&A metrics to LKIF deontic statements, creating semantic links to model obligations, permissions, and prohibitions about financial performance and compliance:

FP&A Metrics

- Variance: Difference between budgeted and actual values.
- Forecast Accuracy: How close forecasts match actual results.

- Key Performance Indicators (KPIs): Quantitative measures of business performance.
- Scenario Variance: Difference between scenario assumptions and realized results.
- Cash Conversion Cycle: Operational efficiency metric measuring cash flow dynamics.

LKIF Deontic Statements (Norms)

- Obligation: A duty to achieve a target metric or stay within thresholds (e.g., "variance must be under 5%").
- Permission: Allowed variance range or forecasting tolerance.
- Prohibition: Forbidden deviations or actions (e.g., misstating forecast).
- Violation: Breach when KPIs or metrics fall outside allowed bounds.
- Sanction: Consequences tied to violation of financial norms or controls.

Property Mappings

FP&A Metric	LKIF Deontic Link	Description
Variance	lkif:obligation	Mandates variance within acceptable limits
Forecast Accuracy	lkif:permission	Allows specific accuracy range for forecasting
KPIs	lkif:obligation / lkif:prohibition	Defines must-meet performance and forbidden outcomes
Scenario Variance	lkif:violation	Flags when actual outcomes deviate beyond scenario assumptions
Cash Conversion Cycle	lkif:obligation	Imperative to maintain operational cash flow efficiency

This mapping enables agentic AI systems to reason over financial metrics semantics linked to formal deontic constraints, automating compliance checks, alerts, and governance decision support in FP&A.

Such integration wraps quantitative FP&A monitoring within a normative framework built on LKIF, improving regulatory alignment and process transparency.

```
@prefix fpaf: < http://example.org/fpaf# > .
@prefix lkif-core: < http://www.estrellaproject.org/lkif-core/lkif-core.owl# > .
@prefix owl: < http://www.w3.org/2002/07/owl# > .
@prefix xsd: < http://www.w3.org/2001/XMLSchema# > .
# Class Definitions
fpaf:FinancialForecast a owl:Class ;
  rdfs:label "Financial Forecast" ;
  rdfs:comment "A predicted financial outcome over a planning horizon." .
lkif-core:Obligation a owl:Class ;
  rdfs:label "Obligation" ;
  rdfs:comment "A deontic statement imposing a duty or requirement." .
lkif-core:Permission a owl:Class ;
  rdfs:label "Permission" ;
  rdfs:comment "A deontic statement granting permission." .
# Instances
fpaf:Q4Forecast2025 a fpaf:FinancialForecast ;
```

```
  rdfs:label "Q4 2025 Financial Forecast" ;
  fpaf:hasValue "1000000"^^xsd:decimal ;
  fpaf:forecastDate "2025-10-01"^^xsd:date .
# Linking forecast to legal rules
fpaf:Q4Forecast2025 lkif-core:isSubjectTo [
  a lkif-core:Obligation ;
  lkif-core:imposesOn fpaf:Q4Forecast2025 ;
  lkif-core:hasCondition "Forecast variance must be less than 5%"^^xsd:string ;
  lkif-core:relatedNorm "SOX Compliance Section 404"^^xsd:string
] .
fpaf:Q4Forecast2025 lkif-core:isSubjectTo [
  a lkif-core:Permission ;
  lkif-core:permitsOn fpaf:Q4Forecast2025 ;
  lkif-core:hasCondition "Adjustments allowed within 2% forecast range"^^xsd:string
] .
```

- The `FinancialForecast` instance `Q4Forecast2025` is linked to LKIF deontic statements of `Obligation` (must keep variance under 5%, e.g., Sarbanes-Oxley compliance) and `Permission` (allowed forecast adjustments).
- `lkif-core:isSubjectTo` connects the forecast to applicable normative statements.
- This structure allows automated reasoning over FP&A forecasts with legal compliance constraints, supporting auditability and decision support.

This example can be extended with more specific predicates and property values, and integrated with vector and agentic AI knowledge scaffolds to enforce real-time compliance and governance.

Mapping FP&A Classes to LKIF Statements

Building on Section A.7, this section maps FP&A classes onto the LKIF Core modules, so every financial process, party, and event can be linked to the norms that govern it.

Core FP&A classes and properties mapped to LKIF legal constraints, based on LKIF's modular legal ontology structure, focus on integrating FP&A financial processes with legal norms, roles, and actions.

Core FP&A Classes to Map

- FinancialPlanningProcess (extends LKIF's Process)
- BudgetingProcess, ForecastingProcess, PerformanceReportingProcess, FinancialModelingProcess, BusinessPartneringProcess (subclasses)
- FinancialAsset (documents, reports, budgets)
- FinanceParty (roles in LKIF legal-role: finance officers, auditors)
- FinancialEvent (actions or triggers linked to LKIF Action or LegalAction)
- FinancialIssue (potentially mapped to LKIF Norm Violations or Legal Issues)
- LegalConstraint (norms, obligations, constraints from LKIF Norm module)

Key Properties (object and data)

- hasParticipant (link Process to LKIF Role/Party involved)
- triggersEvent (Process → Event)
- hasIssue (Process → Issue related to legal non-compliance or risks)
- isSubjectTo (Process → LegalConstraint from LKIF Norm)
- measures (Process → Measures e.g., Variance, KPIs)
- governedBy (link FinancialAsset or Process to specific LKIF legislation or source documents)

Mapping Rationale

- LKIF Process Module models causal financial activities (budgeting, forecasting) as Legal Processes.
- LKIF Role Module defines finance participants' legal roles (e.g., fiduciary duty, auditor role).
- LKIF Action Module captures compliance actions and reporting events.
- LKIF Norm Module defines obligations, permissions, and prohibitions applicable to FP&A processes (e.g., SOX compliance).
- Using these mappings supports semantic reasoning about legal constraints on financial processes, automating compliance checks, and linking regulations to operational FP&A activities.

This approach leverages LKIF's modular ontology design for clear legal-semantic integration, critical for auditability and governance of FP&A workflows.

Scenario Mapping in Vector Databases

A process-centric approach to mapping scenarios in vector databases aligns with best practices in semantic data integration and knowledge engineering. Making processes first-class citizens yields several advantages:

1. Contextual Grounding: Processes provide meaningful context around scenarios, enabling vector embeddings to be semantically anchored rather than floating in isolation. This alignment improves precision in similarity search and recall by factoring in process role and workflow dependencies.
2. Structured Semantic Scaffolding: Mapping vectorized scenarios via domain-specific ontologies centered on processes helps preserve rich relationships between entities, events, assets, and parties. This ensures vector searches respect underlying business logic and ontological constraints.
3. Interoperability and Versioning: Anchoring scenario mappings to formal process instances enables consistent version control, change impact analysis, and lifecycle management using standards-based approaches (e.g., OMG's ODM and DOL).
4. Cross-Domain Integration: The approach supports multi-process and

   cross-ontology scenario mapping, essential for complex enterprise environments where scenarios span functions or business units.
5. Hybrid Reasoning and Querying: This foundation facilitates hybrid search strategies combining vector similarity with symbolic reasoning, maximizing relevancy and interpretability for agentic AI applications.

The approach is practical and conceptually robust, facilitating semantically rich, scalable, and adaptable scenario management in vector-backed knowledge systems. Consider incorporating ongoing validation with stakeholders and iterative alignment of vector embeddings with process semantics to optimize outcomes further.

Yes, inserting a lightweight scaffold into the vector database to maintain lineage back to the richer context in the ontologies for two-level search by domain-specific agents is a sensible and effective strategy.

Reasons:

1. Efficient Indexing & Querying:

- The scaffold acts as a semantic pointer or anchor inside the vector DB, enabling fast approximate similarity searches at the vector level.
  - The detailed ontological context remains in the richer ontology store, referenced as needed for precise symbolic reasoning.
  - This preserves performance by avoiding heavy semantic reasoning at query time over the entire dataset.

2. Lineage and Traceability:

- Embedding process and domain identifiers in the vector scaffold allows each vector entry to be traced back to exact ontology elements (processes, scenarios, legal constraints, assets).

    - This traceability supports explainability and audit requirements, crucial in regulated domains.

3. Stable Backbone with Extensible Flexibility:

- Assuming domains and processes are stable, this scaffold provides a persistent anchoring schema that reduces re-indexing needs when scenario data changes.
    - The scaffold structure can be incrementally evolved alongside ontological updates, supporting versioning and evolution without major disruption.

4. Hybrid 2-Level Search:

- The vector DB performs semantic similarity or approximate nearest neighbor search using lightweight embeddings.
    - Domain-specific agents retrieve richer knowledge via ontology references for confirmatory reasoning, validation, or complex queries.
    - This hybrid approach optimizes speed and accuracy—vector DB filters candidates, then ontology-enabled agents handle interpretation and compliance checks.

5. Best Practice Alignment:

- Research and practice in hybrid graph-vector database systems view ontologies as providing semantic grounding, while vector indexes enable efficient search over unstructured or semi-structured data.
    - A lightweight scaffold in vector DB aligns with this best practice, capturing just enough context for mapping without duplication of the full ontological logic inside vectors.

Conclusion

Inserting a lightweight, process- and domain-tagged scaffold inside vector DBs with explicit references back to rich, versioned ontologies is a practical and effective design. It balances query performance, semantic integrity, and flexibility, enabling two-level searches orchestrated by domain-specific agents that optimize relevancy, explanation, and governance

The following domain-specific vector database schema for FP&A scenarios focuses on capturing vector embeddings, minimal semantic scaffolding for mapping, and supporting metadata for lineage and versioning. The schema uses typical data types for vector storage and supports linking to domain-specific ontological identifiers. `sql`

```
CREATE TABLE fpaf_vectors (
    id SERIAL PRIMARY KEY ,
    process_instance_uri VARCHAR ( 512 ) NOT NULL, -- URI for stable
domain-specific process instance
    scenario_uri VARCHAR ( 512 ) NOT NULL,         -- URI for
specific scenario mapped
    embedding VECTOR( 1024 ) NOT NULL,            -- 1024
dimensional vector embedding
    domain_tag VARCHAR ( 128 ) NOT NULL,           -- Domain
identifier, e.g. 'FP&A'
    asset_uri VARCHAR ( 512 ),                      -- Related
financial asset URI (budget, forecast doc)
    party_uri VARCHAR ( 512 ),                      -- Related party
or role URI (e.g. CFO)
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP , -- Ingestion
or update time
    version INTEGER DEFAULT 1 ,                    -- Version for
```

```
embedding/schema evolution
    metadata JSONB                              -- Flexible
metadata for any additional info or provenance
);
CREATE INDEX idx_fpaf_vectors_process_uri ON
fpaf_vectors(process_instance_uri);
CREATE INDEX idx_fpaf_vectors_domain_tag ON
fpaf_vectors(domain_tag);
CREATE INDEX idx_fpaf_vectors_embedding ON fpaf_vectors USING
ivfflat (embedding vector_l2_ops);  -- For vector similarity
search; example with PGVector
```

Explanation:

- process_instance_uri: URI linking to the stable reference process model in the ontology.
- scenario_uri: URI that identifies the specific scenario mapped within a process instance.
- embedding: Multi-dimensional vector representing semantic content of the scenario.
- domain_tag: Simple tag to filter across domains if multi-domain system.
- asset_uri and party_uri: Optional ontological links for enriched semantic context.
- timestamp & version: For auditing and version control of vector embeddings.
- metadata: JSON for extensible, flexible details including lineage, source system, or calculation method.

This DDL schema is scalable for millions of vectors, supports fast approximate nearest neighbor vector search (optimized via the "embedding" index), and allows critical traceability back to ontological URIs for richer semantic lookup and agentic AI collaboration. It aligns with best practices in vector DB domain modeling and semantic governance.

A more detailed schema, with relational tables for normalized ontological entities or a hybrid graph and vector approach, can be designed as needed.

Vector Database Integration with Ontologies

Design Specification: Ontology-Vector Database Integration Scaffold

Objective

Create a sustainable semantic scaffold that integrates domain-specific ontologies with vector databases to enhance search accuracy, relevancy, and recall, supporting agentic AI applications. The design adopts established standards including OMG's Ontology Definition Metamodel (ODM), ISO/IEC/IEEE 24765 systems engineering standards, and ontology best practices from multiple OMG specifications.

Key Principles and Standards

- Ontology Modeling & Interoperability:
  - Use OMG's Ontology Definition Metamodel (ODM) for formal ontology representation supporting multiple ontology languages (OWL, RDF) with strong model-driven architecture (MDA) foundation.
  - Follow ISO/IEC/IEEE 24765 for systems and software engineering to ensure modular, maintainable, and interoperable architectural design.
  - Employ OMG Distributed Ontology, Model and Specification Language (DOL) for ontology integration and mapping across heterogeneous domain ontologies.
  - Use OMG URI standards for elements to ensure unique identification and standard reference.
- Vector Database Integration:
  - Embed ontological concepts and relationships as vectors maintaining semantic context using ontology embedding techniques.
  - Link vector embeddings with ontological entities via metadata stored alongside vectors, enabling hybrid symbolic and vector-based retrieval.
  - Enable persistent, versioned storage of mappings between ontology elements and vectorized scenarios.

1. Mapping Process Instances to Vector DB Scenarios

- Map individual process instances in domain-specific ontologies as formal ontological entities, each with a unique URI.
- Represent the scenario features for each instance as vector embeddings within the vector database, grounded in the ontology schema using ODM and DOL standards.
- Use metadata tagging within the vector DB to associate each vector scenario representation explicitly with its corresponding ontology process instance URI.
- Implement software adapters to translate between ontology-based process descriptions and vector embeddings, ensuring semantic consistency.

2. Maintaining Mappings and Versioning

- Store mappings in a dedicated ontological metadata repository, separate but linked with the vector database, using version control systems aligned with ISO/IEC/IEEE standards for configuration management.
- Versions of domain-specific ontologies and scenario mappings are tracked using semantic versioning, with persistent URIs referencing specific versions.
- Use OMG standards such as ODM and DOL to capture evolving ontologies and their mappings, supported by tools that manage ontology lifecycle (e.g., ontology repositories with versioning, change tracking, and validation workflows).
- Integration pipelines must automate synchronization between updated ontologies and vector embedding updates, preserving mapping integrity.

3. Handling Scenarios Mapping to Multiple Processes/Ontologies

- Model scenarios as first-class ontological entities that can have relations (e.g., "participatesIn", "relatesToProcess") to multiple process instances across one or multiple domain ontologies.
- Use composite mappings that aggregate vector embeddings corresponding to these scenario entities and their multiple linked processes, applying multi-ontology integration principles via DOL.
- Apply reasoning engines (enabled by ODM semantics) to infer scenario-level connections across process boundaries and ontologies, promoting holistic retrieval and precision.
- Store cross-ontology scenario-process mappings as explicitly modeled relationships with version control to track dependencies and changes.

Implementation Outline

Step	Description	Supported Standards
Ontology Ingestion	Import OWL/RDF ontologies into ODM-compliant repositories	OMG ODM, OWL, RDF
Vector Embedding	Compute embeddings for ontology concepts, relations, and processes	Embedding methods + Metadata via OMG URI
Metadata Linking	Tag vectors with ontology URIs and version info	OMG URI standards, ISO config mgmt
Hybrid Query Engine	Combine vector similarity search + ontology reasoning for precision	OMG DOL, ISO/IEEE 24765 system design
Mapping Persistence	Store mappings/version mappings in dedicated repos with lifecycle mgmt	OMG ODM, DOL, ISO/IEEE config & version
Multi-ontology Merging	Support composite scenario mappings with reasoning across domains	OMG DOL, OWL semantics

This specification ensures a durable, standards-compliant framework that supports evolving ontologies integrated with vector search capabilities tailored for domain-specific agentic AI, addressing accuracy, relevancy, and recall improvements systematically.

FP&A Sequence Diagrams

The following sequence diagrams, presented as textual walkthroughs of typical diagram flows, describe FP&A (Financial Planning & Analysis) processes within a finance sub-domain. These diagrams cover microservices, knowledge graphs (RDF with SHACL), AI agentic/Vector DB flows, MCP middleware integration, and governance service controls. Each sequence represents a core FP&A scenario (such as planning, forecasting, analysis, policy enforcement, and model updates).

FP&A Sequence Diagram 1: Budget Forecast Creation

1. User/Analyst → FP&A Agent (UI/API)

- Requests new budget forecast for a period (e.g., Q1 Revenue by business unit)

2. FP&A Agent → Vector DB Service (domain-aligned)

- Sends query context (domain: FP&A, task: "budget projection", drivers: revenue/expenses, vocab/taxonomy: RDF graph-aligned)

3. Vector DB Service → Knowledge Graph (RDF/SHACL endpoint via MCP)

- Requests entity relationships, policy-constrained assumptions (foreign exchange rates, headcount limits, seasonal factors)

4. Knowledge Graph → SHACL Engine (Policy/Validation Service)

- Applies domain policies (e.g., no scenario uses expense ratios above X%, validates forecast structure, data completeness)

5. SHACL Engine → Knowledge Graph / Agent

- Returns validation results (pass/fail, reasons, constraint messages)

6. Vector DB → FP&A Agent

- Returns synthesized context/embeddings, mapped to planning inputs

7. FP&A Agent → MCP Broker

- Initiates tool invocation: launches budget calculation, report generation, downstream ETL (if compliant)

8. MCP Broker → FP&A Domain Microservices

- Triggers:
  - Budget Calculation Microservice
  - Report Generation Microservice
  - Notification/Audit Logging Microservice

9. Microservices → MCP Broker → FP&A Agent / User

- Reports status, deliverables, exceptions; triggers workflow updates

10. Governance/Audit Service

- Logs all steps, validation checks, and user actions for compliance reporting

FP&A Sequence Diagram 2: Policy Validation During Workflow

1. Scenario Change (User or Automated Event) → FP&A Agent

- E.g., update salary forecast due to new hiring policy

2. FP&A Agent → Knowledge Graph Service

- Updates salary projection data for relevant entities

3. Knowledge Graph Service → SHACL Validation Engine

- Triggers shape validation for:
  - Maximum salary caps
  - FTE restriction
  - Fund allocation policies

4. SHACL Engine → Knowledge Graph Service / FP&A Agent

- Returns validation pass/fail, details (linked to policy artifact)

5. If Fail:

- FP&A Agent → User, with actionable feedback and constraint violations

6. If Pass:

- Proceeds to downstream scenario/planning steps, triggers update in forecast models via MCP

7. Audit Service

- Records validation cycle, policy artifact involved, and user/system actions

FP&A Sequence Diagram 3: AI-Driven Variance Analysis

1. Scheduled Event → FP&A Agent

- Regular variance analysis between planned and actuals for the quarter

2. FP&A Agent → Vector DB Service

- Pulls embeddings for prior plan scenarios, actual results, drivers

3. Vector DB Service → Knowledge Graph (via MCP)

- Asks for relationships, metadata, anomalies by entity (cost centers, revenue streams, macro factors)

4. Knowledge Graph → SHACL Engine

- Validates if all variance explanations are policy-compliant and supported by source data

5. SHACL Engine → Knowledge Graph / FP&A Agent

- Flags cases needing human review (e.g., unexplained >2% deviation from plan)

6. FP&A Agent → User (or Automated Notification)

- Delivers root cause breakdown, supporting data, and action suggestions

7. Governance Service

- Updates audit log with analysis provenance, validation outcomes

FP&A Sequence Diagram 4: Workflow Integration/Orchestration (Tool/Process)

1. External System (e.g., ERP, BI Tool) → MCP Broker

- Requests trigger of workflow or agent action (e.g., new data load, report refresh)

2. MCP Broker → Tool Registry

- Verifies available microservices/tools and required interfaces

3. MCP Broker Coordinates:

- Triggers ETL Microservice (data ingest to KG)
- Launches FP&A Agent action/service
- If needed, calls out to external data sources/tools

4. All integration points pass through audit, policy validation, and governance review, with the flow mirrored in logs and compliance reports

These sequence flows, if mapped visually, would use UML sequence diagram notation or BPMN, showing clear service boundaries, async interactions, policy enforcement points, MCP middleware routing, agent-vector-KG loops, and audit/service governance. Each microservice and process in the FP&A sub-domain is orchestrated securely and transparently, ensuring true standards-based, SHACL-policy compliant, and agentic integration at enterprise scale

FP&A Use Cases

This section breaks down FP&A sequence diagrams for three core use cases—Close, Forecast, and Variance Analysis—illustrating how domain microservices, knowledge graph (KG) services with SHACL, MCP middleware, agentic AI/vector DBs, and governance audit services interact in each scenario.

FP&A Use Case: Period-End Close

Sequence Flow:

1. User/Controller → FP&A Close Service
   - Initiates close cycle (monthly/quarterly/year-end).
2. FP&A Close Service → Data Integration Microservice
   - Ingests finalized transactions, journals, and adjustments from ERP/accounting tools.
3. Data Integration Microservice → Domain Knowledge Graph (RDF via MCP)
   - Loads data into graph, linking all entries, entities, and processes to FP&A domain models.
4. Knowledge Graph → SHACL Validation Service
   - Runs compliance checks for entity completeness, policy adherence (e.g., cutoff dates, approval steps).
5. SHACL Validation Service → FP&A Close Service
   - Flags anomalies or missing statements (e.g., missed expense/report, late entries).
6. FP&A Close Service → Controller/User
   - Reports issues for actionable resolutions, tracks status.
7. FP&A Close Service → Governance/Audit Service
   - Logs all actions, validation checks, and memos.
8. FP&A Close Service → MCP Broker
   - Triggers reporting and compliance tools, downstream processes (external audit prep, archiving, improvement review).

FP&A Use Case: Forecast Process

Sequence Flow:

1. Analyst/FP&A User → Forecast Agent (UI/API)
   - Requests new forecasts or scenario models (e.g., quarterly revenue/expense forecasts).
2. Forecast Agent → Vector DB (FP&A-aligned)
   - Searches historic trends, embeddings for key drivers and prior forecasts.
3. Vector DB → Knowledge Graph Service (via MCP)
   - Pulls authoritative assumptions and policy-controlled limits (org headcounts, cost ratios, macro factors).
4. Knowledge Graph → SHACL Validation Engine

- Validates forecast assumptions, checks policy constraints (e.g., no expense growth outside policy, business rule adherence).

5. SHACL Validation Engine → Forecast Agent
   - Communicates pass/fail, constraint violations.
6. Forecast Agent → MCP Broker
   - Initiates forecast computation, scenario iteration, automated reporting.
7. Forecast Microservices → Forecast Agent → User
   - Returns model outputs, visualizations, and action prompts.
8. Governance/Audit Service
   - Records data lineage, validations, human overrides, compliance outcomes.

FP&A Use Case: Variance Analysis

Sequence Flow:

1. Scheduled Event/User → Variance Analysis Agent
   - Launches analysis (actuals vs. planned results).
2. Variance Analysis Agent → Vector DB Service
   - Gathers embeddings for historical results, actuals, planned scenarios.
3. Vector DB → Knowledge Graph Service (via MCP)
   - Pulls entity relationships, commentary drivers, metadata (product, cost center, business unit).
4. Knowledge Graph → SHACL Engine
   - Validates explanations against supported data and business policies (e.g., deviation thresholds, documentation).
5. SHACL/Knowledge Graph → Variance Analysis Agent
   - Returns flagged cases for review, ties explanations to policy artifacts.
6. Variance Analysis Agent → User (FP&A/Controller)
   - Sends variance dashboard, drill-downs, and actionable insights (trend flags, anomaly causes).
7. Governance/Audit Service
   - Logs analysis steps, variances, and resulting actions for compliance and continuous improvement.

Each use case sequence ensures auditable, standards-driven, and policy-compliant orchestration across microservices, knowledge graphs, agentic AI modules, and middleware within FP&A. The touchpoints with SHACL policy validation and MCP broker integration are central to achieving enterprise-grade FP&A automation and regulatory control.

MCP and Microservices Integration

To design a standards-driven integration topology that unifies knowledge graphs (RDF), AI agents (with domain-specific vector databases), systems, and tools using microservices and Model Context Protocol (MCP), consider the following layered and modular architecture:

Core Integration Principles

- Microservices Orchestration: Each functional capability—domain graph access, agent query interface, workflow control, policy validation—is encapsulated as a separate microservice. These interact via secure APIs and message brokers, which can scale and be managed independently.
- Knowledge Graphs by Domain: Each business or operational domain has its own RDF-based knowledge graph, modeled using core ontologies plus domain taxonomies. Process-centric graph design ensures the graph reflects real-world entities, flows, and dependencies in that domain.
- Policy Compliance via SHACL: SHACL shapes are used within each graph to declare and enforce constraints (validation, access, workflow logic, etc.), ensuring that policy compliance and business logic are embedded and traceable within domain knowledge.
- Agentic Layer (Vector DBs, LLMs): AI-enabled agents leverage domain-specific graph schemas for knowledge retrieval, but execute reasoning and semantic search over dedicated vector databases, tuned to the same vocabularies and ontologies as the underlying knowledge graphs.
- MCP as Universal Integration Tier: MCP servers act as a standards-based middleware, exposing tools, data, and workflow endpoints as callable resources for AI agents, simplifying cross-system integration and providing a policy-aware interface.

Topology: Reference Architecture

Layer	Components/Responsibilities
Presentation	End-user UIs, API gateways, agent frontends.
Agentic/Augmentation	AI agents, RAG chains, vector DBs (with schemas aligned to RDF KGs), microservices for semantic search, dialogue, workflow actions.
MCP Integration	MCP servers with connectors for KG, workflow engines, file systems, external APIs. Tool/resource registries define callable interfaces and policy contracts.
Knowledge Graph	Modular RDF graphs by domain (process-centric), SHACL validation engine(s), ontology management and alignment, ETL into graph stores.
Policy/Control	Central registry of SHACL shapes, policy dashboard, reporting/audit logs for graph and system compliance via SHACL validation, metadata linkage to workflows/tools.
Systems/Tools	Core enterprise systems, event/message buses, process automation platforms, source systems feeding graphs or being automated by agents.

Microservices & MCP-Enabled Flows

- Domain Graph Service: API for graph CRUD, SPARQL/GraphQL endpoint, SHACL validation as a service.

- Policy Validation Service: Asserts SHACL shapes, enforces on data, processes, and triggers event hooks on violations for remediation.
- Vector DB/Agent Service: Provides vector search & context retrieval, aligns embeddings/taxonomy config to graph schema to ensure semantic consistency.
- MCP Broker: Manages registration of tools, resources, and workflow steps; exposes MCP-compliant endpoints to both agents and legacy systems. Facilitates secure, auditable interactions across domains and systems.
- Governance & Audit Service: Tracks integration operations, policy compliance, and graph changes; links validation outcomes and provenance to data and process artifacts.

Standards and Policy Anchoring

- SHACL as Policy Anchor: SHACL shapes serve as structured policy artifacts—both validating RDF data and surfacing policy compliance across the ecosystem, from datasets to workflows to agent actions.
- Ontology Registry: Ensures both knowledge graphs and vector DBs use a consistent, governed set of vocabularies and schemas, preferably with a published versioning scheme and change process.
- MCP for Secure, Standardized Operations: With MCP, agents always access data and trigger tools/workflows via well-defined, standard contracts—supporting security, logging, and rollback.

Example Flow

1. A user or system event triggers an agent (e.g., via UI or integration bus).
2. The agent consults the vector database for context, using embeddings developed from domain-specific KGs.
3. When authoritative or controlled knowledge is required, the agent queries the RDF KG via the MCP endpoint, with SHACL validation ensuring only policy-compliant data/actions are reachable.
4. The MCP server routes requests to the relevant microservices: KG access, policy validation, workflow triggers, or external tools/systems.
5. All transactions and validations are logged by the governance service, linking outcomes to SHACL policy artifacts and enterprise audit records.

This model is modular, standards-based, auditable, and built for process- and policy-driven control over enterprise knowledge, workflows, and agentic AI enablement.

APPENDIX B

Conversational AI

A design and reference architecture for conversational AI that lets subject-matter experts talk to their data: components, layers, phased delivery plan, KPIs, and the context engineering module.

B.1 Conversational AI Design
B.2 Conversational AI Architecture

Conversational AI Design

Detailed Design: Conversational AI for SMEs to Talk to Their Data

Overview

A conversational AI platform tailored for business SMEs enables natural language querying, exploration, and insights generation from their operational and analytical data using a combination of prompt engineering and context engineering to optimize relevance, accuracy, and usability.

Core Components

1. Data Integration Layer
- Connectors to various SME data sources: CRM, ERP, sales, marketing, finance systems, databases, spreadsheets.
    - Preprocessing pipelines convert raw data to structured semantic formats and embed relevant metadata for retrieval.
2. Semantic Knowledge Layer (Knowledge Engineering)
- Develop a domain-specific ontology and knowledge graph representing key business concepts, entities, and relationships.
    - Annotate data sources for semantic retrieval and disambiguation, supporting natural language understanding for SME-specific terminology.
3. Retrieval and Vector Database
- Store embeddings of data segments, documents, and past conversations to retrieve contextually relevant information dynamically.
    - Supports semantic search to surface the most pertinent data elements in response to queries.
4. Context Engineering Module
- Orchestrates the dynamic assembly of context merging: retrieved semantic data, conversation history, SME role/profile, system instructions, and compliance metadata.
    - Filters and ranks contextual inputs to optimize prompt size and relevance while enforcing policy constraints.
5. Prompt Engineering Module
- Template library creating structured, modular prompts customized to common SME intents (reports, insights, alerts, explanations).
    - Constructs and adapts prompts based on user input and enriched context to guide LLMs effectively.
6. Conversational AI Engine (LLM)
- Uses a tuned large language model (OpenAI GPT, Claude, or similar) to interpret user inputs, generate responses, and maintain conversational flow.
    - Supports multi-turn dialogue, clarifications, and follow-ups leveraging contextual memory.
7. Dialogue Management and Orchestration

- Manages conversation state, handles user intents, manages escalation paths (e.g., handoff to human agents), and tracks task progress.
    - Integration with external tools for actions like report generation, alert setup, or data exports.

8. User Interface

- Multi-channel support: web or mobile apps, messaging platforms (Slack, MS Teams), voice assistants.
    - Intuitive interface including text input, voice, dashboards for resuming or exploring results.

9. Monitoring and Analytics

- Tracks usage metrics, response relevancy, error/hallucination rates, user feedback.
    - Continuous learning pipeline refines prompts, context rules, and LLM fine-tuning.

Workflow Example

- User Query: "Show me last quarter's sales trends by region."
- Context Engineering: Retrieves relevant sales data, historical conversation, recent queries, and SME role.
- Prompt Engineering: Creates tailored prompt with system instructions and data summary.
- LLM Response: Generates a clear, data-backed narrative with charts or key metrics.
- Follow-Up: User asks, "Which product categories drove growth?"—system recalls previous context and builds new prompt.

Implementation Considerations

- Security & Governance: Embed semantic governance policies ensuring data access compliance and privacy.
- Domain Adaptation: Invest in SME-specific ontology and prompt tuning.
- Cost & Performance: Balance context size and retrieval latency; use caching and incremental context updates.
- User Training: Provide onboarding and contextual help to increase adoption.
- Iterative Improvement: Use monitoring data to expand FAQ sets, refine knowledge graph, and optimize prompts.

Benefits for SMEs

- Democratizes access to complex data without needing technical expertise.
- Accelerates decision-making with conversational insights and alerts.
- Improves productivity by automating routine queries and report generation.
- Scales conversational intelligence across departments (sales, finance, operations).

This approach combines prompt engineering's role in crafting effective LLM inputs with context engineering's orchestration of dynamic, multi-source information to create a conversational AI that truly lets SMEs "talk to their data" in an intuitive, productive manner.

Conversational AI Architecture

System Architecture for SME Conversational AI

1. User Interface Layer

- Multi-modal interfaces supporting text (chatbots, web, mobile apps), voice assistants, and messaging platforms (Slack, Teams).
- Provides user input capture and display of AI-generated responses.
- Supports conversation history viewing and interaction controls for clarifications or follow-ups.

2. Natural Language Understanding (NLU) Module

- Intent classification to identify user goals from queries.
- Entity extraction to identify domain-specific elements (e.g., products, dates, metrics).
- Language parsing using transformers or transformer-based encoders tailored with SME domain data.

3. Dialogue Management and Orchestration

- Tracks conversation state, manages multi-turn dialogue flows.
- Integrates feedback loops for disambiguation, error correction, and context updates.
- Applies business rules and defines fallback strategies or human handoff.

4. Context Engineering Module

- Dynamically assembles context by combining: user profile, conversation history, retrieved semantic data, business role, and compliance metadata.
- Manages prompt size, relevance filtering, and context versioning.
- Uses context templates and policy-driven context governance to tailor the information sent to the LLM.

5. Prompt Engineering Module

- Maintains a library of prompt templates customized for typical SME requests (financial reports, sales insights, customer queries).
- Adapts and sequences prompts based on real-time context and workflow state.
- Fine-tuned prompt tuning and examples for accuracy and clarity.

6. Semantic Knowledge Layer

- Semantic models including business ontologies and knowledge graphs representing SME domain concepts.
- Supports semantic retrieval, disambiguation, and grounding for LLM inputs.
- Enables precise contextual understanding and reasoning.

7. Retrieval and Vector Search Engine

- Embedding-based retrieval of relevant documents, FAQs, reports, and prior interactions.
- Integrates with knowledge layer to augment context dynamically.

8. Large Language Model (LLM) Integration

- Connects to LLM APIs or on-prem models (OpenAI GPT, Anthropic Claude, etc.).
- Executes the generation of natural language responses based on engineered prompts and context.
- Supports multi-step reasoning, summarization, and explanation.

9. Backend Data and System Connectors

- Interfaces with SME data sources: CRM, ERP, analytics platforms, spreadsheets, and databases.
- Preprocesses and pipelines data into semantic formats.
- Ensures real-time or batch data synchronization.

10. Monitoring, Analytics, and Feedback Loop

- Tracks user engagement, response accuracy, error rates, and latency.
- Collects user feedback for conversational improvements.
- Enables iterative updates to prompt libraries, context rules, and knowledge schemas.

Architectural Flow

1. User submits query via UI.
2. NLU module processes intent and entities.
3. Dialogue manager maintains conversation state.
4. Context engineering module assembles enriched context from retrieval and semantic knowledge.
5. Prompt engineering crafts the LLM input prompt with context and templates.
6. LLM generates a response.
7. Response delivered to user with ongoing monitoring.
8. Feedback loop for continuous adaptation.

This architecture supports scalable, accurate, and domain-specific conversational AI tailored for SMEs to "talk to their data" by tightly integrating prompt and context engineering with semantic knowledge and retrieval layers.

The seven phases below are delivery phases for this project. They are separate from the six-stage Semantic Knowledge Lifecycle (SKL) that governs semantic assets once built; see the lifecycle note in Section 2.4.

Phase 1: Planning & Requirements (Week 1–4)

Goals: Define scope, gather requirements, identify key stakeholders.

Milestones:

- Stakeholder interviews completed
- Use case and domain scope defined
- System architecture outline approved

Deliverables: Requirements specification document, high-level architecture diagram.

Phase 2: Design & Foundation Setup (Weeks 5–8)

Goals: Develop data models, ontologies, knowledge bases, and select technology stack.

Milestones:

- Domain ontology and knowledge graph modeled
- Data sources integrated and preprocessed
- Technology stack selected (LLMs, retrieval tools, UI platforms)

Deliverables: Knowledge schema, data pipeline prototypes, architecture design.

Phase 3: Prototype Development (Weeks 9–14)

Goals: Build core modules—NLU, dialogue management, retrieval, prompt templates.

Milestones:

- Basic conversational interface with data connection
- Retrieval engine operational with relevant data
- Prompt templates and context assembly rules created

Deliverables: Working prototype with basic conversation flows.

Phase 4: Context & Prompt Engineering Tuning (Weeks 15–18)

Goals: Refine prompt templates and context assembly for high accuracy and relevance.

Milestones:

- Engineered prompts stored and tested in real dialogues
- Dynamic context assembly using retrieval and knowledge layers
- Initial user testing completed

Deliverables: Fine-tuned prompt library, context engineering rules documentation.

Phase 5: Testing & Iteration (Weeks 19–22)

Goals: Conduct extensive testing, gather feedback, optimize performance.

Milestones:

- Usability testing with SMEs completed
- Response relevance, accuracy, and coherence metrics improved
- Security and compliance checked

Deliverables: Test reports, usability feedback, performance metrics.

Phase 6: Pilot Deployment & Feedback (Weeks 23–26)

Goals: Launch pilot with select SMEs, monitor engagement, and refine.

Milestones:

- Pilot platform live with real SME users
- Usage analytics and feedback collected

- Rapid iteration based on insights

Deliverables: Pilot report, improvement backlog, user training materials.

Phase 7: Full Deployment & Continuous Improvement (Weeks 27+)

Goals: Scale deployment, establish monitoring, and optimize iteratively.

Milestones:

- Full deployment completed
- Monitoring dashboards operational
- Ongoing updates and tuning based on analytics

Deliverables: Final release, ongoing support plan, metrics and reporting system.

Summary Timeline & Milestones

Phase	Duration	Key Milestones
Planning & Requirements	4 weeks	Stakeholder agreement & requirements sign-off
Design & Foundation Setup	4 weeks	Knowledge graphs and architecture setup
Prototype Development	6 weeks	Initial working prototype
Context & Prompt Tuning	4 weeks	Optimized prompts & context assembly
Testing & Iteration	4 weeks	Performance and usability validated
Pilot Deployment	4 weeks	Pilot launched, initial feedback collected
Full Deployment & Scaling	Ongoing	Full rollout, continuous monitoring & improvement

This roadmap ensures systematic development, testing, validation, and scaling, aligned with typical project management practices and best practices for AI deployment.

Measurable KPIs

1. User Adoption Rate
- Percentage of target SMEs actively using the AI system within a defined period post-launch.
  - Indicator of initial acceptance and onboarding success.
2. Query Resolution Rate
- Percentage of user queries successfully answered or fulfilled by the AI without human intervention.
  - Measures effectiveness and reliability of LLM responses.
3. Response Accuracy / Relevance
- LLM outputs rated for correctness and relevance, assessed by automated scoring or manual audits.
  - Reflects quality of context engineering and prompt design.
4. Task Completion Time
- Average time from user query to delivery of actionable insights or results.
  - Measures system efficiency and user satisfaction.

5. User Satisfaction Score (CSAT)
- Survey score collected post-interaction measuring user happiness with AI responses.
    - Qualitative indicator tied to adoption and trust.
6. Hallucination Rate
- Frequency of AI responses containing fabricated or incorrect information.
    - Reflects prompt and context engineering precision.
7. System Uptime and Latency
- Technical metrics for availability and performance ensuring a smooth user experience.
8. Cost Per Interaction
- Economic metric showing average operational cost per AI user interaction.
    - Useful for budgeting and scaling decisions.

Context Engineering Module: Detailed Design

Purpose

The Context Engineering Module dynamically assembles, manages, and optimizes the information environment delivered to the AI model at inference time. It ensures that the large language model (LLM) sees the most relevant, structured, and policy-compliant context to generate accurate, coherent, and reliable responses.

Key Functional Components

1. Context Orchestrator
- Central controller that manages context assembly workflows.
    - Coordinates inputs from multiple sources such as conversation history, retrieval results, user profile, system instructions, and external tools.
    - Enforces token and size constraints to fit within LLM context windows.
2. Retrieval Integrator
- Interfaces with vector search engines and semantic knowledge graphs to fetch domain-relevant documents, data segments, or FAQ entries.
    - Ranks and filters retrieved content based on relevance scores and operational policies.
    - Supports multi-modal sources (structured data, unstructured text, API outputs).
3. Memory Manager
- Tracks short-term conversation history and long-term user preferences or organizational memories.
    - Prunes or summarizes prior context items to maintain relevance and avoid context overflow.
    - Handles persistent session state and personalized context tailoring.
4. Policy and Compliance Filter
- Applies governance rules and compliance constraints on what data and instructions are included or excluded.
    - Ensures sensitive or regulated information is handled securely and audit trails are maintained.

    - Integrates with semantic governance systems to enforce organizational standards.
5. Prompt Template Engine
- Hosts libraries of reusable prompt templates with placeholders.
    - Dynamically instantiates templates with assembled context pieces.
    - Supports conditional template branching to handle multi-turn dialogue scenarios and diverse user intents.
6. Context Formatter
- Structures and formats the final context bundle into LLM-compatible input.
    - Adds explicit context delimiters, metadata tags, and schema definitions to improve LLM parsing and output consistency.
    - Optimizes token usage by compressing and summarizing content where possible.
7. Feedback & Adaptation Loop
- Monitors LLM output quality, user feedback, and interaction logs.
    - Adjusts retrieval, ranking, and prompt selection strategies based on feedback.
    - Supports active learning pipelines for continuous context engineering improvement.

1. Input Collection: User query, session state, user profile, and operational metadata are received.
2. Context Assembly: Retrieval Integrator fetches relevant knowledge; Memory Manager adds conversational and personalized history.
3. Policy Enforcement: Policy Filter removes or modifies context as required for compliance.
4. Prompt Instantiation: Prompt Template Engine combines and formats context into a structured prompt.
5. Output Formatting: Context Formatter finalizes prompt and sends it to the LLM.
6. Response Generation: LLM produces output, logged for monitoring.
7. Feedback Analysis: Feedback Loop evaluates response and tunes context assembly rules.

Design Considerations

- Modularity: Each functional component is modular, enabling independent updates and scalability.
- Latency Optimization: Context assembly optimized for low-latency applications.
- Security: Sensitive data handling, encryption, and audit logs embedded.
- Extensibility: Supports integration of new data sources, tools, and policy engines.

Technologies & Tools

- Vector databases (e.g., Pinecone, Milvus)
- Knowledge graph frameworks (e.g., GraphDB, Neo4j)
- LLM APIs (OpenAI GPT, Anthropic Claude)
- Workflow orchestrators (e.g., Apache Airflow, Temporal)
- Schema/ontology editors (e.g., Protégé)
- Secure data storage and audit logging systems

This Context Engineering Module design delivers a systematic, scalable, and secure approach to providing AI models with precisely curated, governed, and contextually enriched inputs—enabling sophisticated conversational AI that truly understands and adapts to SME business data and workflows

The proposed module architecture layers and data flow for the Context Engineering Module are as follows.

Module Architecture Layers

1. Input Layer
- Receives inputs from user queries, session state, user profile.
    - Interfaces with retrieval systems and external tools for supplemental data.
2. Context Assembly Layer
- Retrieves relevant external knowledge using vector search and semantic graphs.
    - Combines conversation history, user metadata, and retrieved data.
    - Applies ranking and filtering for relevance.
3. Policy & Compliance Layer
- Enforces semantic governance and compliance policies.
    - Filters or redacts sensitive data.
    - Ensures context complies with organizational rules.
4. Prompt Template Layer
- Applies structured prompt templates dynamically filled with assembled context.
    - Handles multi-turn flow with branching and conditional prompts.
5. Formatter & Optimizer Layer
- Formats final prompt, manages token limits.
    - Summarizes or compresses context to fit model restrictions.
6. Feedback Loop Layer
- Collects output quality metrics, user feedback.
    - Feeds adaptation signals to retrieval, filtering, and templates.

Data Flow Overview

1. User Query Received
- Input Layer captures query and session info.
2. Relevant Context Retrieved
- Context Assembly Layer queries vector DBs and knowledge graphs for relevant docs.
    - Merges retrieved info with conversational memory.
3. Context Filtering
- Policy Layer applies filters and compliance checks on assembled context.
4. Prompt Construction
- Prompt Template Layer inserts filtered context into templates.
5. Prompt Formatting

- Formatter optimizes prompt structure and content length.

6. LLM Interaction
   - Prepared prompt sent to LLM for response.
7. Monitoring & Feedback
   - Outputs delivered to user, monitored, and logged.

Data Flow Diagram (Conceptual Level 1)

- External Inputs: User Query, Metadata, Retrieval Systems
- Processes: Input Processing → Retrieval & Assembly → Filtering & Compliance → Prompt Engineering → Formatting → LLM Invocation → Output Delivery → Feedback Collection
- Data Stores: Conversation Memory, Semantic Knowledge Graphs, Prompt Template Repository, Policy Rules Engine
- Data Flows: Queries & Responses, Context Data, Compliance Flags, Feedback Metrics

Summary

This layered architecture combined with clear data flow enables scalable, flexible, and governed context engineering. It ensures dynamic assembling of relevant, compliant, and optimized inputs, maximizing the conversational AI's reliability, relevance, and responsiveness.

This design aligns with industry best practices for complex AI workflows and knowledge-driven conversational systems.

APPENDIX C

Simulation and Causal Reasoning

Emerging techniques that extend context engineering: simulation-driven context with the Simulator, Reflector, and Curator pattern, and causal reasoning over ontologies with CausalGraph2LLM.

C.1 Simulation for Context Engineering
C.2 Causal Reasoning with CausalGraph2LLM

Simulation for Context Engineering

Simulations can generate dynamic context for language models. A simulation acts as a world model: it represents the current state of an environment and how it changes in response to actions, so a model can reason about possible futures instead of relying on static prompts or retrieval. This section summarizes how simulation fits into context engineering, which kinds of simulation are most useful, and how to measure their effect.

Simulator, Reflector, and Curator

A common pattern separates the work into three layers that form a feedback loop:

- **Simulator:** an environment model (discrete-event, continuous, or hybrid) that produces states, outcomes, and scenario results through an API that other components can query.
- **Reflector:** monitors simulator output alongside interaction history, detecting state changes, anomalies, and insights relevant to the agent's goals.
- **Curator:** filters, ranks, and assembles simulator output and reflections into structured context, enforcing policy and compliance, managing memory, and maintaining prompt templates.

In operation the Simulator runs scenarios, the Reflector interprets the results, and the Curator composes enriched context for the agent, whose decisions in turn steer the next simulation. Modular components, event-driven orchestration, continuous evaluation, and policy encoded in the Curator keep the loop trustworthy.

How simulation produces context

- **State and transitions:** environment state is encoded in structured form, and the model predicts next states from current states and actions.
- **Guided rollouts:** multi-step simulated trajectories provide both training data and evolving context.
- **Action history:** prior actions, observations, and simulated states maintain situational awareness across turns.
- **Hybrid transitions:** deterministic rules combined with generative prediction balance validity and diversity.

Simulation types that produce useful context

Type	What it contributes
Quality-diversity	Archives of diverse, high-performing solutions that models can combine through in-context learning
Agent-based	Realistic interactions among autonomous agents in social, physical, or cyber settings
Hybrid rule-based and generative	Novel but plausible futures that remain logically consistent
Scenario and state	Step-wise environment states that support multi-turn reasoning and planning

Measuring the effect

Metric	What it shows
Answer relevancy	Whether responses address the task given the simulated context
Faithfulness	Whether outputs are consistent with the simulated facts
Context relevancy	Whether the simulation contributes useful, concise information
Task completion rate	How often multi-step tasks succeed with simulated context
Hallucination rate	How often outputs contain fabricated information
Robustness	How stable outputs are when the simulated context varies slightly
Throughput and latency	Whether simulation can run within real-time constraints

How Simulation Connects to the Core Pipeline

- **Mapping quality assurance (Section 2.3, steps 6–7):** replay representative queries against a simulated copy of the knowledge graph with the proposed mappings applied, and promote them only if answer relevancy and faithfulness improve.
- **FP&A scenario modeling (Appendix A.11 and A.12):** the Forecast and Variance Analysis agents use scenario simulations as dynamic context, validated by the same SHACL policy gate.
- **Feedback controls (Section 2.1):** simulation results feed the staging partition, never the authoritative vector store, until reviewed.

Causal Reasoning with CausalGraph2LLM

To leverage CausalGraph2LLM with a semantic knowledge layer built with ontologies designed as process-central models for business domains, mapping to metadata for all digital assets, the following approach applies:

Integration Strategy

1. Map Ontologies to Causal Graph Structures: Use the domain ontologies to

   define causal relationships between entities, processes, and data assets. Since CausalGraph2LLM works with causal graphs (directed acyclic graphs), represent the ontological classes and relationships as nodes and directed edges describing causal influence or dependency.

2. Generate Verbalized or Encoded Causal Queries: Encode the causal graph knowledge extracted from the semantic layer in one of the supported formats (e.g., JSON, adjacency matrix, GraphML) compatible with CausalGraph2LLM. This serves as input for LLMs to interpret causal relationships per the domain models.
3. Use CausalGraph2LLM as an Evaluation and Inference Tool: Run causal reasoning queries on the domain causal graphs through LLMs guided by CausalGraph2LLM tasks, which include parent/child identification, confounder detection, and causal intervention analysis. This tests and generates insights aligned with the process models and metadata mappings.
4. Iterate for Domain-Specific Tuning: Use the feedback loop from causal query results to refine the semantic models, enhancing the expressiveness and accuracy of causal relationships within ontologies. This iterative tuning supports alignment between the formal semantic layer and practical causal grounding.
5. Incorporate Causal Reasoning into Enterprise Workflows: Embed the causal insights obtained via CausalGraph2LLM evaluations into operational decision-making workflows, data governance, risk management, and scenario simulations, all grounded in the semantic knowledge fabric.

Benefits of This Approach

- Enhanced causal interpretability of ontology-driven models through rigorous LLM-based causal reasoning.
- Systematic evaluation of causal assumptions encoded in ontologies against benchmarked tasks.
- Scalable, domain-aligned causal inference enabling robust explanation, intervention, and decision-making.
- Semantic integration ensures causal models are directly linked to metadata and digital assets, enriching context and traceability.

Encoding the semantic ontologies as causal graphs and using CausalGraph2LLM as a benchmark and reasoning engine creates a powerful synergy between formal semantic governance and advanced causal reasoning for enterprise AI applications

How Causal Reasoning Connects to the Core Pipeline

- **Variance analysis (Appendix A.12):** the Variance Analysis agent uses causal queries over the FP&A ontology to rank likely root causes before presenting them to the user.
- **Ontology quality assurance (Section 2.3):** causal checks flag mapped relationships whose implied cause and effect contradicts observed data, routing them to manual review.
- **Status:** like Appendix C.1, this is an optional enhancement planned after MVP1.

APPENDIX D

Statement of Work (MVP1 Prep)

The Statement of Work for semantic services development and the governance framework: scope, deliverables, timeline, team, assumptions, pricing, and acceptance criteria.

D.1 Statement of Work (MVP1 Prep)

Statement of Work (MVP1 Prep)

Statement of Work: Semantic Services Development & Governance Framework

Project Title: Semantic Services Enablement and Governance Implementation

Date: May 20, 2025

Client: KE Team

Prepared For: [Client Organization Name/Stakeholder - Placeholder]

Prepared By: [Service Provider/Resource(s) - Placeholder]

1. Introduction & Background

Terms used in this SOW: KE refers to **Knowledge Engineering**, the discipline and team that build and manage the ontologies, vocabularies, taxonomies, and knowledge graphs described in Parts I and II. Bracketed values in this SOW are placeholders listed in Section 12, Open Items.

This Statement of Work (SOW) outlines the plan for developing and implementing a suite of Semantic Services and establishing a corresponding Operating Model for Semantic Governance. The primary goal is to enhance Knowledge Engineering (KE) by enabling sophisticated data interpretation, mapping, inferencing, and search capabilities. These services will facilitate the ingestion, processing, and outbound utilization of data, with an initial focus on integration with the Marketplace.

This initiative recognizes the need for a collaborative approach between the KE team and the engaged resource(s), particularly in the design phase. The build activities will involve practical, hands-on development to create initial, functional service footprints that the KE team can subsequently extend and scale.

2. Project Goals & Objectives

- **Develop robust Semantic Processing Services:** To accurately map ingested data structures against defined ontologies, dynamically extend these ontologies, and apply inferencing rules.
- **Implement effective Semantic Outbound Services:** To provide advanced search capabilities ("Search as a Service") for various user interactions and integrate seamlessly with the Marketplace.
- **Establish a comprehensive Operating Model for Semantic Governance:** To ensure the consistency, quality, and lifecycle management of all semantic components.
- **Deliver a working set of services and governance model by the end of Q3.**

3. Scope of Work

The scope of this project is divided into three primary workstreams: Semantic Services (Processing), Semantic Services (Outbound), and Semantic Governance Operating Model.

3.1. Develop Semantic Services (Processing)

This workstream focuses on the services required to interpret, map, and enrich data as it enters or is processed within Knowledge Engineering.

Caveat for resource(s):

- Design for each of the following breakouts will require assistance and collaborative effort between the KE team and resource(s).
- Build activities will involve hands-on stubbing out of working services (initial footprints) to get Knowledge Engineering up and running for future extension and scaling.

3.1.1. Mapping Ontologies (Overview) + Inferencing Rules

- **Activity:** Define and document the overarching strategy for mapping diverse data sources to the KE core and domain-specific ontologies.
    - **Activity:** Develop and implement a foundational set of inferencing rules (e.g., subsumption, equivalence, disjointness, custom rules) to enable automated reasoning and knowledge discovery from the mapped data. This includes:
        - Rule identification workshops with KE subject matter experts.
            - Rule formalization using appropriate rule languages (e.g., SWRL, SHACL, or custom rule engines).
            - **Rule technology:** SHACL (including SHACL rules) is the primary validation and rule mechanism, consistent with Parts I and II. SWRL is listed only as an option: it is not executed natively by the GraphDB/RDFLib stack, so any SWRL rules would need a separate OWL reasoner or translation into SHACL rules or GraphDB rulesets. The final choice is confirmed in the Phase 1 design workshops.
            - Initial rule testing and validation against sample datasets.
    - **Deliverable:** Mapping Strategy Document; Initial Inferencing Rule Set (documented and stubbed in the system).

3.1.2. Semantic Services (Extend Vocabularies)

- **3.1.2.1. Core Vocabularies:**
    - **Activity:** Identify, define, and document foundational vocabularies (e.g., based on Dublin Core, SKOS, FOAF, or custom-defined core concepts) essential for broad interoperability across Knowledge Engineering.
        - **Activity:** Develop services/APIs to manage (create, read, update, delete - CRUD) and extend these core vocabularies dynamically as new concepts are encountered during data ingestion or processing.
            - **Deliverable:** Documented Core Vocabularies; Vocabulary Extension Service (stubbed API).
    - **3.1.2.2. Domain-Specific Vocabularies:**
        - **Activity:** Identify, define, and document vocabularies specific to key business domains within Knowledge Engineering (e.g., product, customer, finance, research).
            - **Activity:** Develop services/APIs to manage and extend these domain-specific vocabularies, ensuring alignment with core vocabularies and domain expert input.

- **Deliverable:** Documented Domain-Specific Vocabularies (for selected initial domains); Domain Vocabulary Extension Service (stubbed API).

3.1.3. Semantic Services (Extend Taxonomies)

- **3.1.3.1. Core Taxonomies:**
  - **Activity:** Develop or adopt foundational hierarchical classification systems (taxonomies) that provide a structured way to organize core concepts within Knowledge Engineering.
    - **Activity:** Develop services/APIs for managing and extending these core taxonomies, including establishing relationships (e.g., broader/narrower terms).
      - **Deliverable:** Defined Core Taxonomies; Taxonomy Extension Service (stubbed API).
  - **3.1.3.2. Domain-Specific Taxonomies:**
    - **Activity:** Develop hierarchical classification systems tailored to specific business domains, enabling more granular organization and retrieval of domain-specific information.
      - **Activity:** Develop services/APIs for managing and extending these domain-specific taxonomies, ensuring consistency with core taxonomies and domain requirements.
      - **Deliverable:** Defined Domain-Specific Taxonomies (for selected initial domains); Domain Taxonomy Extension Service (stubbed API).

3.1.4. Semantic Services (Extend Ontologies)

- **3.1.4.1. Core Ontologies:**
  - **Activity:** Define, model, and implement core ontologies that represent fundamental entities, properties, and relationships across Knowledge Engineering (e.g., building upon existing standards like BFO, or developing a custom foundational ontology).
    - **Activity:** Develop services/APIs to manage and dynamically extend these core ontologies, allowing for the addition of new classes, properties, and relationships as Knowledge Engineering evolves.
      - **Deliverable:** Documented Core Ontologies (OWL/RDF or chosen formalism); Ontology Extension Service (stubbed API for core ontologies).
  - **3.1.4.2. Domain-Specific Ontologies:**
    - **Activity:** Define, model, and implement ontologies tailored to specific business domains, capturing the nuanced semantics, entities, and relationships within those domains.
      - **Activity:** Develop services/APIs to manage and dynamically extend these domain-specific ontologies, ensuring alignment with core ontologies and facilitating deeper domain understanding.
      - **Deliverable:** Documented Domain-Specific Ontologies (for selected initial domains); Domain Ontology Extension Service (stubbed API).
  - **3.1.4.3. Domain Mapping Ontologies:**
    - **Activity:** Develop specialized ontologies and associated services focused on mediating and aligning different semantic models.
      - **Sub-Activity (inter-domain mapping):** Create mappings between different domain-specific ontologies within Knowledge Engineering to facilitate cross-domain analysis

and data integration. This includes defining equivalent classes, properties, and relationships.

- **Sub-Activity (intra-domain mapping):** Create mappings within a single domain ontology to reconcile variations in terminology or representation that may arise from different sources or perspectives.
- **Sub-Activity (data asset mapping):** Develop services to map incoming raw data structures (e.g., database schemas, CSV headers, API response fields) to the concepts and properties defined in the KE ontologies. This will involve creating transformation rules or using mapping languages (e.g., R2RML, SPARQL-Generate).
- **Sub-Activity (Graph mapping):** Define and implement strategies for mapping and transforming data between different graph models or representations, if applicable (e.g., property graphs to RDF graphs).
- **Deliverable:** Domain Mapping Ontology Definitions; Mapping Services (stubbed APIs for inter-domain, intra-domain, data asset, and graph mapping).

3.2. Develop Semantic Services (Outbound)

This workstream focuses on services that enable users and other systems (like the Marketplace) to consume and interact with the knowledge managed by Knowledge Engineering.

Caveat for resource(s):

- Design for each of the following breakouts will require assistance and collaborative effort between the KE team and resource(s).
- Build activities will involve hands-on stubbing out of working services (initial footprints) to get Knowledge Engineering up and running for future extension and scaling.

3.2.1. Search as a Service

- **3.2.1.1. Get - Simple Parameters Submission from a UX:**
  - **Activity:** Design and develop an API endpoint that accepts simple keyword-based search queries and structured parameters (e.g., filters, facets) from a user interface.
    - **Activity:** Implement basic query processing, including tokenization and matching against indexed semantic data.
      - **Deliverable:** Simple Search API (stubbed and documented).
  - **3.2.1.2. Get - Phrases Submitted from a UX:**
    - **Activity:** Design and develop an API endpoint that can process natural language phrases, identifying key concepts and relationships for more nuanced search.
      - **Activity:** Incorporate techniques for phrase recognition and potentially basic intent detection.
      - **Deliverable:** Phrase-Based Search API (stubbed and documented).
  - **3.2.1.3. Get - Questions Submitted from a UX (Natural Language Query - NLQ):**
    - **Activity:** Design and develop an API endpoint capable of interpreting natural language questions (e.g., "What are the key products in the X category related to Y?").
      - **Activity:** Implement mechanisms to parse questions, identify entities and intents, and translate them into formal queries against the semantic backend (e.g., SPARQL queries).

        - **Deliverable:** Question-Answering Search API (stubbed and documented).
    - **3.2.1.4. Put - List of Criteria for Building a Relational Query (One Per Table):**
        - **Activity:** Design and develop an API endpoint that accepts a structured list of criteria (e.g., field, operator, value) intended for constructing queries against underlying relational data sources, guided by semantic understanding.
            - **Activity:** This service will act as an intermediary, translating semantic search intent into parameters suitable for relational database queries where appropriate (e.g., for federated search scenarios).
            - **Deliverable:** Relational Query Criteria API (stubbed and documented). *

3.2.1.5. Put - List of Criteria for Prompt to Guide Conversation (Focus & Clarification):

- **Activity:** Design and develop an API endpoint that accepts criteria to generate or refine prompts for conversational AI systems or guided search interactions.
    - **Activity:** This service will leverage semantic context to help focus conversations, suggest clarifying questions, or guide users towards relevant information.
        - **Deliverable:** Conversational Prompt Guidance API (stubbed and documented).
    - **3.2.1.6. Put - Error Management for Handling Invalid Responses or Insufficient Data:**

- **Activity:** Design and develop an API endpoint or mechanism within the search services to manage and communicate errors gracefully.
    - **Activity:** This includes handling scenarios like invalid search queries, no results found, ambiguous queries, or insufficient data to fulfill a request, providing meaningful feedback to the calling application/UX.
        - **Deliverable:** Error Management API/Specification (documented).

3.2.2. Semantic Services (Search Results Handoff)

- **Activity:** Define and implement a standardized format and protocol for handing off search results to consuming applications (e.g., the Marketplace).
    - **Activity:** This includes specifying the structure of result sets, metadata, snippets, links to source data, and any associated semantic annotations or confidence scores.
    - **Activity:** Initially, this may be a manual handoff process for validation, with a plan for automation.
    - **Deliverable:** Search Results Handoff Specification; Initial (manual) Handoff Process Document.

3.2.3. Semantic Services (Search Pre-Processor)

- **Activity:** Design and develop a service that acts as a pre-processor for incoming search queries.
    - **Activity:** This service will perform tasks such as query understanding, disambiguation, expansion (e.g., using synonyms from vocabularies, broader/narrower terms from taxonomies), and entity recognition before the query is passed to the core search engine or graph database.
    - **Deliverable:** Search Pre-Processor Service (stubbed API and functional description).

3.2.4. Extend Search for Marketplace (Meaning)

- **Activity:** Enhance search capabilities specifically for the Marketplace by incorporating deeper semantic understanding.
  - **Activity:** This involves leveraging ontologies and vocabularies to interpret user intent beyond keywords, understanding the meaning behind search terms in the context of Marketplace offerings.
  - **Deliverable:** Enhanced Marketplace Search (Meaning) Design Document; Initial implementation stubs.

3.2.5. Extend Search for Marketplace (Relevancy)

- **Activity:** Improve the relevancy ranking of search results within the Marketplace by utilizing semantic relationships and context.
  - **Activity:** Develop algorithms or configure search platforms to prioritize results based on semantic similarity, user context, and business rules defined within Knowledge Engineering.
  - **Deliverable:** Enhanced Marketplace Search (Relevancy) Design Document; Initial implementation stubs.

3.2.6. Semantic Services (Search Persona Relevancy)

- **Activity:** Design and develop mechanisms to tailor search results based on user personas or roles.
  - **Activity:** This involves defining user personas, associating them with specific interests or information needs (potentially captured semantically), and adjusting search relevancy accordingly.
  - **Deliverable:** Persona-Based Relevancy Design Document; Service stubs for persona integration.

3.2.7. Semantic Services (Search Contextual Relevancy)

- **Activity:** Design and develop services to enhance search relevancy by considering the broader context of the user's session, task, or historical interactions.
  - **Activity:** This could involve leveraging session data, project context, or other contextual cues to refine search results and provide more pertinent information.
  - **Deliverable:** Contextual Relevancy Design Document; Service stubs for context integration.

3.3. Establish Operating Model for Semantic Governance

This workstream focuses on defining the processes, roles, responsibilities, and tools for managing the lifecycle of semantic assets.

3.3.1. Incorporate KE Standards into Operating Model

- **Activity:** Review existing KE standards (data quality, security, architecture, etc.) and define processes to ensure all semantic components (vocabularies, taxonomies, ontologies, rules) adhere to these standards.
  - **Activity:** Develop guidelines for the creation, modification, and deprecation of semantic assets in line with KE policies.
  - **Deliverable:** Semantic Governance Standards Integration Plan.

3.3.2. Align with Better Practices (Industry & Experience)

- **Activity:** Research and incorporate industry best practices for semantic governance (e.g., DAMA DMBOK, FAIR principles, W3C standards).
  - **Activity:** Leverage the experience of the resources and KE team to tailor these practices to the organization's specific needs and culture.
  - **Deliverable:** Best Practices Alignment Report and Recommendations.

3.3.3. Leverage SKL (Semantic Knowledge Lifecycle) to Capture Lifecycle Artifacts for All Components

- **Activity:** Define how a Semantic Knowledge Lifecycle (SKL) management approach will be implemented. This includes defining stages (e.g., ideation, development, testing, deployment, maintenance, retirement) for all semantic components.
  - **Note:** these six SKL stages are the canonical asset lifecycle used throughout this document (see Section 2.4).
  - **Activity:** Specify the artifacts to be captured at each stage (e.g., requirements, design documents, models, test cases, version history, change logs, approval records).
  - **Activity:** Identify or recommend tools/platforms for managing these SKL artifacts (this may involve existing KE tools or proposing new ones).
  - **Deliverable:** SKL Process Definition Document; Artifact Management Plan.

3.3.4. Define Monitoring & Measurement for Semantic Governance

- **Activity:** Identify key performance indicators (KPIs) and metrics to monitor the health, quality, usage, and impact of semantic assets and services.
  - **Activity:** Examples include: ontology completeness, vocabulary adoption rates, query performance, data mapping accuracy, governance process compliance.
  - **Activity:** Define processes and tools for collecting, analyzing, and reporting on these metrics. This includes defining staging, reporting, and publishing mechanisms for governance information.
  - **Deliverable:** Semantic Governance Monitoring & Measurement Framework (including KPIs, reporting processes).

4. Deliverables Summary

A summary of key deliverables (detailed within Scope of Work):

Processing Services:

- Mapping Strategy Document
- Initial Inferencing Rule Set
- Documented Core & Domain-Specific Vocabularies
- Vocabulary Extension Service (stubbed API)
- Defined Core & Domain-Specific Taxonomies
- Taxonomy Extension Service (stubbed API)
- Documented Core & Domain-Specific Ontologies
- Ontology Extension Service (stubbed API for core & domain)

- Domain Mapping Ontology Definitions
- Mapping Services (stubbed APIs for inter-domain, intra-domain, data asset, graph mapping)

Outbound Services:

- Simple Search API (stubbed)
- Phrase-Based Search API (stubbed)
- Question-Answering Search API (stubbed)
- Relational Query Criteria API (stubbed)
- Conversational Prompt Guidance API (stubbed)
- Error Management API/Specification
- Search Results Handoff Specification & Initial Manual Process Document
- Search Pre-Processor Service (stubbed API & functional description)
- Enhanced Marketplace Search (Meaning) Design Document & Stubs
- Enhanced Marketplace Search (Relevancy) Design Document & Stubs
- Persona-Based Relevancy Design Document & Service Stubs
- Contextual Relevancy Design Document & Service Stubs

Governance Model:

- Semantic Governance Standards Integration Plan
- Best Practices Alignment Report and Recommendations
- SKL Process Definition Document & Artifact Management Plan
- Semantic Governance Monitoring & Measurement Framework

5. Timeline for Deliverable(s)

The overarching goal is to have a working set of services (inbound integration, processing, outbound) and the governance model in place by the **End of Q3 [Year]**. A high-level phased approach is proposed, assuming the availability of 3 resources to work in parallel with cross-pollination:

- **Phase 1: Foundation & Core Design (Month 1-2)**
  - Detailed design workshops for all Processing and Outbound services.
    - Definition of Core Vocabularies, Taxonomies, and Ontologies.
    - Initial draft of Governance Model framework (Standards, SKL, Monitoring).
    - Stubbing out key Processing Service APIs (Mapping, Vocabulary, Taxonomy, Ontology extensions).
    - Stubbing out foundational Outbound Service APIs (Simple Search, Conversational Search - Prompt Iteration, Pre-processor)
- **Phase 2: Development & Initial Integration (Month 2-3)**
  - Development of initial footprints for Processing Services (focus on mapping and inferencing).
    - Development of initial footprints for Outbound Services (focus on Marketplace integration: Meaning & Relevancy, Handoff).
    - Refinement and detailing of the Governance Model.

        - Begin implementation of SKL artifact capture.
- **Phase 3: Testing, Refinement & Deployment (Month 3 - End of Q3)**
    - Testing of Processing and Outbound services with sample data and Marketplace integration points.
        - Refinement of services based on testing feedback.
        - Full documentation of all services and the Governance Model.
        - Deployment of initial working services.
        - Establishment of monitoring and reporting mechanisms for governance.

Priorities (as stated):

1. **Processing Services:** Focus on getting mapping and inferencing up and running early.
2. **Outbound Services:** Sufficient functionality to enable a service call from Marketplace.
3. **Governance Model:** Lifecycle e2e monitoring, staging, reporting, publishing.

6. Project Team & Responsibilities (Illustrative)

Role	Responsibility	Resource
KE Team Lead/Sponsor	Overall project oversight, strategic direction, approvals, internal coordination.	[KE Team]
KE Subject Matter Experts	Provide domain knowledge and input for vocabularies, taxonomies, and ontologies; validate rules and mappings.	[KE Team]
Lead Semantic Architect	Lead design of semantic services and governance, ensure architectural coherence, guide build efforts.	[Resource 1]
Semantic Developer 1	Processing Services development (mapping, inferencing, ontology/vocabulary/taxonomy extensions).	[Resource 2]
Semantic Developer 2	Outbound Services development (Search as a Service, Marketplace integration, relevancy enhancements).	[Resource 3]
Governance Specialist	Develop the Semantic Governance Operating Model (may be shared with or part of the Architect role).	[Resource 1/TBD]

7. Assumptions

- **Resource Availability:** Three (3) 3rd Party Resources will be available as outlined, possessing sufficient independence, leadership, and architecture experience to lead design and build efforts collaboratively with the KE team.
- **KE Team Collaboration:** Active participation and timely input from the KE team (including SMEs) for design, validation, and feedback.
- **Inbound Services:** Functional inbound services will be established and provided, sufficient to ramp up the ingestion of data structures required by the semantic processing services. Clear interface specifications for inbound data will be available.
- **Technology Stack:** Access to and decisions on the necessary technology stack (e.g., ontology editors, triple stores/graph databases, rule engines, API gateways, development environments) will be made in a timely manner. If specific technologies are already in place, access will be provided.

- **Development & Test Environments:** Suitable development and testing environments will be available or provisioned.
- **Access to Data:** Sample data and access to relevant data sources will be provided for development and testing purposes.
- **Marketplace Integration Point:** A clear point of contact and technical specifications for integrating Outbound Services with the Marketplace will be available.

Inbound Dependency: Interface (to be confirmed)

The Processing Services depend on inbound integration services for data (Section 1.4, Inbound Semantic Services). The following interface is assumed for planning and must be confirmed in Phase 1:

Interface	Assumed content	Consumer
Schema and metadata feed	Source schemas, tables, columns, data types, and descriptions, delivered as JSON or via a catalog API (for example, Dataplex)	Processing Services, Section 2.3 ingestion
Change events	Notification when a schema is created or updated, with source, version, and timestamp	SG Scheduler and mapping workflow
Access and credentials	Service accounts and access points for retrieving digital assets (OIDC or API keys)	Inbound microservices (Section 1.4)
Service levels	Delivery frequency, latency, and error-handling and retry contract	SG Operations

8. Desired Outcomes (Reiteration from Outline)

- **Processing Services Support:** Processing Services effectively support the mapping of ingested data structures against KE ontologies.
- **Dynamic Ontology Extension:** Ontologies are dynamically extended (new concepts, relationships) during data ingestion and processing based on defined rules and new information.
- **Published Outbound Services:** Outbound Services are published and support simple, effective integration with the Marketplace.
- **Input Capture:** Ability to capture basic tokens, boolean logic, and prompts as inputs for search and conversational guidance.
- **Query Translation:** Translation of these inputs into domain-specific graph queries.
- **Federated Query Support:** Ability to support federated graph queries (potentially spanning multiple underlying graph stores or data sources).
- **Conditional Logic Handoff:** Ability to hand off conditional logic and filtering requirements for downstream orchestration by other systems.
- **Initial Manual Handoff:** Search results handoff will initially be manual for validation, with a plan for future automation (TBD).

9. Pricing & Payment Schedule

```
[Placeholder: This section would typically detail the costs associated with the resources,
```

payment terms, and schedule. This needs to be determined based on resource rates and project duration.]

Example:

- Resource Rates:
  - Lead Semantic Architect: $[Rate]/hour
    - Semantic Developer: $[Rate]/hour
- Estimated Hours: [Total estimated hours per phase/deliverable]
- Payment Schedule: [e.g., Monthly invoicing based on hours worked, milestone-based payments]

10. Acceptance Criteria

Each deliverable will be considered complete and accepted upon:

- Formal review and approval by the KE Team Lead/Sponsor.
- Demonstration that the deliverable meets the requirements and functionalities outlined in this SOW.
- For service stubs: Successful API call and response according to specification.
- For initial footprints: Demonstration of core functionality with sample data.
- For documentation: Completeness and clarity as agreed upon.

11. SOW Amendments

Any changes or deviations from this SOW must be documented in a formal Change Request, reviewed, and approved by both the Client and the Service Provider before implementation. This SOW should be reviewed with the delivery team, particularly the technical choices, phase-level task breakdowns, and pricing.

12. Open Items

The following values are intentionally left open in this SOW and must be completed before signature:

Item	Location	Owner
Client organization and stakeholder	Header: Prepared For	KE Team Lead/Sponsor
Service provider and named resources	Header: Prepared By; Section 6	Procurement / Service Provider
Target year for end of Q3	Section 5, Timeline	KE Team Lead/Sponsor
Hourly rates and estimated hours	Section 9, Pricing & Payment Schedule	Procurement / Service Provider
Inbound interface confirmation (content, access, service levels)	Inbound Dependency	KE Team Lead/Sponsor

APPENDIX E

Applying Semantic Standards Using RDF Studio

A stepwise guide to applying the Part III naming, definition, label, taxonomy, mapping, and change control standards when modeling in RDF Studio.

E.1 Purpose, Roles, and Setup
E.2 Stepwise Guide
E.3 Checklists and Tool Behaviors

Purpose, Roles, and Setup

This appendix is a stepwise guide to applying the semantic standards in Part III when modeling in **RDF Studio**, the visual modeling tool used for ontologies, taxonomies, and R2RML mappings. It turns the standards into concrete actions: what each file must contain before RDF Studio will accept it, how to check the result, and how work leaves RDF Studio and enters change control.

The guide reflects tool behavior observed during the Knowledge-Driven Analytics pilot. Menu names and screens vary by release; confirm them against the installed version.

Roles

Role	Responsibility in RDF Studio
Knowledge engineer	Builds and extends ontologies; applies naming, URI, and term definition standards
Data steward	Maintains vocabularies and taxonomies; reviews labels and mappings
Subject-matter expert	Confirms definitions, rules, and relationships
AI drafting assistant	Drafts definitions, labels, mappings, and shapes for review; never commits on its own
Change reviewer	Approves exports before they enter source control and the release pipeline

What RDF Studio Is, and Is Not

- **It is the design canvas:** visual ontology and taxonomy design, schema inspection, R2RML mapping design, and read-only SPARQL exploration.
- **It is not the system of record:** source control is. Every artifact is exported, committed with a change record, and released through the pipeline in Section 3.3.
- **It is not a run-time component:** nothing in production depends on RDF Studio being available.

Workspace Setup

1. Agree the prefixes for the work, all under the Company base URL (Section 3.2):

Prefix	Namespace	Use
core:	https://www.insperity.com/ontology/core/	Core ontology
{domain}:	https://www.insperity.com/ontology/{domain}/	Domain ontology, for example payroll:
voc:	https://www.insperity.com/vocabulary/{domain}/	Vocabulary terms and label resources
tax:	https://www.insperity.com/taxonomy/{domain}/	Taxonomy concept schemes
map:	https://www.insperity.com/mapping/{domain}/	Mappings
shape:	https://www.insperity.com/shapes/{domain}/	SHACL shapes
res:	https://www.insperity.com/resource/{domain}/	Instance identifiers: /resource/{domain}/{Class}/{id}

2. Check out the component's files from source control into a working folder; never start from an old export.
3. Configure the AI drafting assistant in the application's settings screen. Settings such as the API key and model temperature must be set in the settings screen itself: importing an exported configuration file blanks the key and resets the temperature.
4. Agree the review rule before drafting: no relationship, domain, or range enters a model without a person confirming it.

Stepwise Guide

Follow the steps in order for a new domain component; for a change to an existing component, start at the step that matches the artifact being changed. Each step names the standard it applies and the check that proves it.

Step 1. Load the Core Ontology

Standard: Design (Section 3.1); BFO alignment (Section 3.10).

- **Ontology header:** the file declares one owl:Ontology with owl:versionIRI, owl:versionInfo, dcterms:title, and dcterms:creator.
- **Declared classes and properties:** the file declares owl:Class, owl:ObjectProperty, and owl:DatatypeProperty explicitly. A file containing only named individuals and annotation properties loads as an empty ontology, because individuals alone do not populate the class tree.
- **Domain and range on every property:** the model editor draws a relationship between two classes only when the property has both rdfs:domain and rdfs:range. Relationships expressed only as OWL restrictions or SHACL shapes are valid but are not drawn; keep them, and document them in the term definition.

Check: the class tree shows the core classes, and the core relationships (for example, Process Step performedBy Party) appear as edges.

Step 2. Build the Domain Ontology

Standard: Naming and URI conventions (Section 3.2); term definition standard (Section 3.7); attributes, properties, and relations (Sections 3.9 and 3.10).

1. Create domain classes as subclasses of core classes; never create a top-level class in a domain.
2. Name classes in UpperCamelCase and properties in lowerCamelCase, in full words with no acronyms; record acronyms as alternate labels.
3. Complete the required term fields for each class: label, definition, is defined by, and at least one named parent.
4. Place each new object property under its family in the taxonomy of relations, and each data property under its family in the taxonomy of attributes, with domain, range, and datatype.
5. Model "includes" with the specific relation it means (Section 3.9), never as a generic includes property.

```
@prefix core:    <https://www.insperity.com/ontology/core/> .
@prefix payroll: <https://www.insperity.com/ontology/payroll/> .

payroll:PayrollRun a owl:Class ;
    rdfs:subClassOf  core:ProcessStep ;
    rdfs:label       "Payroll run"@en ;
    skos:definition  "A process step that calculates and issues pay for one pay period."@en ;
    rdfs:isDefinedBy <https://www.insperity.com/ontology/payroll/> .

payroll:hasPayPeriod a owl:ObjectProperty ;
```

```
    rdfs:domain payroll:PayrollRun ;
    rdfs:range  payroll:PayPeriod .
```

Check: the new classes appear under their core parents and every new property is drawn as an edge.

Step 3. Import Vocabularies and Taxonomies

Standard: Labels and personas (Section 3.8); taxonomy attributes (Section 3.9); ISO 25964.

- **One concept scheme per file:** each taxonomy file declares exactly one skos:ConceptScheme. Files with none, or with two schemes sharing one IRI, are rejected.
- **Every concept in the scheme:** each concept carries skos:inScheme and exactly one skos:prefLabel per language.
- **Top concepts flagged:** first-level concepts are marked with skos:topConceptOf, and the scheme lists them with skos:hasTopConcept.
- **SKOS, not OWL:** an rdfs:subClassOf hierarchy is not accepted as a taxonomy; convert it to skos:broader and skos:narrower.
- **External parents:** where a parent concept belongs to another scheme or namespace, link it with skos:broadMatch rather than skos:broader.

```
@prefix tax: <https://www.insperity.com/taxonomy/payroll/> .

tax:PayrollTopics a skos:ConceptScheme ;
    dcterms:title "Payroll topics"@en ;
    skos:hasTopConcept tax:WagesAndHours .

tax:WagesAndHours a skos:Concept ;
    skos:prefLabel "Wages and hours"@en ;
    skos:inScheme  tax:PayrollTopics ;
    skos:topConceptOf tax:PayrollTopics .

tax:Overtime a skos:Concept ;
    skos:prefLabel "Overtime"@en ;
    skos:altLabel  "OT"@en ;
    skos:inScheme  tax:PayrollTopics ;
    skos:broader   tax:WagesAndHours .
```

Check: the scheme imports without errors, the hierarchy shows every concept under a top concept, and no concept appears without a preferred label.

Step 4. Map Sources with R2RML

Standard: URI conventions (Section 3.2); inbound processing (Sections 1.4 and 2.6); W3C R2RML.

1. Create each mapping in RDF Studio's mapping editor against the source table or view, targeting core and domain classes and properties.
2. Build subject IRIs from stable source keys with the pattern https://www.insperity.com/resource/{domain}/{Class}/{id}; never use row numbers or names that can change.
3. Give every literal its XML Schema datatype and every text value its language tag.

4. Keep mappings as RDF Studio exports. The mapping import accepts only files that RDF Studio wrote itself, each declaring exactly one mapping definition, so mappings are edited in the tool rather than written by hand. Ontology and vocabulary files are loaded through the model and taxonomy imports, not the mapping import.

The underlying triples maps use the standard R2RML vocabulary, for example:

```
<#PayrollRunMap> a rr:TriplesMap ;
    rr:logicalTable [ rr:tableName "PAY_RUN" ] ;
    rr:subjectMap [
        rr:template "https://www.insperity.com/resource/payroll/PayrollRun/{RUN_ID}" ;
        rr:class    payroll:PayrollRun ] ;
    rr:predicateObjectMap [
        rr:predicate payroll:hasCheckDate ;
        rr:objectMap [ rr:column "CHECK_DT" ; rr:datatype xsd:date ] ] .
```

Check: a sample of mapped records resolves to the expected classes, and no subject IRI contains a value that can change.

Step 5. Validate with Read-Only Queries

Standard: Testing and impact assessment (Section 3.3). The SPARQL panel accepts read queries (SELECT and CONSTRUCT) only; changes are made in the editors or in the source files, never with SPARQL UPDATE.

Check	Query pattern
Classes without a definition	SELECT ?c WHERE { ?c a owl:Class . FILTER NOT EXISTS { ?c skos:definition ?d } }
Properties missing domain or range	SELECT ?p WHERE { ?p a owl:ObjectProperty . FILTER NOT EXISTS { ?p rdfs:domain ?d ; rdfs:range ?r } }
Concepts without a preferred label	SELECT ?c WHERE { ?c a skos:Concept . FILTER NOT EXISTS { ?c skos:prefLabel ?l } }
Classes without a named parent	SELECT ?c WHERE { ?c a owl:Class . FILTER NOT EXISTS { ?c rdfs:subClassOf ?p . FILTER(isIRI(?p)) } }
Possible acronyms in preferred labels	SELECT ?s ?l WHERE { ?s skos:prefLabel ?l . FILTER regex(str(?l), "\\b[A-Z]{2,}\\b") }

Expected result for each check: no rows, or only rows with an approved exception recorded in the term's editorial note. SHACL shapes run again in the build pipeline, which is the authoritative gate.

Step 6. Use the AI Drafting Assistant

Standard: Human in the loop (Section 3.3); agent guardrails (Sections 1.5 and 2.6).

- **Good uses:** drafting definitions in the genus-and-differentia form, proposing alternate and persona labels, converting class hierarchies into SKOS, drafting mappings, and drafting SHACL shapes.
- **Always reviewed:** every draft is read by the owner before it is saved; relationships, domains, and ranges are never accepted on the assistant's suggestion alone.

- **Traceable:** the pull request records which items were drafted by the assistant and who approved them.

Step 7. Export, Commit, and Release

Standard: Change records and the release pipeline (Section 3.3).

1. Export the ontology, taxonomy, and mapping files from RDF Studio into the component's working folder, keeping the original file names.
2. Review the differences against source control; confirm nothing was dropped on export (for example, restrictions or label resources the editor does not display).
3. Update owl:versionInfo and the component manifest according to the size of the change.
4. Open a pull request with its change record, linked to the approved feature.
5. The pipeline validates, builds, and promotes the release through test, stage, and production; RDF Studio is not used to load production.

Step 8. Maintain

- **Start every change from source control,** not from a previous export or a copy on a desktop.
- **Deprecate, never delete:** mark retired terms owl:deprecated with dcterms:isReplacedBy before removing them from any view.
- **Re-run the checks** in Step 5 after every import, and keep the queries in the component so everyone uses the same ones.

Checklists and Tool Behaviors

Standards Checklist

Standard	Applied in step	Proof
Base URL and URI patterns (3.2)	Setup, Steps 2 and 4	All IRIs start with the Company base URL and follow the patterns
No acronyms in identifiers or preferred labels (3.2)	Steps 2 and 3	Acronym query returns no rows; acronyms are alternate labels
Term definition fields (3.7)	Step 2	Definition and parent queries return no rows
Alternate and persona labels (3.8)	Step 3	Labels carry language tags and sources
Taxonomy attributes, ISO 25964 (3.9)	Step 3	One scheme per file; every concept in scheme with one preferred label
Taxonomy of relations and attributes (3.10)	Step 2	Every property has a parent family, domain, and range
Process step as anchor (3.1)	Steps 1 and 2	Domain process classes are subclasses of core Process Step
R2RML and stable identifiers	Step 4	Subject IRIs built from stable keys; literals typed
Change control and release (3.3)	Step 7	Pull request with change record; release through the pipeline

Known Tool Behaviors

Behavior	What to do
Model editor reports the ontology is empty	Declare owl:Class and typed properties; individuals and annotations alone do not populate the editor
Relationships are not drawn between classes	Give the property both rdfs:domain and rdfs:range; restrictions and SHACL shapes are not drawn as edges
Taxonomy file is rejected	Declare exactly one skos:ConceptScheme; add skos:inScheme and one skos:prefLabel to every concept; flag top concepts
Mapping import is rejected	Import only mappings exported by RDF Studio; load ontologies and vocabularies through their own imports
SPARQL panel fails with "expected CONSTRUCT"	Use SELECT or CONSTRUCT only; make changes in the editors or source files
Assistant settings revert after a configuration import	Re-enter the API key and temperature in the settings screen after any import

APPENDIX F

Terms & Acronyms

Definitions of the acronyms and specialist terms used throughout this document.

F.1 Terms & Acronyms

APPENDIX F • SECTION F.1

Terms & Acronyms

The table below defines the acronyms and specialist terms used in this document.

Term	Meaning	Description
A2A	Agent-to-Agent protocol	Open protocol for discovery, delegation, and secure messaging between autonomous AI agents.
Agentic fabric	—	The governed set of AI agents that build, maintain, and use the knowledge fabric; agents propose and people approve (Section 1.5).
AI	Artificial Intelligence	Systems that perform tasks requiring human-like perception, reasoning, or language.
Akoma Ntoso	Architecture for Knowledge-Oriented Management of African Normative Texts using Open Standards and Ontologies	OASIS XML standard (LegalDocML) for legislative and legal documents.
API	Application Programming Interface	A defined contract through which software components call one another.
BFO	Basic Formal Ontology	Top-level ontology standardized as ISO/IEC 21838-2; the core ontology aligns to it.
BI	Business Intelligence	Reporting and analytics on business data.
BPMN	Business Process Model and Notation	OMG standard notation for modeling business processes.
CFO	Chief Financial Officer	Executive responsible for the finance function.
Claim check	—	Pattern in which large content is stored separately and the event carries only its location and checksum.
CloudEvents	—	Cloud Native Computing Foundation specification for a common event envelope.
CRM	Customer Relationship Management	Systems that manage customer data and interactions.
CRUD	Create, Read, Update, Delete	The four basic operations on stored data.
DB	Database	An organized store of data; a vector DB stores embeddings.
DCAT	Data Catalog Vocabulary	W3C RDF vocabulary for describing datasets and data catalogs.
DDL	Data Definition Language	SQL statements that define database structures, such as CREATE TABLE.
Dead-letter topic	—	Kafka topic holding events that could not be processed after all allowed recovery actions.
DMN	Decision Model and Notation	OMG standard for decision tables; the authoring format for business rules in the SSOT.
DOL	Distributed Ontology, Model and Specification Language	OMG standard for integrating and mapping heterogeneous ontologies and models.
DPROD	Data Product Ontology	OMG specification for describing data products, their

Term	Meaning	Description
		metadata, and lineage.
DS	Domain-Specific	Applies to a single business domain, as in DS ontologies or DS agents.
ERP	Enterprise Resource Planning	Integrated systems for finance, supply chain, and operations.
ETL	Extract, Transform, Load	Process for moving and reshaping data between systems.
FAIR	Findable, Accessible, Interoperable, Reusable	Principles for making data and metadata reusable.
FIBO	Financial Industry Business Ontology	EDM Council's open ontology of financial concepts.
FP&A	Financial Planning and Analysis	Finance function for budgeting, forecasting, and performance analysis.
FTE	Full-Time Equivalent	Unit of workforce capacity.
GCP	Google Cloud Platform	Google's cloud computing services.
GL	General Ledger	The master record of an organization's financial transactions.
GQL	Graph Query Language	ISO/IEC 39075 query language for property graphs.
IAO	Information Artifact Ontology	BFO-based ontology for information entities such as documents, records, and identifiers; source of the is-about relation.
IFRS	International Financial Reporting Standards	Global accounting standards issued by the IASB.
IRI / URI	Internationalized / Uniform Resource Identifier	Global identifiers for resources; every ontology term has one.
ISO	International Organization for Standardization	International standards body (ISO/IEC and ISO/IEC/IEEE denote joint standards).
JSON / JSON-LD	JavaScript Object Notation / JSON for Linked Data	Lightweight data format; JSON-LD encodes RDF in JSON.
Kafka	Apache Kafka	Distributed event streaming platform used as the inbound event backbone (Section 2.6).
KE	Knowledge Engineering	The discipline and team that build and manage ontologies, vocabularies, taxonomies, and knowledge graphs; owner of the semantic services in Appendix D.
KG	Knowledge Graph	A graph of entities and relationships, governed by ontologies.
KGCL	Knowledge Graph Change Language	Standard language for describing and applying changes to ontologies (see Section A.5).
Knowledge component	—	A domain packaged with its ontology, vocabulary, rules, shapes, mappings, agents, and published interface, released as one versioned unit (Section 1.5).
Knowledge fabric	—	The governed, federated body of ontologies, vocabularies, taxonomies, rules, and mappings connected through the core (Section 1.5).

Term	Meaning	Description
KPI	Key Performance Indicator	A measurable value that tracks performance against goals.
LegalRuleML	Legal Rule Markup Language	OASIS standard for representing legal norms and rules.
LinkML	Linked Data Modeling Language	Schema language used, among other things, to define KGCL's data model.
LKIF	Legal Knowledge Interchange Format	OWL-based format and ontology for legal knowledge (see Section A.7).
LLM	Large Language Model	A neural model trained on large text corpora to understand and generate language.
MBSE	Model-Based Systems Engineering	Engineering approach that uses formal models as the primary artifacts.
MCP	Model Context Protocol	Open protocol that exposes tools, data, and workflows to AI agents through standard interfaces.
MVP1	Minimum Viable Product, release 1	The first deliverable increment of the semantic governance platform.
NIST AI RMF	NIST AI Risk Management Framework	U.S. framework for managing risks of AI systems.
NLQ	Natural Language Query	A question posed in everyday language rather than a query language.
NLU	Natural Language Understanding	Interpreting intent and entities from text.
OAK	Ontology Access Kit	Python library and command-line tool for working with ontologies, including applying KGCL changes.
OASIS	Organization for the Advancement of Structured Information Standards	Open standards consortium (LegalRuleML, Akoma Ntoso).
OBO	Open Biological and Biomedical Ontologies	Community of interoperable open ontologies where KGCL originated.
ODM	Ontology Definition Metamodel	OMG standard metamodel for ontologies (OWL, RDF) in model-driven architecture.
OMG	Object Management Group	Standards consortium behind ODM, DOL, BPMN, UML, and DPROD.
OpenTelemetry	—	Open standard for traces, metrics, and logs used to observe services and agents.
ORSD	Ontology Requirements Specification Document	NeOn Methodology template for ontology requirements (see Section A.4).
OWL	Web Ontology Language	W3C language for defining ontologies with formal semantics (OWL 2 DL is a decidable profile).
Persona	—	A governed description of a group of users who share a role, vocabulary, and information needs (Section 3.8). Not a security role.
PGVector	PostgreSQL vector extension	Adds vector storage and similarity search to PostgreSQL (used in Appendix A.9).
PROV-O	PROV Ontology	W3C vocabulary for recording provenance: who changed what, when, and from which sources.

Term	Meaning	Description
R2RML	RDB to RDF Mapping Language	W3C language for exposing relational databases as virtual RDF graphs.
RAG	Retrieval-Augmented Generation	Grounding LLM answers in retrieved, governed content; RAG 2.0 is the federated design in Section 2.1.
RDF / RDFS	Resource Description Framework / RDF Schema	W3C graph data model of subject-predicate-object triples, and its basic schema vocabulary.
RDF 1.2 / RDF-star	Resource Description Framework 1.2	W3C update to RDF that adds triple terms, so properties can be attached to individual statements.
RDF Studio	—	Visual modeling tool used to design ontologies, taxonomies, and R2RML mappings (Appendix E).
RML	RDF Mapping Language	Declarative language for mapping structured sources (CSV, JSON, XML, databases) to RDF.
RO	OBO Relations Ontology	Library of BFO-aligned relations such as has input, has output, and has active participant.
SemVer	Semantic Versioning	MAJOR.MINOR.PATCH version numbering used for all knowledge artifacts.
SG	Semantic Governance	The discipline and operating model described in this document.
SHACL	Shapes Constraint Language	W3C language for validating RDF data against rules (shapes); used here as a policy gate.
SKL	Semantic Knowledge Lifecycle	Six-stage lifecycle for semantic assets: ideation, development, testing, deployment, maintenance, retirement.
SKOS	Simple Knowledge Organization System	W3C vocabulary for taxonomies, thesauri, and classification schemes.
SKOS-XL	SKOS eXtension for Labels	W3C extension that makes labels resources, so a label can be linked to a persona, source, or date.
SLM	Small (Specialized) Language Model	A smaller, domain-tuned model. In this document, "virtual SLM" means a domain vector partition used with one LLM (Section 2.1).
SLO / SLA	Service Level Objective / Agreement	Target and committed levels of service, monitored in the target operating model.
SME	Subject-Matter Expert	A business user with deep domain knowledge.
SOW	Statement of Work	Contractual description of scope, deliverables, and timeline (Appendix D).
SOX	Sarbanes-Oxley Act	U.S. law governing financial reporting controls (Section 404 covers internal controls).
SPARQL	SPARQL Protocol and RDF Query Language	W3C query language for RDF graphs.
SSOT	Single Source of Truth	One governed, versioned definition of each rule, constraint, logic component, and calculation (Section 3.5).
Star topology	—	Federation pattern in which every domain connects to the core and never directly to another domain.
SWRL	Semantic Web Rule Language	Rule language that extends OWL with if-then rules.

Term	Meaning	Description
TOM	Target Operating Model	How knowledge engineering is run, monitored, and improved (Section 3.6).
TriG	TriG RDF syntax	Text format for RDF datasets with named graphs (.trig files).
UI / UX	User Interface / User Experience	The screens users interact with, and the quality of that interaction.
UML	Unified Modeling Language	OMG standard for software and system models.
W3C	World Wide Web Consortium	Standards body for RDF, OWL, SHACL, SPARQL, SKOS, and DCAT.